
DATA ANALYST

Série 2 — Volume IV

Cahier d'exercices et d'applications

Exercices corrigés · Applications guidées · Projet
Compagnon du volume IV : Visualisation et communication

À utiliser avec le livre : chaque chapitre renvoie ici
pour s'entraîner.

Table des matières

Mode d'emploi	1
Comment utiliser ce cahier	1
Numérotation et niveaux	1
Données et environnement	2
Lancer le code	3
Vérifier son installation	3
Mini-diagnostic de départ	4
Chapitre 1 : Principes de visualisation — exercices et applications	8
Applications	8
Application 1.1 — Une question, une forme (section 1.1)	8
Application 1.2 — Camembert contre barres : mesurer ce que l'œil perd (section 1.1.2)	9
Application 1.3 — Redessiner en cinq gestes (section 1.2)	10
Application 1.4 — Trois titres pour une même courbe, et une annotation (section 1.2.3)	11
Application 1.5 — Petits multiples à échelle commune (section 1.2.5)	12
Application 1.6 — Simuler le daltonisme sur les trois canaux (section 1.3.3)	12
Application 1.7 — Les contrastes de la palette, et un texte qui passe (section 1.3.4)	13
Application 1.8 — Un axe tronqué et une période choisie, mesurés (sections 1.4.1 et 1.4.5)	13
Application 1.9 — Entonnoir des petits effectifs (section 1.4.6)	14
Exercices	14
Exercice 1.1 ★ — Six questions, six formes (section 1.1.1)	14
Exercice 1.2 ★ — Ranger les encodages (section 1.1.2)	14
Exercice 1.3 ★★ — Une cascade du chiffre d'affaires au résultat (section 1.1.4)	15
Exercice 1.4 ★ — Critiquer sans tracer (section 1.2.1)	15
Exercice 1.5 ★★ — Trois titres pour un résultat (section 1.2.3)	15
Exercice 1.6 ★★★ — Échelle commune ou libre ? (section 1.2.5)	15
Exercice 1.7 ★ — Un gris qui passe, un gris qui ne passe pas (section 1.3.4)	15
Exercice 1.8 ★★ — Rouge, vert, bleu pour trois canaux (section 1.3.3)	15
Exercice 1.9 ★ — Un axe coupé sur le budget (section 1.4.1)	15
Exercice 1.10 ★★ — La même fenêtre, un autre cycle (section 1.4.5)	15
Exercice 1.11 ★★ — Taux et effectifs côte à côte (section 1.4.6)	16
Exercice 1.12 ★★ — Corrélation et saison (section 1.4.7)	16
Exercice 1.13 ★★ — Simpson trimestre par trimestre (section 1.4.9)	16
Exercice 1.14 ★★★ — Auditer une page de tableau de bord (section 1.4)	16
Corrigés	16
Corrigé 1.1	16

Corrigé 1.2	16
Corrigé 1.3	17
Corrigé 1.4	17
Corrigé 1.5	18
Corrigé 1.6	18
Corrigé 1.7	19
Corrigé 1.8	19
Corrigé 1.9	19
Corrigé 1.10	20
Corrigé 1.11	20
Corrigé 1.12	20
Corrigé 1.13	21
Corrigé 1.14	21
Chapitre 2 : Tableaux de bord — exercices et applications	22
Applications	22
Application 2.1 — Le modèle en étoile et son contrôle (section 2.1.2)	22
Application 2.2 — Mesures, ratios et totaux (section 2.1.3)	23
Application 2.3 — Une feuille façon Tableau (section 2.2)	24
Application 2.4 — La page de la gérante pour n'importe quelle semaine (section 2.3.3)	26
Application 2.5 — Contraste et lisibilité (section 2.3.4)	27
Application 2.6 — Du DAX à pandas (section 2.4.1)	28
Application 2.7 — Sécurité au niveau des lignes (section 2.4.3)	29
Application 2.8 — Choisir un outil (section 2.5)	30
Exercices	31
Exercice 2.1 ★ — Fait ou dimension ? (section 2.1.2)	31
Exercice 2.2 ★★ — Une dimension qui contient des doublons (section 2.1.2)	31
Exercice 2.3 ★ — Mesure ou colonne calculée ? (section 2.1.3)	31
Exercice 2.4 ★★ — Parts et rangs par canal (section 2.2.2)	31
Exercice 2.5 ★★ — Un niveau de détail : clients par fréquence (section 2.2.2)	31
Exercice 2.6 ★ — La décision de la responsable logistique (section 2.3.1)	31
Exercice 2.7 ★★ — Quelle référence pour quel indicateur ? (section 2.3.3)	31
Exercice 2.8 ★★ — Un dictionnaire qui se teste (section 2.3.5)	32
Exercice 2.9 ★★★ — Une recette qui trouve l'erreur (section 2.3.5)	32
Exercice 2.10 ★★ — À période égale (section 2.4.1)	32
Exercice 2.11 ★★★ — Droits : le cas qui échappe (section 2.4.3)	32
Exercice 2.12 ★★ — Pondérer vos propres critères (section 2.5.2)	32
Corrigés	32
Corrigé 2.1	32
Corrigé 2.2	33
Corrigé 2.3	33
Corrigé 2.4	33
Corrigé 2.5	34
Corrigé 2.6	34
Corrigé 2.7	35
Corrigé 2.8	35
Corrigé 2.9	35
Corrigé 2.10	36
Corrigé 2.11	36
Corrigé 2.12	37

Chapitre 3 : Visualisation avec Python — exercices et applications	38
Applications	38
Application 3.1 — Du graphique par défaut au graphique qui dit quelque chose (sections 3.1.1 et 3.1.2)	38
Application 3.2 — Séries temporelles et petits multiples (sections 3.1.3 à 3.1.5)	40
Application 3.3 — Un thème, un contraste, un format (section 3.1.6)	42
Application 3.4 — seaborn : distribution, carte thermique et intervalle (section 3.1.7)	44
Application 3.5 — Une figure testée, et la même en R (sections 3.1.8 et 3.1.9)	46
Application 3.6 — plotly : une page interactive (section 3.2)	47
Application 3.7 — Streamlit et Dash : tester sans navigateur (section 3.3)	48
Application 3.8 — Shiny : un graphe réactif qui ne recalcule qu'une fois (section 3.3.4)	49
Application 3.9 — Cartes sur un plan fictif (section 3.4)	49
Exercices	50
Exercice 3.1 ★ — Trois couches (section 3.1.1)	50
Exercice 3.2 ★ — Critiquer et refaire (section 3.1.2)	50
Exercice 3.3 ★★ — Un titre qui dit le message, et qui reste vrai (section 3.1.3)	51
Exercice 3.4 ★★ — Échelle commune ou libre ? (section 3.1.5)	51
Exercice 3.5 ★ — Un orange lisible (section 3.1.6)	51
Exercice 3.6 ★★ — Quel format pour quel usage ? (section 3.1.6)	51
Exercice 3.7 ★★ — Écrire la légende d'un intervalle (section 3.1.7)	51
Exercice 3.8 ★★ — Le test qui attrape une erreur invisible (section 3.1.8)	51
Exercice 3.9 ★★ — De matplotlib à ggplot2 (section 3.1.9)	51
Exercice 3.10 ★ — Même figure, deux niveaux (section 3.2.1)	52
Exercice 3.11 ★★ — Une infobulle utile (section 3.2.2)	52
Exercice 3.12 ★★ — Interactif ou statique ? (section 3.2.3)	52
Exercice 3.13 ★★ — Que rejoue Streamlit ? (section 3.3.2)	52
Exercice 3.14 ★★ — Le cas limite (section 3.3.3)	52
Exercice 3.15 ★★ — Normaliser une carte (section 3.4)	52
Exercice 3.16 ★★ — Le piège de la moyenne des taux (section 3.4)	52
Corrigés	53
Corrigé 3.1	53
Corrigé 3.2	53
Corrigé 3.3	53
Corrigé 3.4	55
Corrigé 3.5	55
Corrigé 3.6	56
Corrigé 3.7	56
Corrigé 3.8	56
Corrigé 3.9	57
Corrigé 3.10	58
Corrigé 3.11	58
Corrigé 3.12	59
Corrigé 3.13	60
Corrigé 3.14	60
Corrigé 3.15	60
Corrigé 3.16	61
Chapitre 4 : Storytelling et rédaction de rapports — exercices et applications	62
Applications	62
Application 4.1 — Le storyboard des livraisons (section 4.1)	62
Application 4.2 — Des titres qui concluent (section 4.1)	63

Application 4.3 — Relire un brouillon (section 4.2)	64
Application 4.4 — Un résumé par édition (section 4.2)	64
Application 4.5 — Sensibilité d'une recommandation (section 4.3)	65
Application 4.6 — Un rapport qui se tait, pour le panier moyen (section 4.4)	65
Exercices	66
Exercice 4.1 ★ — Journal ou récit ? (section 4.1.1)	66
Exercice 4.2 ★ — Écrire un titre qui conclut (section 4.1.4)	66
Exercice 4.3 ★★ — Le test du verbe (section 4.1.7)	66
Exercice 4.4 ★★ — La baisse de janvier (section 4.1.7)	66
Exercice 4.5 ★ — Écrire les chiffres (section 4.2.3)	66
Exercice 4.6 ★★ — Réécrire un paragraphe (section 4.2.2)	66
Exercice 4.7 ★★ — Savoir, ne pas savoir, trancher (section 4.2.5)	67
Exercice 4.8 ★★ — Un résumé généré pour les livraisons (section 4.2.7)	67
Exercice 4.9 ★★ — Le résumé en cinq lignes du transporteur C (section 4.3.1)	67
Exercice 4.10 ★★ — Le point d'équilibre de la remise (section 4.3.4)	67
Exercice 4.11 ★★ — Une référence qui tient compte de la croissance (section 4.4.3)	67
Exercice 4.12 ★★ — Un contrôle d'ordre de grandeur (section 4.4.4)	67
Corrigés	67
Corrigé 4.1	67
Corrigé 4.2	68
Corrigé 4.3	68
Corrigé 4.4	68
Corrigé 4.5	69
Corrigé 4.6	69
Corrigé 4.7	69
Corrigé 4.8	70
Corrigé 4.9	70
Corrigé 4.10	70
Corrigé 4.11	71
Corrigé 4.12	71
Pistes des applications	72
Chapitre 5 : Présenter à des interlocuteurs non techniques — exercices et applications	73
Applications	73
Application 5.1 — Un résultat, trois publics (section 5.1.3)	73
Application 5.2 — Points, pour cent et « sur 100 » (sections 5.1.4 et 5.1.5)	74
Application 5.3 — Contraste et daltonisme (section 5.1.6)	75
Application 5.4 — Dire l'incertitude, formuler la recommandation (sections 5.2.3 et 5.2.4)	75
Application 5.5 — Un plan de dix minutes, chronométré (sections 5.2.2 et 5.2.8)	76
Application 5.6 — Une fiche de cadrage (section 5.3.5)	76
Application 5.7 — Compte rendu et désaccord de chiffres (sections 5.2.8 et 5.4.4)	77
Exercices	78
Exercice 5.1 ★ — Qui décide ? (section 5.1.1)	78
Exercice 5.2 ★ — Points ou pour cent ? (section 5.1.5)	78
Exercice 5.3 ★★ — Traduire le jargon (section 5.1.4)	78
Exercice 5.4 ★★ — Le bon texte sur le bon fond (section 5.1.6)	78
Exercice 5.5 ★ — La réponse d'abord (section 5.2.1)	78
Exercice 5.6 ★★ — Des titres qui disent quelque chose (section 5.2.2)	78
Exercice 5.7 ★★ — Une phrase qui dit l'incertitude (section 5.2.3)	78
Exercice 5.8 ★★ — Rendre une recommandation exécutable (section 5.2.4)	79
Exercice 5.9 ★★ — Une erreur découverte en séance (sections 5.2.6 et 5.2.7)	79

Exercice 5.10 ★ — Une question qui se teste (section 5.3.1)	79
Exercice 5.11 ★★ — Entretien et carte des parties prenantes (sections 5.3.2 et 5.3.3)	79
Exercice 5.12 ★★ — Négocier un délai (section 5.4.2)	79
Exercice 5.13 ★★★ — Deux chiffres de conversion (section 5.4.4)	79
Corrigés	79
Corrigé 5.1	80
Corrigé 5.2	80
Corrigé 5.3	80
Corrigé 5.4	80
Corrigé 5.5	81
Corrigé 5.6	81
Corrigé 5.7	81
Corrigé 5.8	82
Corrigé 5.9	82
Corrigé 5.10	82
Corrigé 5.11	83
Corrigé 5.12	83
Corrigé 5.13	83
Pistes des applications	84
Projet du volume et auto-évaluation — exercices et applications	85
Projet du volume	85
P.1 La demande, et sa traduction en question	85
P.2 Étape 1 : les chiffres, calculés une seule fois	86
P.3 Étape 2 : choisir les graphiques	87
P.4 Étape 3 : le tableau de bord d'une page	87
P.5 Étape 4 : le vérifier	87
P.6 Étape 5 : le scénario	89
P.7 Étape 6 : le récit en cinq diapositives	90
P.8 Étape 7 : le message et les questions attendues	90
P.9 Étape 8 : la critique, et les limites	91
P.10 Variante : la même chose pour les promotions	91
Auto-évaluation	92
Principes de visualisation (chapitre 1)	92
Tableaux de bord (chapitre 2)	92
Visualisation avec Python (chapitre 3)	93
Storytelling et rapports (chapitre 4)	93
Présenter à des non-techniciens (chapitre 5)	93
Corrigés des questions	93
Votre grille d'auto-évaluation	96

Mode d'emploi

« On ne sait pas ce qu'un graphique dit tant qu'on n'a pas écrit la phrase. »

Ce cahier est le **compagnon du livre** du volume IV (*Visualisation et communication*). Le livre explique les principes ; le cahier les fait travailler. Il contient, chapitre par chapitre, des **applications guidées**, des **exercices** et leurs **corrigés**, puis le **projet du volume** (un tableau de bord et une courte présentation pour un décideur) et l'**auto-évaluation**. Dans ce volume, la plupart des exercices demandent de **produire** ou de **critiquer** un graphique ou un texte, et leurs corrigés **montrent** la figure ou le texte attendu.

Comment utiliser ce cahier

1. **Lisez d'abord la section du livre.** Chaque section qui a un prolongement ici se termine par une ligne de ce genre : > ■ **Pour s'entraîner.** Cahier, chapitre 1 : application 1.1, exercices 1.1 à 1.4.
2. **Posez les trois questions avant de tracer.** Avant tout code, écrivez : *qui lit ? quelle décision cela éclaire-t-il ? quelle est la seule chose à retenir ?* C'est l'habitude centrale du volume.
3. **Esquissez à la main.** Un croquis sur papier, en trente secondes, évite de se laisser guider par les réglages par défaut d'un outil.
4. **Écrivez le titre comme une phrase.** « Chiffre d'affaires par mois » est une étiquette ; « le Site dépasse la Boutique depuis septembre 2024 » est un message.
5. **Critiquez avant de corriger.** Pour un graphique à améliorer, nommez d'abord ce qui trompe ou ce qui encombre, puis refaites-le.
6. **Essayez avant de regarder le corrigé.** Les énoncés sont regroupés dans la partie *Exercices*, les corrigés dans la partie *Corrigés* du même chapitre.
7. **Vérifiez vos chiffres.** Une figure est un énoncé : un chiffre écrit sur un graphique doit se retrouver par un calcul.

Numérotation et niveaux

Chaque chapitre du cahier correspond au chapitre du livre de même numéro.

Élément	Numérotation	Exemple
Application	Application N.k	Application 1.1 : première application du chapitre 1
Exercice	Exercice N.k	Exercice 1.3 : troisième exercice du chapitre 1

Élément	Numérotation	Exemple
Corrigé	Corrigé N.k	Corrigé 1.3 : correction de l'exercice 1.3

La difficulté des exercices est indiquée par des étoiles :

Niveau	Signification
★	application directe d'une idée ou d'un geste vu dans la section
★★	demande de combiner deux idées, ou un petit raisonnement
★★★	demande de la réflexion, un choix de forme ou une petite expérience

Un exercice cite la section du livre qu'il exerce, par exemple « (section 1.2) ».

Données et environnement

Les données sont celles du livre, dans le dossier `donnees/` : les jeux du volume III (la boutique, ses commandes, ses sessions web, ses comptes, sa logistique, ses ressources humaines) et `villes.csv`. Elles sont toutes **simulées**, avec des graines fixes (script `build/donnees_a4.py`) : vos résultats seront identiques à ceux du livre. Le catalogue complet figure dans la section « Carte du volume, données et environnement ».

Fichier	Contenu	Chapitres
<code>commandes.csv</code> , <code>lignes_commande.csv</code> , <code>clients.csv</code> , <code>produits.csv</code> , <code>retours.csv</code>	la base de la boutique	1 à 4
<code>jours_exploitation.csv</code> , <code>jours_incidents.csv</code>	une ligne par jour (et la même série avec des incidents)	1 à 4
<code>sessions_web.csv</code> , <code>campagnes.csv</code>	trafic du site et dépenses publicitaires	2, 3
<code>budget_reel_2025.csv</code> , <code>compte_resultat_mensuel.csv</code> , <code>bilan_annuel.csv</code> , <code>benchmark_secteur.csv</code>	pilotage et comptes	2, 4, 5
<code>livraisons.csv</code> , <code>reappro_fournisseur.csv</code> , <code>stock_quotidien.csv</code>	logistique	1, 2, 5
<code>employes.csv</code> , <code>employes_annees.csv</code> , <code>departs.csv</code>	ressources humaines (petits effectifs)	5
<code>villes.csv</code>	villes fictives dans un plan fictif	3

Les chiffres sont fictifs. La boutique, ses clients et ses résultats sont inventés. Ne tirez de ces données aucune conclusion sur le monde réel : elles servent à apprendre à **montrer**.

Chaque chapitre du cahier est **autonome** : il commence par ses imports et recharge ses données. Les applications sont dimensionnées pour s'exécuter en **quelques secondes** sur un ordinateur ordinaire, sans réseau.

Lancer le code

Placez-vous dans le dossier du volume (celui qui contient `donnees/` et `figures/`), car les chemins sont relatifs (`donnees/commandes.csv`). Vous pouvez travailler dans un notebook Jupyter, dans un éditeur avec la commande `python`, ou dans RStudio pour R.

Les chemins. Si Python répond `FileNotFoundError`, c'est presque toujours que vous n'êtes pas dans le bon dossier : vérifiez avec `import os; print(os.getcwd())`.

Enregistrer une figure. Pour garder une figure, enregistrez-la avec `fig.savefig("mon-graphique.png", dpi=200, bbox_inches="tight")` plutôt que de compter sur l'affichage de l'éditeur : c'est ce fichier que vous mettrez dans un rapport.

Les outils commerciaux. Les exercices du chapitre 2 qui parlent de Power BI ou de Tableau se font sur **papier ou avec des maquettes** : ces outils ne sont pas exécutés ici. Si vous les avez, la documentation de votre version fait foi.

Vérifier son installation

Trois courts blocs vérifient que les bibliothèques sont installées, que les données se chargent et que l'on sait produire un fichier de figure, en Python, en SQL puis en R.

```
import os, tempfile
from importlib.metadata import version
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
```

```
print({p: version(p) for p in ["pandas", "matplotlib", "seaborn", "plotly"]})
cmd = pd.read_csv("donnees/commandes.csv"); jours = pd.read_csv("donnees/jours_exploitation.csv")
print("commandes :", len(cmd), "| jours d'exploitation :", len(jours))
dossier = tempfile.mkdtemp(dir=os.environ.get("TMPDIR"))
fig, ax = plt.subplots(figsize=(3, 2)); ax.plot([0, 1], [0, 1]); chemin = os.path.join(dossier,
↳ "essai.png"); fig.savefig(chemin); plt.close(fig)
print("figure enregistrée :", os.path.getsize(chemin) > 1000)
```

```
{'pandas': '3.0.6', 'matplotlib': '3.11.2', 'seaborn': '0.13.2', 'plotly': '7.1.0'}
commandes : 36395 | jours d'exploitation : 1096
figure enregistrée : True
```

```
import sqlite3
con = sqlite3.connect(":memory:")
cmd.to_sql("commandes", con, index=False)
print(pd.read_sql("select canal, count(*) as commandes from commandes group by canal order by canal",
↳ con).to_string(index=False))
```

canal	commandes
Boutique	16975
Réseaux	3957
Site	15463

```
library(ggplot2)
cmd <- read.csv(file.path(Sys.getenv("DONNEES"), "commandes.csv"))
f <- tempfile(fileext = ".png")
ggsave(f, ggplot(cmd, aes(canal)) + geom_bar(), width = 3, height = 2)
print(table(cmd$canal)); cat("figure enregistrée :", file.size(f) > 1000, "\n")
```

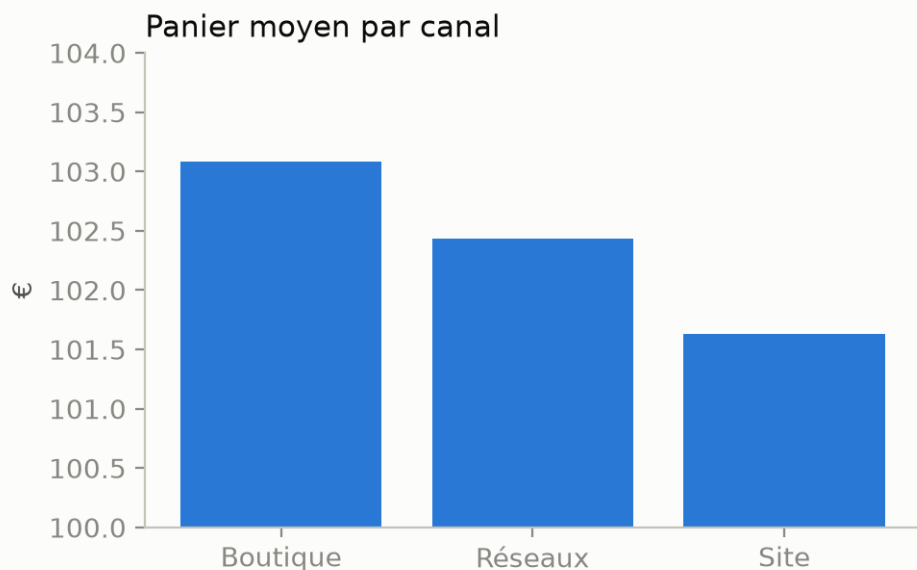
```
Boutique Réseaux Site
16975 3957 15463
figure enregistrée : TRUE
```

Vous devez lire une version pour chaque bibliothèque, **36 395 commandes** et **1 096 jours** d'exploitation, **le même tableau** par canal en SQL et en R, et la confirmation qu'une figure a bien été enregistrée. Si tout concorde, vous êtes prêt.

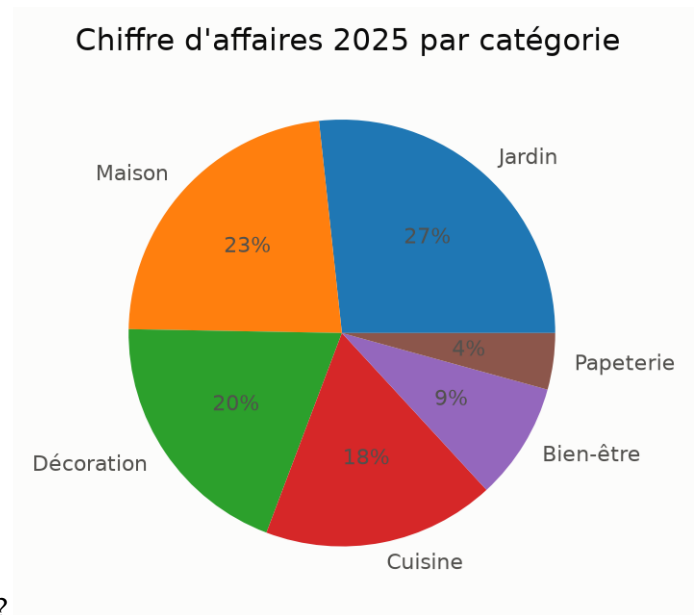
Mini-diagnostic de départ

Huit questions pour vérifier que les réflexes de base de la visualisation sont en place. Répondez par écrit, puis comparez avec les corrigés plus bas ; chaque corrigé indique le chapitre à relire si la réponse vous a échappé.

1. Pour chacune de ces questions de la gérante, quel type de graphique choisissez-vous (ligne, barres, barres horizontales triées, nuage de points, histogramme, camembert) ? (a) « Comment évolue le chiffre d'affaires mois après mois depuis trois ans ? » (b) « Quel canal vend le plus ? » (c) « Quelle part du chiffre d'affaires représente chaque catégorie ? » (d) « Les jours où l'on dépense plus en publicité, vend-on plus ? » (e) « À quoi ressemblent mes paniers : beaucoup de petits, quelques gros ? »
2. Le graphique ci-dessous compare le panier moyen des trois canaux en 2025. Quelle impression donne-t-il ? Quel est le véritable écart relatif entre le canal le plus haut et le plus bas ?

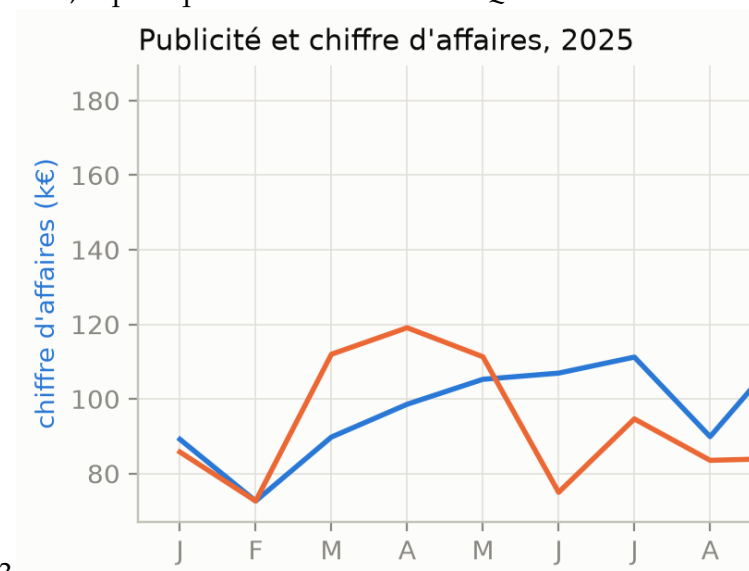


3. Le graphique suivant répartit le chiffre d'affaires 2025 entre six catégories. Pourquoi est-il



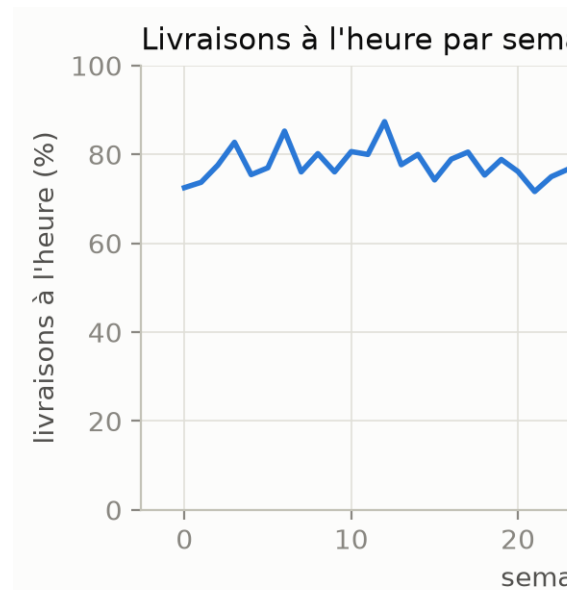
difficile à lire, et que proposez-vous à la place ?

4. Un titre propose : « Depuis septembre 2024, le Site vend plus que la Boutique **chaque mois** ». Est-il exact ? Écrivez le titre correct.
5. Que laisse croire ce graphique à deux axes verticaux, et pourquoi faut-il s'en méfier ? Quelle



corrélation mensuelle donnent ces deux séries ?

6. (a) Un texte gris clair (#999999) est écrit sur fond blanc : le contraste est-il suffisant pour un texte courant (seuil de 4,5 : 1) ? (b) Un tableau de bord signale « bon » en vert (#1baf7a) et « mauvais » en rouge (#e34948) : pourquoi est-ce risqué ?
7. Voici le taux de livraisons à l'heure par semaine en 2025. Écrivez un **titre qui énonce la**



conclusion, puis une **phrase** de commentaire avec les chiffres.

8. Un graphique à barres montre le chiffre d'affaires 2025 des vingt villes. Est-ce lisible ? Quelle part du chiffre d'affaires les cinq premières villes représentent-elles, et comment améliorer le graphique ?

Les **corrigés** s'appuient sur des calculs : le code est ci-dessous.

```
ecart = panier_canal.max() / panier_canal.min() - 1
print("Q2 paniers :", panier_canal.round(2).to_dict(), "| écart relatif maximal :", round(ecart *
↳ 100, 1), "%")
print("Q3 parts (%) :", (ca_cat / ca_cat.sum() * 100).round(1).to_dict())
print("Q4 mois depuis 2024-09 :", len(depuis), "| Site > Boutique :", int((depuis["Site"] >
↳ depuis["Boutique"]).sum()), "| exceptions :", depuis.index[depuis["Site"] <=
↳ depuis["Boutique"]].tolist())
print("Q5 corrélation mensuelle CA / publicité :", round(float(np.corrcoef(ca_mois25.values,
↳ pub_mois.values)[0, 1]), 2))
```

```
Q2 paniers : {'Boutique': 103.08, 'Réseaux': 102.44, 'Site': 101.63} | écart relatif maximal : 1.4 %
Q3 parts (%) : {'Jardin': 26.7, 'Maison': 23.0, 'Décoration': 19.5, 'Cuisine': 17.6, 'Bien-être': 8.9,
↳ 'Papeterie': 4.3}
Q4 mois depuis 2024-09 : 16 | Site > Boutique : 13 | exceptions : ['2024-11', '2025-04', '2025-08']
Q5 corrélation mensuelle CA / publicité : 0.81
```

```
def luminance(h):
    r, g, b = [int(h.lstrip("#")[i:i + 2], 16) / 255 for i in (0, 2, 4)]
    f = lambda c: c / 12.92 if c <= 0.03928 else ((c + 0.055) / 1.055) ** 2.4
    return 0.2126 * f(r) + 0.7152 * f(g) + 0.0722 * f(b)

def contraste(a, b):
    la, lb = luminance(a), luminance(b)
    return (max(la, lb) + 0.05) / (min(la, lb) + 0.05)

print("Q6a contraste de #999999 sur blanc :", round(contraste("#999999", "#ffffff"), 2), ": 1")
print("Q6b contraste entre le vert et le rouge :", round(contraste("#1baf7a", "#e34948"), 2), ": 1")
print("Q7 à l'heure : semaines avant décembre", round(heure.iloc[:4].mean(), 1), "% | quatre
↳ dernières semaines", round(heure.iloc[-4:].mean(), 1), "%")
print("Q8 part des cinq premières villes :", round(ville_ca.head(5).sum() / ville_ca.sum() * 100, 1),
↳ "% | ville n°1 :", ville_ca.index[0], "|", round(ville_ca.iloc[0] / ville_ca.sum() * 100, 1), "%")
```

```
Q6a contraste de #999999 sur blanc : 2.85 : 1
Q6b contraste entre le vert et le rouge : 1.4 : 1
Q7 à l'heure : semaines avant décembre 77.7 % | quatre dernières semaines 45.8 %
Q8 part des cinq premières villes : 52.1 % | ville n°1 : Ville A | 13.9 %
```

1. (a) Une **ligne** : une évolution dans le temps se lit comme une trajectoire (36 points, un axe temporel). (b) Des **barres horizontales triées** : on compare des quantités entre catégories, du plus grand au plus petit. (c) Des **barres** (triées) ; un camembert ne se justifie que pour **deux à quatre** parts très différentes, ce qui n'est pas le cas des six catégories. (d) Un **nuage de points** : on cherche une relation entre deux variables numériques. (e) Un **histogramme** : on regarde la forme d'une distribution (beaucoup de petits paniers, quelques gros). À relire : chapitre 1, section 1.1.
2. L'axe commence à 100 € au lieu de 0 : l'écart entre le canal le plus haut et le plus bas paraît énorme, alors que les paniers valent 103,08 €, 102,44 € et 101,63 € : un écart relatif de **1,4 %** seulement. Pour des barres, l'axe doit commencer à **zéro** (section ④ 1.4). Si l'on veut montrer de petits écarts, on change de forme (points avec intervalle) et l'on dit que l'écart est petit. À relire : chapitre 1, section ④ 1.4.
3. Six parts, six couleurs vives sans lien avec le sens, et des parts voisines (Décoration 19,5 %, Cuisine 17,6 %) qu'on ne sait pas classer à l'œil : l'œil compare mal des **angles** (ici les pourcentages écrits sur les parts font le travail : c'est le texte qui informe, pas la forme). On propose des **barres horizontales triées**, avec la valeur écrite au bout, et une seule couleur (en accentuant éventuellement la catégorie dont on parle). Les trois premières catégories (Jardin, Maison, Décoration) font à elles seules 69 % du chiffre d'affaires : c'est le message, il peut devenir le titre. À relire : chapitre 1, sections 1.1 et 1.2.
4. Non. Depuis septembre 2024 (16 mois, jusqu'à décembre 2025), le Site vend plus que la Boutique **13 mois sur 16** ; les exceptions sont novembre 2024, avril 2025 et août 2025. Titre correct : « **Depuis septembre 2024, le Site dépasse la Boutique 13 mois sur 16** ». « Chaque mois » aurait été faux et se serait vu à la première vérification : un titre énonce une conclusion, **exacte**. À relire : chapitre 1, section 1.2, et chapitre 4, section 4.1.
5. Les deux courbes montent et descendent ensemble, ce qui laisse croire que la publicité **fait** le chiffre d'affaires. Mais les deux axes sont **choisis librement** (on peut les étirer pour que les courbes se superposent), et la corrélation mensuelle (0,81) est portée par la **saison** : novembre et décembre sont des mois forts à la fois en ventes et en dépense. Le volume III a montré qu'à saison égale l'effet de la publicité n'est pas détectable. Mieux vaut deux graphiques l'un sous l'autre sur la même période, ou un nuage de points, et un titre prudent. À relire : chapitre 1, section ④ 1.4.
6. (a) Le contraste de #999999 sur blanc est de **2,85 : 1**, inférieur à 4,5 : 1 : c'est insuffisant pour un texte courant (et même sous le seuil de 3 : 1 des éléments graphiques). (b) Le vert et le rouge ont un contraste mutuel de **1,4 : 1** seulement : ils ont presque la **même luminosité**, donc se confondent pour une personne qui ne distingue pas ces couleurs (environ 8 % des hommes) et à l'impression en noir et blanc. On double toujours la couleur d'un **signe** (symbole, texte « bon » et « mauvais », forme) ou l'on choisit des couleurs de luminosité différente (bleu et orange, par exemple). À relire : chapitre 1, section ④ 1.3.
7. **Titre** : « **Les livraisons à l'heure s'effondrent en décembre** » (ou « ...passent de 78 % à moins de 50 % en décembre »). **Phrase** : « Jusqu'à fin novembre, environ 78 % des livraisons arrivent à l'heure ; sur les quatre dernières semaines de décembre, le taux tombe en moyenne à 46 %, il faut donc renforcer le transport avant les fêtes. » Le titre énonce le **fait**, la phrase donne les **deux chiffres** et le **sens pour l'action**. À relire : chapitre 4, section 4.1.
8. Vingt barres dans un ordre alphabétique sont **illisibles** : on ne voit ni le classement ni l'essentiel. Les cinq premières villes représentent **52,1 %** du chiffre d'affaires (la première, Ville A, près de 14 %). On **trie** les barres, on affiche les **cinq premières** et l'on regroupe le reste dans « autres villes », ou l'on utilise une barre horizontale triée avec les valeurs écrites. Le message devient : « cinq villes font la moitié du chiffre d'affaires ». À relire : chapitre 1, section 1.2.

Chapitre 1 : Principes de visualisation — exercices et applications

Ce chapitre du cahier prolonge le chapitre 1 du livre. Les **applications** sont de petites études guidées sur les données de la boutique : vous les refaites pas à pas, en produisant ou en critiquant des graphiques. Les **exercices** (★ à la main ou en une ligne, ★★ calcul puis code, ★★★ étude plus ouverte) sont **corrigés** à la fin. Presque tous demandent de **produire ou de juger un graphique ou un texte** : relisez toujours votre réponse comme le ferait la gérante, en trois secondes. Les données sont **simulées** ; les figures du cahier portent le préfixe ch01-c-.

Une première cellule charge les bibliothèques et les tables de la boutique ; les suivantes reprennent les noms ainsi définis.

```
import os, sys, itertools
sys.path.insert(0, "build")
import numpy as np, pandas as pd
import matplotlib
import matplotlib.pyplot as plt
import style, utils_ch01 as U

c = 0.charger()                                # les tables de la boutique (simulées)
x, j, liv, sess = c["x"], c["j"], c["liv"], c["sess"]
fr = 0.fr                                       # nombre à la française : 1 325,8
print(len(x), "lignes |", len(j), "jours |", len(liv), "livraisons |", len(sess), "sessions")
```

```
83905 lignes | 1096 jours | 19420 livraisons | 127022 sessions
```

Applications

Application 1.1 — Une question, une forme (section 1.1)

Objectif. Partir d'une question écrite, et en déduire la forme du graphique.

Étape 1 — la question. Écrivez-la en une phrase : « *Quelle part du chiffre d'affaires 2025 vient de chaque canal ?* » C'est une **composition** (des parts d'un total) avec trois catégories : des barres triées conviennent, et un camembert serait ici acceptable (trois parts, l'une proche de la moitié).

Étape 2 — le calcul. On calcule les parts, on les trie.

```
ca = 0.ca_mensuel(2025, "canal").sum().sort_values()    # CA 2025 par canal, en €
part = ca / ca.sum() * 100
```

```
print(part.round(1).to_string())
```

```
canal
Réseaux    11.0
Boutique   42.3
Site       46.6
```

Étape 3 — la figure. Barres horizontales triées, une couleur d'accent sur le canal du message, valeurs écrites, titre qui énonce la conclusion.

```
fig, ax = plt.subplots(figsize=(7, 2.4))
ax.barh(part.index, part.values, color=[0.GRIS, 0.GRIS, 0.BLEU]); ax.grid(False); ax.set_xticks([])
for i, v in enumerate(part.values):
    ax.text(v + 0.8, i, fr(v, 1) + " %", va="center")
ax.set_xlim(0, 56)
ax.set_title(fr"Le site fait {fr(part['Site'], 1)} % du chiffre d'affaires, devant la boutique",
            loc="left", fontweight="bold", fontsize=10)
style.save(fig, "ch01-c-canaux.png")
```

figure : ch01-c-canaux.png

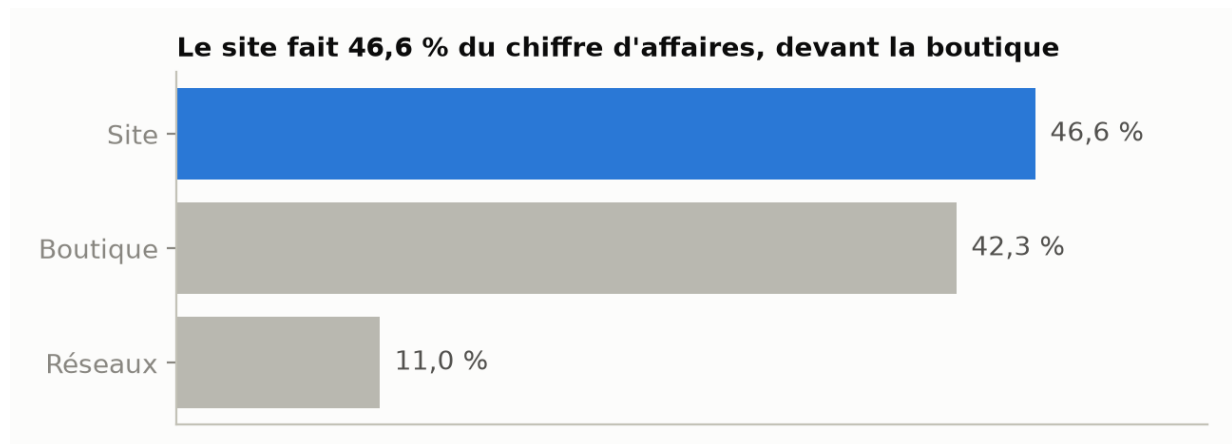


Figure 1: Part du chiffre d'affaires 2025 par canal en barres triées, le site en bleu (46,6 %), la boutique (42,3 %) et les réseaux (11,0 %) en gris. Figure construite avec matplotlib (données simulées).

À vous. Réécrivez la question pour obtenir une **évolution** (« comment la part du site évolue-t-elle de mois en mois ? ») : quelle forme choisissez-vous ? (Réponse : une courbe, parts en ordonnée, mois en abscisse.)

Application 1.2 — Camembert contre barres : mesurer ce que l'œil perd (section 1.1.2)

Objectif. Chiffrer la difficulté de lire des angles voisins.

On calcule, pour les six catégories triées, l'écart entre une part et la suivante : en points de pourcentage, en degrés d'angle (100 % = 360 degrés) et en relatif.

```
cat = 0.ca_categories()
part = cat / cat.sum() * 100
haut, bas = part.iloc[:-1], part.iloc[1:].values
ecart = pd.DataFrame({"points": haut - bas, "degrés": (haut - bas) * 3.6, "relatif %": (haut / bas -
    ↪ 1) * 100})
print(ecart.round(1).to_string())
```

	points	degrés	relatif %
categorie			
Jardin	3.7	13.4	16.2
Maison	3.5	12.5	17.7
Décoration	1.9	6.9	10.9
Cuisine	8.7	31.5	98.5
Bien-être	4.6	16.5	107.5

À vous. Quelle paire de catégories voisines serait la plus difficile à départager sur un camembert ? Sur des barres, que verrait-on ? (Indice : cherchez l'écart le plus faible en degrés.)

Application 1.3 — Redessiner en cinq gestes (section 1.2)

Objectif. Refaire le chemin du brouillon au graphique final sur un nouveau jeu : la part de livraisons en retard par transporteur.

```
ret = liv.groupby("transporteur")["retard"].mean().mul(100).sort_values()      # % de livraisons en
↳ retard
fig, (avant, apres) = plt.subplots(1, 2, figsize=(10, 3.2))
avant.bar(ret.index, ret.values, color=["red", "green", "blue"]); avant.set_title("Graphique 2")
↳ # le brouillon
apres.barh(ret.index, ret.values, color=[0.GRIS, 0.GRIS, 0.ROUGE]); apres.grid(False);
↳ apres.set_xticks([])
for i, v in enumerate(ret.values):
    apres.text(v + 1, i, fr(v, 1) + " %", va="center")
apres.set_xlim(0, 62)
for bord in ("top", "right", "bottom"):
    apres.spines[bord].set_visible(False)
apres.set_title("Un colis sur deux du transporteur C arrive en retard,\nun sur six chez le
↳ transporteur A", loc="left", fontweight="bold", fontsize=10)
apres.text(0, -0.95, "Livraisons arrivées après le délai promis, 2023-2025. Source : livraisons.",
↳ fontsize=8, color=0.MUET)
fig.tight_layout(w_pad=3)
style.save(fig, "ch01-c-redessin.png")
print(ret.round(1).to_string())
```

```
figure : ch01-c-redessin.png
transporteur
Transporteur A    16.0
Transporteur B    26.6
Transporteur C    51.0
```

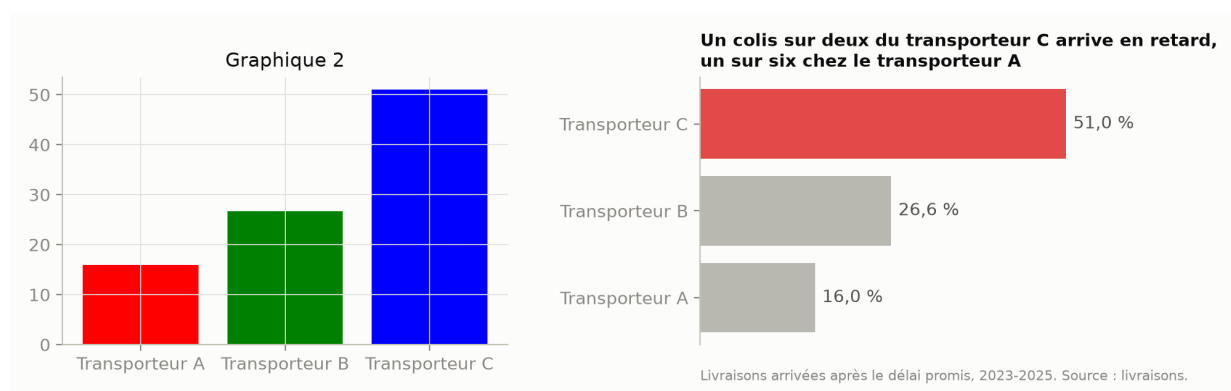


Figure 2: À gauche, le brouillon (trois couleurs vives, aucun titre informatif, axe sans unité) ; à droite, le même graphique redessiné : barres triées horizontales, accent sur le transporteur C, valeurs écrites, titre qui énonce la conclusion et source. Figure construite avec matplotlib (données simulées).

À vous. Listez les cinq gestes appliqués (ranger, une couleur d'accent, étiqueter, titrer, relire) et ce que vous retireriez encore. Vérifiez la phrase du titre : « un sur six » correspond-il à 16,0 % ?

Application 1.4 — Trois titres pour une même courbe, et une annotation (section 1.2.3)

Objectif. Écrire un titre descriptif, un titre informatif, un titre à éviter, en faisant calculer les chiffres.

```
m = 0.ca_mensuel(2025) / 1000 # CA mensuel 2025, en k€
dec24 = 0.ca_mensuel(2024)[12] / 1000
titres = {
    "descriptif": "Chiffre d'affaires mensuel 2025",
    "informatif": f"Le chiffre d'affaires de décembre ({fr(m[12])} k€) dépasse de {fr((m[12] / dec24 - 1) * 100)} % celui de décembre 2024",
    "à éviter": "Les ventes explosent en fin d'année",
}
for nom, t in titres.items():
    print(f"{nom:11s}: {t}")
fig, ax = plt.subplots(figsize=(8, 3.2))
ax.plot(m.index, m.values, color=0.BLEU); ax.set_xticks(range(1, 13));
    ↪ ax.set_xticklabels([k[1].upper() for k in 0.MOIS])
ax.annotate(f"décembre : {fr(m[12])} k€", (12, m[12]), (7.6, m[12] * 0.96),
    ↪ arrowprops=dict(arrowstyle="-", color=0.MUET), fontsize=9)
ax.set_ylim(0, 200); ax.set_ylabel("k€")
ax.set_title(titres["informatif"], loc="left", fontweight="bold", fontsize=9.5)
style.save(fig, "ch01-c-titre.png")
```

```
descriptif : Chiffre d'affaires mensuel 2025
informatif : Le chiffre d'affaires de décembre (184 k€) dépasse de 17 % celui de décembre 2024
à éviter   : Les ventes explosent en fin d'année
figure : ch01-c-titre.png
```

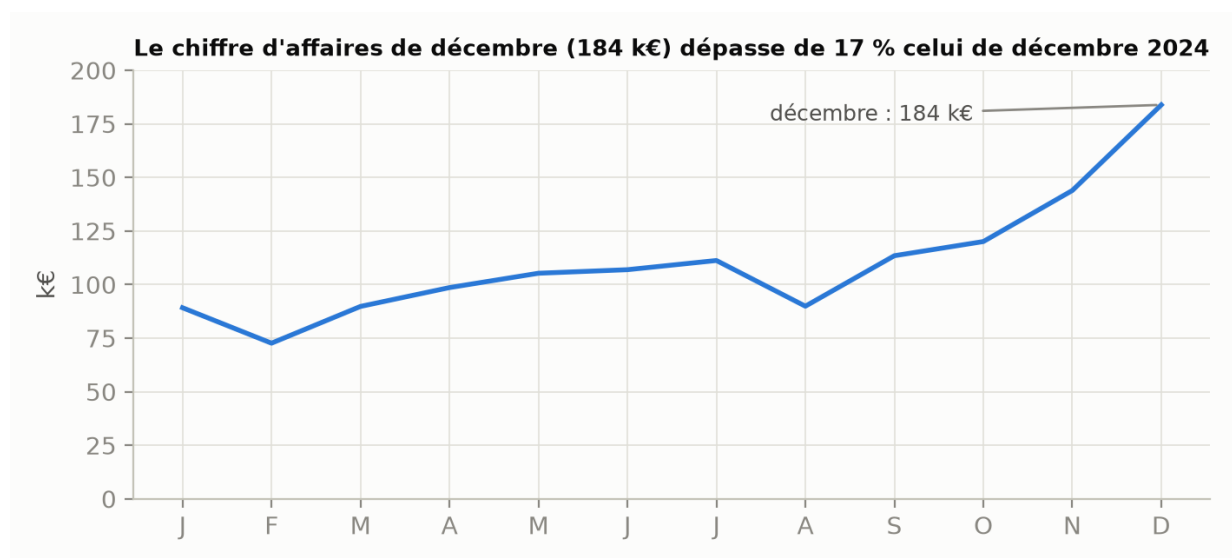


Figure 3: Chiffre d'affaires mensuel 2025 avec le titre informatif et le repère de décembre annoté. Figure construite avec matplotlib (données simulées).

À vous. Pourquoi « explosent » est-il un titre à éviter ? Réécrivez-le avec un chiffre et une comparaison. Ajoutez une seconde annotation (par exemple le creux de février).

Application 1.5 — Petits multiples à échelle commune (section 1.2.5)

Objectif. Montrer la part de retards, mois par mois, pour chaque transporteur, avec les deux autres en gris.

```
liv["mois"] = pd.to_datetime(liv["date_commande"]).dt.month
p = liv.pivot_table(index="mois", columns="transporteur", values="retard", aggfunc="mean") * 100
fig, axs = plt.subplots(1, 3, figsize=(10, 2.8), sharey=True)
for a, t in zip(axs, p.columns):
    for autre in p.columns:
        a.plot(p.index, p[autre], color=0.GRILLE, lw=1)
    a.plot(p.index, p[t], color=0.BLEU, lw=2); a.set_title(t, loc="left", fontsize=9);
    a.set_xticks([1, 6, 12])
axs[0].set_ylabel("% de livraisons en retard")
fig.suptitle("En décembre, les trois transporteurs décrochent, le transporteur C le plus", x=0.01,
            ha="left", fontweight="bold", fontsize=10, y=1.08)
style.save(fig, "ch01-c-multiples.png")
print(p.loc[12].round(1).to_string())
```

```
figure : ch01-c-multiples.png
transporteur
Transporteur A    39.7
Transporteur B    59.2
Transporteur C    85.4
```

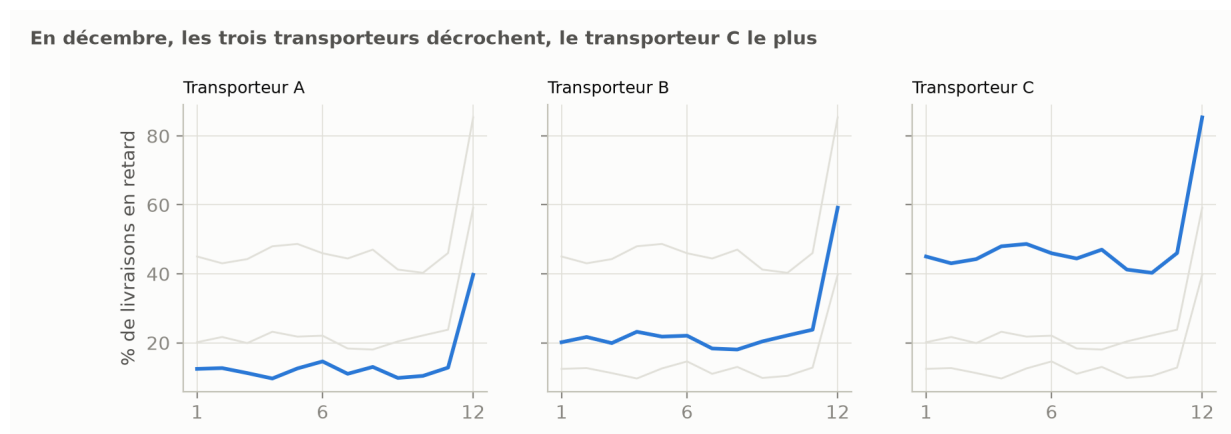


Figure 4: Trois petits graphiques à échelle commune : part des livraisons en retard par mois, pour chacun des trois transporteurs, avec les deux autres en gris. Figure construite avec matplotlib (données simulées).

À vous. Que se passerait-il avec `sharey=False` ? Quel message perdrait-on ? (Voir l'exercice 1.6.)

Application 1.6 — Simuler le daltonisme sur les trois canaux (section 1.3.3)

Objectif. Mesurer, pour la palette (bleu, orange, aqua) du canal Boutique, Site, Réseaux, la paire de couleurs la plus proche selon la vision.

```
cols = {"Boutique": 0.BLEU, "Site": 0.ORANGE, "Réseaux": 0.AQUA}
dist = lambda a, b: float(np.linalg.norm((np.array(a) - np.array(b)) * 255))
vues = ["Vision typique", "Protanopie", "Deutéranopie", "Tritanopie", "Niveaux de gris"]
lignes = []
for nom in vues:
    sim = {k: (matplotlib.colors.to_rgb(v) if nom == "Vision typique" else 0.simuler(v, nom)) for k, v
           in cols.items()}
    d = {f"{a}-{b}": dist(sim[a], sim[b]) for a, b in itertools.combinations(sim, 2)}
    pire = min(d, key=d.get)
    lignes.append((nom, pire, round(d[pire])))
```

```
print(pd.DataFrame(lignes, columns=["vision", "paire la plus proche",
↳ "distance"]).to_string(index=False))
```

	vision	paire la plus proche	distance
Vision typique		Boutique-Réseaux	108
Protanopie		Site-Réseaux	88
Deutéranopie		Site-Réseaux	80
Tritanopie		Boutique-Réseaux	33
Niveaux de gris		Site-Réseaux	20

À vous. Dans quelle vision la paire la plus proche devient-elle la plus problématique ? Que feriez-vous pour que le graphique reste lisible ? (Étiquettes directes, marqueurs différents, traits pleins et pointillés.)

Application 1.7 — Les contrastes de la palette, et un texte qui passe (section 1.3.4)

Objectif. Calculer les contrastes de la palette sur fond blanc, puis **assombrir** une couleur jusqu'à ce qu'elle serve à écrire du texte courant.

```
def assombrir(hexa, cible=4.5, fond="#ffffff"):
    v = np.array(matplotlib.colors.to_rgb(hexa))
    while 0.contraste(tuple(v), fond) < cible:
        v = v * 0.97
    return matplotlib.colors.to_hex(v)
for nom, h in (("bleu", 0.BLEU), ("orange", 0.ORANGE), ("aqua", 0.AQUA), ("violet", 0.VIOLET),
↳ ("rouge", 0.ROUGE)):
    sombre = assombrir(h)
    print(f"{nom:7s} sur blanc : {0.contraste(h, '#ffffff'):4.1f} -> {sombre} : {0.contraste(sombre,
↳ '#ffffff'):4.1f}")
```

bleu	sur blanc :	4.4 -> #2974d0 :	4.7
orange	sur blanc :	3.2 -> #be542a :	4.7
aqua	sur blanc :	2.8 -> #15855d :	4.6
violet	sur blanc :	8.6 -> #4a3aa7 :	8.6
rouge	sur blanc :	4.0 -> #cf4342 :	4.6

À vous. Quelles couleurs de la palette passent **telles quelles** pour du texte courant (4,5 : 1 au moins) ? Quelle couleur assombrie garderiez-vous pour écrire en orange ? Pourquoi ne pas simplement noircir toutes les couleurs ?

Application 1.8 — Un axe tronqué et une période choisie, mesurés (sections 1.4.1 et 1.4.5)

Objectif. Mesurer ce qu'un axe tronqué et une fenêtre de trois mois permettent de dire.

```
l = x.merge(c["ret"][["id_ligne"]].assign(r=1), on="id_ligne", how="left").fillna({"r": 0})
taux = l.groupby("categorie")["r"].mean().mul(100).sort_values()
print(taux.round(1).to_string())
app = (taux - 5) / (taux.min() - 5)
↳ l'axe part de 5 %
print("catégorie la plus haute / la plus basse : axe à 5 % ->", fr(app.max(), 1), "; axe à zéro ->",
↳ fr(taux.max() / taux.min(), 2))
m = 0.ca_mensuel(2025)
var = {k: m[k + 2] / m[k] - 1 for k in range(1, 11)}
↳ mois, à partir du mois k
print("plus forte hausse : à partir du mois", max(var, key=var.get), "->", fr(max(var.values()) * 100,
↳ 1), "%")
print("plus forte baisse : à partir du mois", min(var, key=var.get), "->", fr(min(var.values()) * 100,
↳ 1), "%")
```

```

categorie
Papeterie      5.4
Maison         5.9
Décoration     6.0
Bien-être     6.1
Cuisine        6.1
Jardin         6.2
catégorie la plus haute / la plus basse : axe à 5 % -> 2,7 ; axe à zéro -> 1,13
plus forte hausse : à partir du mois 10 -> 53,1 %
plus forte baisse : à partir du mois 6 -> -15,9 %

```

À vous. Rédigez, pour chaque cas, la phrase **trompeuse** et la phrase **honnête**.

Application 1.9 — Entonnoir des petits effectifs (section 1.4.6)

Objectif. Appliquer le diagramme en entonnoir aux taux de retour de **tous** les canaux et comparer à la situation des seuls réseaux du livre.

```

g = l.groupby("id_produit")["r"].agg(["size", "sum"]).rename(columns={"size": "n", "sum": "r"})
g["taux"] = g["r"] / g["n"]
p0 = g["r"].sum() / g["n"].sum()
e = np.sqrt(p0 * (1 - p0) / g["n"])
for nom, z in (("95 %", 1.96), ("99,8 %", 3.09)):
    print(f"borne à {nom} : {int((g['taux'] > p0 + z * e).sum())} produits au-dessus, {int((g['taux']
    < p0 - z * e).sum())} en dessous")
print("effectifs de", g["n"].min(), "à", g["n"].max(), "lignes ; taux moyen", fr(p0 * 100, 1), "%")

```

```

borne à 95 % : 3 produits au-dessus, 4 en dessous
borne à 99,8 % : 0 produits au-dessus, 0 en dessous
effectifs de 200 à 1628 lignes ; taux moyen 6,0 %

```

À vous. Comparez avec le livre (section 1.4.6) : pourquoi, avec des effectifs de 200 à 1 600 lignes plutôt que de 17 à 174, y a-t-il moins de produits « suspects » ? Que conclure du nombre de produits au-dessus de la borne à 95 % par rapport au nombre attendu par hasard (trois sur cent vingt) ?

Exercices

Exercice 1.1 ★ — Six questions, six formes (section 1.1.1)

Pour chaque question de la gérante, nommez la **famille** de graphique et la **forme** : (a) « Quels produits se vendent le mieux ? » (b) « Les commandes montent-elles depuis un an ? » (c) « Comment se répartissent les montants de commande ? » (d) « La météo joue-t-elle sur les ventes ? » (e) « Où perd-on de l'argent entre la marge brute et le résultat ? » (f) « Combien de visiteurs perd-on entre la visite et le paiement ? »

Exercice 1.2 ★ — Ranger les encodages (section 1.1.2)

- (a) Classez de la plus à la moins précise : la couleur, la longueur, l'aire, la position sur une échelle commune, l'angle. (b) Calculez l'angle d'une part de 8,9 % et celui d'une part de 4,3 % (100 % = 360 degrés), puis leur rapport : le « deux fois plus » se lit-il bien sur un camembert ?

Exercice 1.3 ★★ — Une cascade du chiffre d'affaires au résultat (section 1.1.4)

À partir du compte de résultat 2025, construisez les **étapes** d'une cascade allant du chiffre d'affaires hors taxe au résultat d'exploitation (achats, variation de stock, puis les sept charges), et vérifiez que la somme des étapes redonne le résultat.

Exercice 1.4 ★ — Critiquer sans tracer (section 1.2.1)

Un collègue vous envoie un camembert en 3D à neuf parts de couleurs vives, avec une légende à droite, un fond dégradé, une ombre portée, sans titre, représentant les ventes par ville. Listez **huit défauts**, puis proposez le graphique que vous feriez.

Exercice 1.5 ★★ — Trois titres pour un résultat (section 1.2.3)

Le retard de livraison dépasse 55 % en décembre. (a) Calculez la part de retards en décembre et hors décembre. (b) Écrivez un titre descriptif, un titre informatif, et un titre exagéré. (c) Dites pourquoi le troisième est à proscrire.

Exercice 1.6 ★★★ — Échelle commune ou libre ? (section 1.2.5)

Tracez, pour les trois canaux, le chiffre d'affaires mensuel 2025 en **petits multiples** deux fois : une ligne à échelle commune, une ligne à échelle libre. Écrivez ce que chaque ligne permet de dire, et ce qu'elle cache.

Exercice 1.7 ★ — Un gris qui passe, un gris qui ne passe pas (section 1.3.4)

Un collègue propose du texte en gris #767676 sur blanc, un autre en #777777. Calculez le rapport de contraste de chacun. Lequel est conforme à la recommandation de 4,5 : 1 pour du texte courant ?

Exercice 1.8 ★★ — Rouge, vert, bleu pour trois canaux (section 1.3.3)

On propose le rouge #d62728, le vert #2ca02c et le bleu #1f77b4 pour la boutique, le site et les réseaux. Mesurez, pour les trois visions, la distance entre chaque paire de couleurs. Quelle paire est la plus exposée ? Proposez deux remèdes qui ne changent pas la palette.

Exercice 1.9 ★ — Un axe coupé sur le budget (section 1.4.1)

Le chiffre d'affaires réalisé 2025 (1 325 k€) est comparé au budget (1 279 k€) sur des barres dont l'axe part de 1 250 k€. Calculez la hauteur apparente du réalisé par rapport au budget, et le rapport réel. Rédigez les deux phrases (trompeuse, honnête).

Exercice 1.10 ★★ — La même fenêtre, un autre cycle (section 1.4.5)

On annonce : « les ventes ont progressé de 53 % en trois mois ». Pour **2023, 2024 et 2025**, calculez la variation d'octobre à décembre, et celle de juin à août. Que concluez-vous sur le choix de la fenêtre ?

Exercice 1.11 ★★ — Taux et effectifs côte à côte (section 1.4.6)

Calculez la part de livraisons en retard pour chaque couple (transporteur, mode de livraison), avec l'effectif et la demi-largeur de l'intervalle à 95 %. Peut-on ici classer les cellules sans craindre le hasard ? Pourquoi la situation diffère-t-elle de celle des produits du livre ?

Exercice 1.12 ★★ — Corrélation et saison (section 1.4.7)

Calculez la corrélation mensuelle entre dépense publicitaire et commandes sur les 36 mois, puis **après retrait de la moyenne de chaque mois-calendrier** (on soustrait à chaque valeur la moyenne des trois années pour le même mois). Que devient la corrélation ? Rédigez le titre honnête du nuage.

Exercice 1.13 ★★ — Simpson trimestre par trimestre (section 1.4.9)

Comparez le chiffre d'affaires moyen par jour avec et sans promotion : globalement, puis à **trimestre égal**. Écrivez le message que la gérante doit retenir.

Exercice 1.14 ★★★ — Auditer une page de tableau de bord (section 1.4)

Une page contient : (1) des barres de ventes par catégorie dont l'axe part de 90 % de la plus petite ; (2) deux courbes (publicité, commandes) à deux axes ; (3) un camembert à douze parts ; (4) un taux de retour par produit sans effectif ; (5) l'évolution « depuis septembre » du chiffre d'affaires. Pour chaque élément, remplissez : **qui est trompé, par quoi, avec quelle conséquence, quel correctif**.

Corrigés

Corrigé 1.1

Question	Famille	Forme
(a) produits qui se vendent le mieux	comparer des quantités	barres horizontales triées (les dix premiers)
(b) les commandes montent-elles depuis un an	évolution dans le temps	courbe mensuelle, au moins un cycle complet
(c) répartition des montants	distribution	histogramme (ou boîte à moustaches par canal)
(d) la météo joue-t-elle ?	relation entre deux variables	nuage de points (température, ventes du jour)
(e) où perd-on de l'argent ?	passage d'un total à un autre	cascade
(f) visiteurs perdus entre visite et paiement	conversion étape par étape	barres d'entonnoir

Corrigé 1.2

- (a) Du plus au moins précis : la **position** sur une échelle commune, la **longueur**, l'**angle**, l'**aire**, la **couleur**.

```
parts = pd.Series({"bien-être": 8.9, "papeterie": 4.3})
ang = parts * 3.6
print(ang.round(1).to_string())
print("rapport des angles :", fr(ang["bien-être"] / ang["papeterie"], 2))
```

```
bien-être    32.0
papeterie    15.5
rapport des angles : 2,07
```

- (b) Le rapport est de 2,07 : « deux fois plus ». Les angles (32 et 15,5 degrés) sont **petits**, et la comparaison de deux petits angles est peu précise ; sur des barres, la longueur de l'une est visiblement le double de celle de l'autre, sans calcul.

Corrigé 1.3

```
cr = c["cr"]; d = cr[cr["mois"].str[:4] == "2025"].sum(numeric_only=True) / 1000 # k€
etapes = [("CA hors taxe", d["ca_ht"]), ("Achats", -d["achats"]), ("Variation de stock",
↳ d["variation_stock"])]
etapes += [(nom, -d[k]) for nom, k in (("Personnel", "frais_personnel"), ("Loyers",
↳ "loyers_charges"), ("Marketing", "marketing"), ("Livraison", "livraison"),
↳ ("Frais bancaires", "frais_bancaires"), ("Amortissements",
↳ "amortissements"), ("Autres charges", "autres_charges"))]
cumul = np.cumsum([v for _, v in etapes])
for (nom, v), cu in zip(etapes, cumul):
    print(f"{nom:20s} {v:9.0f} cumul {cu:8.0f}")
print("résultat d'exploitation :", round(d["resultat_exploitation"]), "| somme des étapes :",
↳ round(cumul[-1]))
```

CA hors taxe	1104	cumul	1104
Achats	-685	cumul	419
Variation de stock	-2	cumul	417
Personnel	-141	cumul	276
Loyers	-65	cumul	211
Marketing	-73	cumul	138
Livraison	-32	cumul	107
Frais bancaires	-20	cumul	87
Amortissements	-23	cumul	64
Autres charges	-24	cumul	40
résultat d'exploitation : 40 somme des étapes : 40			

La somme des étapes redonne le résultat : c'est le test qu'une cascade doit toujours passer. (La variation de stock est ajoutée avec son signe : ici elle retire 2 k€. Ce qui compte pour une cascade est que les étapes **somment** exactement au résultat ; si le total diffère, un signe est faux, comme on le voit en inversant celui du stock.)

Corrigé 1.4

Huit défauts : (1) la **3D** déforme les angles ; (2) **neuf parts** : illisible ; (3) **couleurs vives** sans sens ; (4) **légende** à droite qui oblige aux allers-retours ; (5) **fond dégradé** qui réduit le contraste ; (6) **ombre portée** qui n'apporte rien ; (7) **aucun titre**, donc aucun message ; (8) **ni valeurs ni source**. Le graphique à faire : des **barres horizontales triées** des huit premières villes plus « autres villes » en gris, valeurs en pourcentage au bout des barres, un titre du genre « Deux villes font plus d'un quart du chiffre d'affaires » (13,9 % et 12,4 %, soit 26,3 %), une source. (Voir 1.4.8.)

Corrigé 1.5

```
liv["dec"] = pd.to_datetime(liv["date_commande"]).dt.month == 12
t = liv.groupby("dec")["retard"].mean().mul(100)
print("hors décembre :", fr(t[False], 1), "% | décembre :", fr(t[True], 1), "%")
```

hors décembre : 21,6 % | décembre : 55,5 %

- (b) **Descriptif** : « Part des livraisons en retard, décembre et autres mois ». **Informatif** : « En décembre, plus d'un colis sur deux arrive en retard, contre un sur cinq le reste de l'année ». **Exagéré** : « La logistique s'effondre en fin d'année ». (c) Le troisième affirme une **cause** et une **gravité** que le graphique ne montre pas ; le deuxième dit ce que l'on mesure et laisse la gérante juger.

Corrigé 1.6

```
p = 0.ca_mensuel(2025, "canal") / 1000
fig, axs = plt.subplots(2, 3, figsize=(10, 4.4), sharex=True)
for k, nom in enumerate(p.columns):
    for ligne, partage in enumerate((True, False)):
        a = axs[ligne, k]; a.plot(p.index, p[nom], color=0.BLEU, lw=2); a.set_title(nom + (" : échelle
        commune" if partage else " : échelle libre"), loc="left", fontsize=9)
        a.set_ylim(0, p.values.max() * 1.05 if partage else a.set_ylim(0, p[nom].max() * 1.1)
        a.set_xticks([1, 6, 12])
fig.suptitle("Échelle commune : le site pèse le plus ; échelle libre : chaque canal a sa saison",
            x=0.01, ha="left", fontweight="bold", fontsize=10)
fig.tight_layout()
style.save(fig, "ch01-c-echelles.png")
print((p.max() / p.min()).round(1).to_string())
```

figure : ch01-c-echelles.png
 canal
 Boutique 2.2
 Réseaux 3.7
 Site 2.6

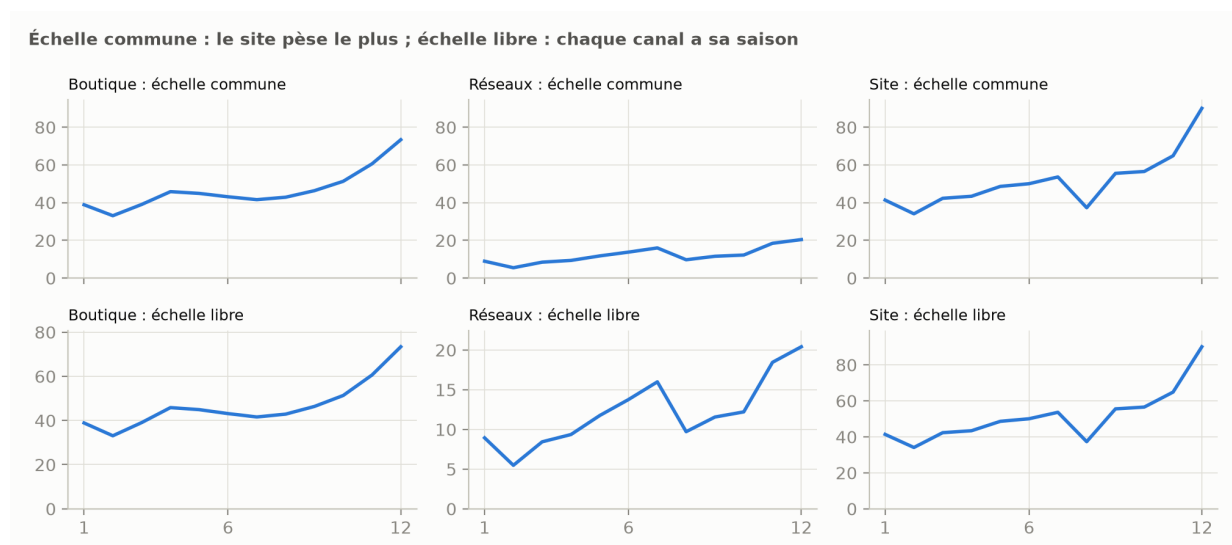


Figure 5: Le chiffre d'affaires mensuel 2025 des trois canaux en petits multiples, en haut à échelle commune, en bas à échelle libre. Figure construite avec matplotlib (données simulées).

Échelle commune (ligne du haut) : on compare les **niveaux** (le site et la boutique dominant,

les réseaux restent petits), mais la forme des réseaux s'aplatit. **Échelle libre** (ligne du bas) : on compare les **formes** (la même saison en fin d'année, la même baisse d'août pour le site), mais on perd le poids relatif des canaux. La sortie donne le rapport maximum/minimum de chaque canal sur l'année : $\times 2,2$ pour la boutique, $\times 3,7$ pour les réseaux, $\times 2,6$ pour le site.

Corrigé 1.7

```
for h in ("#767676", "#777777"):
    print(h, round(0.contraste(h, "#ffffff"), 2))
```

```
#767676 4.54
#777777 4.48
```

Le gris #767676 donne **4,54 : 1**, au-dessus de 4,5 : il est conforme pour du texte courant. Le gris #777777, presque identique à l'œil, donne **4,48 : 1** : il passe sous le seuil. À une marche de gris près, la conclusion change : c'est pourquoi on **calcule** au lieu de juger à l'œil.

Corrigé 1.8

```
cols = {"rouge": "#d62728", "vert": "#2ca02c", "bleu": "#1f77b4"}
dist = lambda a, b: round(float(np.linalg.norm(np.array(a) - np.array(b)) * 255))
for nom in ("Protanopie", "Deutéranopie", "Tritanopie"):
    sim = {k: 0.simuler(v, nom) for k, v in cols.items()}
    print(f"{nom:13s}", {f"{a}-{b}": dist(sim[a], sim[b]) for a, b in itertools.combinations(sim, 2)})
print("vision typique", {f"{a}-{b}": dist(matplotlib.colors.to_rgb(cols[a]),
    ↪ matplotlib.colors.to_rgb(cols[b])) for a, b in itertools.combinations(cols, 2)})
```

```
Protanopie    {'rouge-vert': 89, 'rouge-bleu': 149, 'vert-bleu': 176}
Deutéranopie  {'rouge-vert': 30, 'rouge-bleu': 165, 'vert-bleu': 150}
Tritanopie    {'rouge-vert': 298, 'rouge-bleu': 289, 'vert-bleu': 22}
vision typique {'rouge-vert': 209, 'rouge-bleu': 244, 'vert-bleu': 143}
```

La paire rouge-vert s'effondre en deutéranopie (30, contre 209 en vision typique) et reste faible en protanopie (89) ; en tritanopie, c'est la paire vert-bleu qui devient la plus proche (22). Deux remèdes **sans changer la palette** : (1) **étiqueter directement** les courbes ou les barres par le nom du canal ; (2) **doubler** la couleur par un **marqueur** (cercle, carré, triangle) ou un **style de trait** (plein, pointillé). On peut aussi faire varier la **luminosité** (un vert plus clair, un rouge plus foncé).

Corrigé 1.9

```
reel, budget = 1324.76, 1278.70 # k€ (voir la section 1.3.1)
print("hauteur apparente :", fr((reel - 1250) / (budget - 1250), 1), "| rapport réel :", fr(reel /
    ↪ budget, 3), f"({(reel / budget - 1) * 100:.1f} %)".replace(".", ","))
```

```
hauteur apparente : 2,6 | rapport réel : 1,036 (+3,6 %)
```

Trompeur : « Le réalisé écrase le budget » (barre 2,6 fois plus haute). **Honnête** : « Le chiffre d'affaires dépasse le budget de 3,6 % (1 325 contre 1 279 k€) », avec un axe à zéro.

Corrigé 1.10

```
for an in (2023, 2024, 2025):
    m = 0.ca_mensuel(an)
    print(an, "oct->déc :", fr((m[12] / m[10] - 1) * 100, 1), "% | juin->août :", fr((m[8] / m[6] - 1)
    ↪ * 100, 1), "%")
```

```
2023 oct->déc : 53,0 % | juin->août : -13,8 %
2024 oct->déc : 54,7 % | juin->août : -21,5 %
2025 oct->déc : 53,1 % | juin->août : -15,9 %
```

La hausse d'octobre à décembre (environ 53 %) revient **chaque année**, et la baisse de juin à août aussi (de -14 % à -22 %) : ce sont des **saisons**. Annoncer l'une ou l'autre sans montrer les années précédentes suggère un événement là où il y a un calendrier. (On a vu en 1.4.5 qu'il faut un cycle complet et la comparaison au même mois de l'an dernier.)

Corrigé 1.11

```
t = liv.groupby(["transporteur", "mode_livraison"])["retard"].agg(["size",
    ↪ "mean"]).rename(columns={"size": "n", "mean": "taux"})
t["± 95 %"] = 1.96 * np.sqrt(t["taux"] * (1 - t["taux"]) / t["n"])
print((t.assign(taux=t["taux"] * 100, **{"± 95 %": t["± 95 %"] * 100}).sort_values("taux",
    ↪ ascending=False)).round(1).to_string())
```

		n	taux	± 95 %
transporteur	mode_livraison			
Transporteur C	Point relais	1397	66.9	2.5
	Domicile	2099	42.0	2.1
	Retrait magasin	275	38.9	5.8
Transporteur B	Point relais	2571	38.0	1.9
Transporteur A	Point relais	3342	23.5	1.4
Transporteur B	Domicile	3873	20.2	1.3
	Retrait magasin	471	17.6	3.4
Transporteur A	Domicile	4765	11.4	0.9
	Retrait magasin	627	10.8	2.4

Ici, les neuf cellules ont **plusieurs centaines à plusieurs milliers** de livraisons ; les demi-largeurs vont de 0,9 à 5,8 points, alors que les taux vont de 10,8 % à 66,9 % : le classement est **fiable**, sauf entre rangs voisins (38,9 % et 38,0 %, ou 11,4 % et 10,8 %), que les intervalles ne départagent pas. Pour les produits du livre, les effectifs sont de 17 à 174 lignes et les taux voisins (6,7 % en moyenne) : les écarts entre produits sont du même ordre que les intervalles. La règle est la même ; ce sont les **effectifs** et les **écarts** qui décident.

Corrigé 1.12

```
j2 = j.assign(mois=j["date"].dt.to_period("M"))
mo = j2.groupby("mois").agg(pub=("depense_pub", "sum"), cmd=("nb_commandes", "sum"))
mo["mn"] = mo.index.month
res = mo[["pub", "cmd"]] - mo.groupby("mn")[["pub", "cmd"]].transform("mean") # on retire la
    ↪ moyenne de chaque mois-calendrier
print("corrélation brute :", fr(np.corrcoef(mo["pub"], mo["cmd"])[0, 1], 2), "| après retrait de la
    ↪ saison :", fr(np.corrcoef(res["pub"], res["cmd"])[0, 1], 2))
```

```
corrélation brute : 0,81 | après retrait de la saison : -0,11
```

La corrélation brute (0,81) **disparaît** une fois la saison retirée (elle devient légèrement négative : le résidu est de l'ordre du bruit). Le titre honnête : « Publicité et commandes montent ensemble en fin d'année ; une fois la saison retirée, il ne reste aucune relation visible ».

Corrigé 1.13

```
j3 = j.assign(t=j["date"].dt.quarter)
glob = j3.groupby("promo_active")["chiffre_affaires"].mean()
trim = j3.groupby(["t", "promo_active"])["chiffre_affaires"].mean().unstack()
print("global :", glob.round(0).to_dict(), "->", fr((glob[1] / glob[0] - 1) * 100, 1), "%")
print(((trim[1] / trim[0] - 1) * 100).round(1).to_string())
```

```
global : {0: 3339.0, 1: 3300.0} -> -1,2 %
t
1      3.6
2      5.2
3      6.7
4     11.4
```

Globalement la promotion paraît **faire perdre** 1,2 % ; à **trimestre égal**, elle fait gagner de 3,6 à 11,4 % de chiffre d'affaires par jour. Le message : « La comparaison globale est faussée par le calendrier (les promotions tombent hors de décembre, le meilleur mois) ; à saison égale, un jour de promotion rapporte plus. » (La question de la **marge**, que les remises dégradent, est une autre histoire, traitée au volume III.)

Corrigé 1.14

Élément	Qui est trompé	Par quoi	Conséquence	Correctif
(1) barres à axe coupé	la gérante	la longueur des barres (1.4.1)	écarts de 3 % perçus comme énormes	axe à zéro, valeurs écrites
(2) double axe	la direction	deux échelles choisies (1.4.4)	« la pub fait vendre »	deux graphiques ou un nuage, mention de la saison
(3) camembert à douze parts	le responsable de zone	angles voisins (1.4.8)	classement faux	barres triées, « autres » en gris
(4) taux sans effectif	la responsable des achats	petits effectifs (1.4.6)	produit retiré à tort	effectifs affichés, entonnoir
(5) « depuis septembre »	le financeur	choix des bornes (1.4.5)	pente saisonnière extrapolée	au moins un an, comparaison au même mois

Chapitre 2 : Tableaux de bord — exercices et applications

Ce chapitre du cahier accompagne le chapitre 2 du livre. Il contient **huit applications guidées** (modèle en étoile, mesures, feuille façon Tableau, page paramétrée, contraste, équivalents DAX, sécurité par lignes, choix d'un outil) et **douze exercices** de difficulté croissante (★ réfléchir, ★★ réfléchir puis calculer, ★★★ étude plus ouverte), tous **corrigés** en fin de chapitre. Les données sont **simulées**. Power BI, Tableau, Looker, Metabase, Superset et Qlik ne sont **pas exécutés** : tout ce qui se vérifie ici se vérifie avec pandas et SQL. Le fichier est **autonome** : une seule cellule recharge tout.

```
import os, sys, warnings
import numpy as np, pandas as pd
warnings.filterwarnings("ignore")
sys.path.insert(0, "build")
import outils_ch02 as O
d = O.charger(os.environ["DONNEES"])
star = O.etoile(d)
faits = star["fait_ventes"]
ventes = d["fv"]
v25 = ventes[ventes["annee"] == 2025]
hebdo = O.serie_hebdo(d)
print(len(faits), "lignes de faits |", len(hebdo), "semaines | CA 2025 :",
      ↪ round(v25["montant"].sum()), "€")
```

83905 lignes de faits | 157 semaines | CA 2025 : 1324764 €

Applications

Application 2.1 — Le modèle en étoile et son contrôle (section 2.1.2)

Objectif. Contrôler un modèle en étoile, voir ce qui se passe quand il casse, et mesurer l'effet d'une clé qui n'est pas unique.

Étape 1 — le contrôle de base. Les effectifs, l'unicité des clés et les lignes orphelines de chaque dimension.

```
print(O.controle_etoile(star).to_string(index=False))
```

dimension	lignes	cle_unique	faits_orphelins
dim_date	1096	True	0
dim_produit	120	True	0
dim_client	6000	True	0

dim_canal	3	True	0
-----------	---	------	---

Étape 2 — une dimension amputée. On retire trois produits de la dimension (les trois plus vendus de 2025 : 86, 87 et 82) et l'on regarde ce que devient le contrôle, puis ce que perdrait un tableau de bord qui joindrait **en ne gardant que les correspondances**.

```
dim_amputee = star["dim_produit"][~star["dim_produit"]["id_produit"].isin([86, 87, 82])]
star_2 = {**star, "dim_produit": dim_amputee}
print(0.controle_etoile(star_2).query("dimension == 'dim_produit'").to_string(index=False))
garde = faits.merge(dim_amputee[["id_produit"]], on="id_produit", how="inner")
print("CA avant :", round(faits["montant"].sum()), "| après jointure interne :",
      ↪ round(garde["montant"].sum()), "| perdu :", round(faits["montant"].sum() -
      ↪ garde["montant"].sum()), "€")
```

```
dimension lignes cle_unique faits_orphelins
dim_produit 117 True 3235
CA avant : 3653157 | après jointure interne : 3222338 | perdu : 430819 €
```

Étape 3 — la clé qui multiplie. On relie les faits à la dimension **par le nom** et l'on vérifie le total.

```
dim_nom = star["dim_produit"][["id_produit", "nom_produit"]]
par_nom = faits.merge(dim_nom,
  ↪ on="id_produit").drop(columns="id_produit").merge(dim_nom.drop(columns="id_produit"),
  ↪ on="nom_produit")
print(len(faits), "->", len(par_nom), "| CA :", round(faits["montant"].sum()), "->",
      ↪ round(par_nom["montant"].sum()))
print("noms portés par plus d'un produit :", int((dim_nom.groupby("nom_produit").size() > 1).sum()),
      ↪ "sur", dim_nom["nom_produit"].nunique())
```

```
83905 -> 167810 | CA : 3653157 -> 7306315
noms portés par plus d'un produit : 60 sur 60
```

Lecture. Les trois produits retirés laissent **3 235 lignes orphelines** : une jointure interne les ferait disparaître et le tableau de bord perdrait **430 819 €** de chiffre d'affaires, sans aucun message. À l'inverse, joindre par le nom **double** le total (3 653 157 € devient 7 306 315 €) parce que chacun des 60 noms est porté par deux produits. Les deux erreurs ont la même parade : contrôler les effectifs et l'unicité des clés **avant** de calculer.

Pour conclure. Écrivez en deux phrases ce que vous diriez à la gérante si le total de la page ne correspondait pas à celui de la comptabilité : les trois premières questions à poser sur le modèle.

Application 2.2 — Mesures, ratios et totaux (section 2.1.3)

Objectif. Écrire une fonction « mesure » unique, l'appliquer dans plusieurs contextes de filtres et vérifier quand un total est la somme des lignes.

Étape 1 — une recette, plusieurs contextes. La fonction renvoie le chiffre d'affaires, les commandes distinctes, le panier moyen et le taux de marge **pour les lignes qu'on lui donne**.

```
def mesures(t):
    ca, n = t["montant"].sum(), t["id_commande"].nunique()
    return pd.Series({"ca": ca, "commandes": n, "panier": ca / n, "taux_marge": t["marge_ht"].sum() /
  ↪ (ca / 1.2)})
contextes = {"tout": v25, "Site": v25[v25["canal"] == "Site"], "Jardin": v25[v25["categorie"] ==
  ↪ "Jardin"],
  ↪ "Jardin sur le Site": v25[(v25["categorie"] == "Jardin") & (v25["canal"] == "Site")]}
print(pd.DataFrame({k: mesures(t) for k, t in contextes.items()}).T.round(4))
```

	ca	commandes	panier	taux_marge
tout	1324763.72	12946.0	102.3300	0.3796
Site	617715.45	6078.0	101.6314	0.3788
Jardin	353954.64	4315.0	82.0289	0.3827
Jardin sur le Site	166201.16	1991.0	83.4762	0.3831

Étape 2 — additif ou non ? On compare le total au **cumul des lignes** d'un tableau croisé, par canal puis par catégorie.

```
for dim in ["canal", "categorie"]:
    tab = v25.groupby(dim).apply(mesures)
    print(dim, "| commandes : total", int(mesures(v25)["commandes"]), "vs somme des lignes",
          ↪ int(tab["commandes"].sum()),
          "| CA : total", round(mesures(v25)["ca"]), "vs somme", round(tab["ca"].sum()))
```

```
canal | commandes : total 12946 vs somme des lignes 12946 | CA : total 1324764 vs somme 1324764
categorie | commandes : total 12946 vs somme des lignes 25163 | CA : total 1324764 vs somme 1324764
```

Étape 3 — le ratio qu'on moyenne. On compare le taux de marge **global** à la **moyenne** des taux par catégorie, puis à la moyenne **pondérée** par le chiffre d'affaires.

```
tab = v25.groupby("categorie").apply(mesures)
print("global :", round(mesures(v25)["taux_marge"], 4), "| moyenne simple :",
      ↪ round(tab["taux_marge"].mean(), 4),
      "| moyenne pondérée par le CA :", round(np.average(tab["taux_marge"], weights=tab["ca"]), 4))
```

```
global : 0.3796 | moyenne simple : 0.3753 | moyenne pondérée par le CA : 0.3796
```

Lecture. Le panier du Jardin (82,03 €) est inférieur à celui de la boutique (102,33 €) parce que la mesure divise le chiffre d'affaires **du Jardin** par les commandes qui **contiennent** du Jardin : un contexte de filtres change le périmètre du comptage, pas seulement la somme. Le nombre de commandes est **additif sur les canaux** (chaque commande a un seul canal : 12 946 des deux côtés) mais **pas sur les catégories** (25 163 contre 12 946). Enfin, la moyenne des taux de marge pondérée par le chiffre d'affaires redonne exactement le taux global (37,96 %) alors que la moyenne simple s'en écarte (37,53 %) : « somme sur somme » **est** une moyenne pondérée.

Pour conclure. Une des deux dimensions (canal ou catégorie) rend le comptage des commandes additif, l'autre non : laquelle, et pourquoi ? Qu'est-ce que cela change pour le total d'un tableau croisé ?

Application 2.3 — Une feuille façon Tableau (section 2.2)

Objectif. Reproduire en pandas les calculs de table et un niveau de détail.

Étape 1 — parts de la ligne et du total. Chiffre d'affaires par catégorie et par canal, puis deux « parts du total » : dans la catégorie (la somme d'une ligne fait 100 %) et dans l'ensemble.

```
f = v25.pivot_table(index="categorie", columns="canal", values="montant", aggfunc="sum")
part_categorie = f.div(f.sum(axis=1), axis=0) * 100
part_ensemble = f / f.to_numpy().sum() * 100
print("part de chaque canal dans la catégorie (%)\\n", part_categorie.round(1))
print("part dans l'ensemble (%), somme =", round(part_ensemble.to_numpy().sum(), 1), "\\n",
      ↪ part_ensemble.round(1))
```

```
part de chaque canal dans la catégorie (%)
canal      Boutique Réseaux Site
categorie
Bien-être   41.4     11.6  46.9
Cuisine     42.2     10.8  47.0
Décoration  43.2     10.8  45.9
```

Jardin	42.1	10.9	47.0
Maison	42.4	11.4	46.2
Papeterie	41.8	10.3	47.9
part dans l'ensemble (%), somme = 100.0			
canal	Boutique	Réseaux	Site
categorie			
Bien-être	3.7	1.0	4.2
Cuisine	7.4	1.9	8.3
Décoration	8.4	2.1	9.0
Jardin	11.3	2.9	12.5
Maison	9.7	2.6	10.6
Papeterie	1.8	0.4	2.0

Étape 2 — rang, cumul et moyenne mobile. Le rang des catégories dans chaque canal, puis, pour le canal Site, la moyenne mobile sur quatre semaines.

```
print(f.rank(ascending=False).astype(int))
site = ventes[ventes["canal"] == "Site"].assign(sem=lambda t:
    ↪ t["date_commande"].dt.to_period("W-SUN").dt.start_time)
site_hebdo = site.groupby("sem")["montant"].sum()
print(pd.DataFrame({"semaine": site_hebdo.tail(4), "moyenne mobile 4":
    ↪ site_hebdo.rolling(4).mean().tail(4)}).round(0))
```

canal	Boutique	Réseaux	Site
categorie			
Bien-être	5	5	5
Cuisine	4	4	4
Décoration	3	3	3
Jardin	1	1	1
Maison	2	2	2
Papeterie	6	6	6
semaine moyenne mobile 4			
sem			
2025-12-08	20293.0		18881.0
2025-12-15	20888.0		20545.0
2025-12-22	21582.0		21367.0
2025-12-29	4581.0		16836.0

Étape 3 — un niveau de détail FIXED. Le total de chaque client, puis le nombre de clients qui dépensent plus de 1 000 € par canal où ils ont le plus dépensé.

```
par_client_canal = v25.groupby(["id_client", "canal"])["montant"].sum().reset_index()
total_client = par_client_canal.groupby("id_client")["montant"].transform("sum")
principal = par_client_canal.loc[par_client_canal.groupby("id_client")["montant"]
    ↪ ].idxmax()].assign(total=lambda t: total_client.loc[t.index])
gros = principal[principal["total"] > 1000]
print(len(gros), "clients au-dessus de 1 000 € sur", len(principal))
print(gros["canal"].value_counts().to_string())
```

```
204 clients au-dessus de 1 000 € sur 3875
canal
Site      113
Boutique   82
Réseaux    9
```

Lecture. La répartition entre canaux est presque la même dans toutes les catégories (le Site pèse de 45,9 % à 47,9 % de chacune) et le **rang** des catégories est identique dans les trois canaux : le canal n'explique pas les différences entre catégories. Une absence de différence est un résultat : un graphique ventilé par canal n'apporterait rien à la gérante. Remarquez aussi la dernière ligne de l'étape 2 : la semaine du 29 décembre est **incomplète** (trois jours), sa moyenne mobile s'effondre, et une page automatique doit **exclude** les semaines incomplètes. Enfin, 204 clients sur 3 875 ont

dépensé plus de 1 000 € en 2025, surtout par le Site (113) et la boutique (82).

Pour conclure. Pour chaque calcul des étapes 1 à 3, dites s'il s'agit d'un **calcul de table** (dépend de la structure affichée) ou d'un **niveau de détail** (indépendant).

Application 2.4 — La page de la gérante pour n'importe quelle semaine (section 2.3.3)

Objectif. Rendre la page **paramétrable** : une fonction qui renvoie les chiffres d'une semaine donnée et ses alertes.

Étape 1 — la fonction. Pour chaque semaine : le chiffre d'affaires, son évolution sur l'an dernier, et pour les deux indicateurs d'alerte la valeur, l'habituel et l'état.

```
def cartes(semaine):
    t = pd.Timestamp(semaine)
    k, _ = 0.kpi_semaine(d, t)
    cs, ad = k["cette_semaine"], k["an_dernier"]
    out = {"CA (k€)": cs["ca"] / 1000, "évolution CA": cs["ca"] / ad["ca"] - 1}
    for nom, sens in [("a_l_heure", "bas"), ("rupture", "haut")]:
        m, bas, haut = 0.limite hebdo[nom], t)
        out[nom] = cs[nom]; out[nom + " habituel"] = m
        out[nom + " alerte"] = int(cs[nom] < bas if sens == "bas" else cs[nom] > haut)
    return pd.Series(out)
semaines = ["2025-06-23", "2025-09-01", "2025-11-24", "2025-12-08", "2025-12-15", "2025-12-22"]
print(pd.DataFrame({s: cartes(s) for s in semaines}).round(3).to_string())
```

	2025-06-23	2025-09-01	2025-11-24	2025-12-08	2025-12-15	2025-12-22
CA (k€)	23.958	27.166	39.551	39.088	42.729	44.168
évolution CA	-0.115	0.151	0.170	0.037	0.129	0.349
a_l_heure	0.738	0.804	0.784	0.490	0.462	0.445
a_l_heure habituel	0.748	0.782	0.783	0.786	0.775	0.763
a_l_heure alerte	0.000	0.000	0.000	1.000	1.000	1.000
rupture	0.036	0.107	0.157	0.207	0.121	0.350
rupture habituel	0.049	0.046	0.061	0.073	0.079	0.084
rupture alerte	0.000	0.000	0.000	0.000	0.000	1.000

Étape 2 — les phrases. Une alerte n'est utile que si elle est lue : on la transforme en phrase.

```
def phrase(s):
    c = cartes(s)
    msgs = []
    if c["a_l_heure alerte"]: msgs.append(f"livraisons à l'heure : {c['a_l_heure']:.0%} (habituel {c['a_l_heure habituel']:.0%})")
    if c["rupture alerte"]: msgs.append(f"rupture : {c['rupture']:.0%} (habituel {c['rupture habituel']:.0%})")
    return f"semaine du {s} : " + (" ; ".join(msgs) if msgs else "rien à signaler")
for s in semaines:
    print(phrase(s))
```

```
semaine du 2025-06-23 : rien à signaler
semaine du 2025-09-01 : rien à signaler
semaine du 2025-11-24 : rien à signaler
semaine du 2025-12-08 : livraisons à l'heure : 49% (habituel 79%)
semaine du 2025-12-15 : livraisons à l'heure : 46% (habituel 77%)
semaine du 2025-12-22 : livraisons à l'heure : 45% (habituel 76%); rupture : 35% (habituel 8%)
```

Lecture. L'alerte de livraison se déclenche dès la semaine du 8 décembre (49 % contre 79 % d'habitude) et dure trois semaines ; celle de rupture ne se déclenche que la semaine du 22 (35 % contre 8 %), alors que la rupture atteignait déjà 20,7 % le 8 décembre sans franchir sa limite. À l'inverse, la semaine du 23 juin est en **baisse** de 11,5 % sur l'an dernier sans qu'aucune alerte ne soit levée : la page ne confond pas une variation de ventes avec un défaut de fonctionnement.

Pour conclure. Sur ces cinq semaines, à quelle date les alertes se déclenchent-elles ? Avec quelle semaine de retard par rapport au début réel du problème ?

Application 2.5 — Contraste et lisibilité (section 2.3.4)

Objectif. Mesurer le contraste des couleurs d'une page et corriger celles qui sont insuffisantes.

Étape 1 — la mesure. Le rapport de contraste entre deux couleurs ; on l'applique aux couleurs de la page de la gérante, sur le fond blanc des cartes et sur le fond rose de la zone d'alerte.

```
def contraste(a, b="#ffffff"):
    def lum(h):
        c = [int(h[i:i + 2], 16) / 255 for i in (1, 3, 5)]
        c = [x / 12.92 if x <= 0.03928 else ((x + 0.055) / 1.055) ** 2.4 for x in c]
        return 0.2126 * c[0] + 0.7152 * c[1] + 0.0722 * c[2]
    hi, lo = sorted([lum(a), lum(b)], reverse=True)
    return (hi + 0.05) / (lo + 0.05)
couleurs = {"bleu": "#2a78d6", "orange": "#eb6834", "vert": "#1baf7a", "rouge": "#e34948", "gris
↳ moyen": "#898781", "gris texte": "#52514e"}
tab = pd.DataFrame({"sur blanc": {k: contraste(c) for k, c in couleurs.items()}, "sur rose": {k:
↳ contraste(c, "#fdecec") for k, c in couleurs.items()}})
print(tab.round(2))
```

	sur blanc	sur rose
bleu	4.42	3.87
orange	3.20	2.80
vert	2.82	2.46
rouge	3.95	3.46
gris moyen	3.59	3.14
gris texte	7.94	6.95

Étape 2 — la correction. Pour chaque couleur de texte sous 4,5 : 1, on la **fonce** progressivement jusqu'à atteindre le seuil sur le fond rose.

```
def foncer(c, fond, seuil=4.5):
    r, g, b = (int(c[i:i + 2], 16) for i in (1, 3, 5))
    while contraste(f"#{r:02x}{g:02x}{b:02x}", fond) < seuil:
        r, g, b = int(r * 0.97), int(g * 0.97), int(b * 0.97)
    return f"#{r:02x}{g:02x}{b:02x}"
for nom in ["rouge", "gris moyen"]:
    nouveau = foncer(couleurs[nom], "#fdecec")
    print(nom, couleurs[nom], "->", nouveau, round(contraste(nouveau, "#fdecec"), 2))
```

```
rouge #e34948 -> #c13c3c 4.62
gris moyen #898781 -> #6c6a65 4.73
```

Lecture. Sur fond blanc, seul le gris texte (7,94 : 1) atteint 4,5 : 1 ; le bleu en est tout près (4,42) et convient à des traits. Sur le fond rose de la zone d'alerte, **tous** les textes colorés passent sous 4,5 : 1. Le vert (2,82) et l'orange (3,20) sont **inutilisables pour du texte** : on les réserve à des aplats accompagnés d'une étiquette. Les couleurs corrigées sont #c13c3c pour le rouge (4,62) et #6c6a65 pour le gris (4,73).

Pour conclure. Écrivez la règle de couleur que vous donneriez à une équipe : pour quels éléments (texte, trait, aplat) le seuil est-il de 4,5 : 1 ou de 3 : 1 ?

Application 2.6 — Du DAX à pandas (section 2.4.1)

Objectif. Vérifier en pandas ce que des mesures de comparaison dans le temps devraient afficher.

Étape 1 — « cumul à date » et « même période de l'an dernier ». Une fonction qui calcule le cumul depuis le 1er janvier jusqu'à un jour donné, appliquée aux fins de trimestre.

```
def cumul_a_date(annee, mois_jour):
    fin = pd.Timestamp(f"{annee}-{mois_jour}")
    t = faits[(faits["date"] >= f"{annee}-01-01") & (faits["date"] <= fin)]
    return t["montant"].sum()
fins = ["03-31", "06-30", "09-30", "12-31"]
tab = pd.DataFrame({"2024": [cumul_a_date(2024, m) for m in fins], "2025": [cumul_a_date(2025, m) for
    m in fins]}, index=fins)
tab["évolution"] = tab["2025"] / tab["2024"] - 1
print(tab.round(3).to_string())
```

	2024	2025	évolution
03-31	225969.34	251608.69	0.113
06-30	522363.05	562391.37	0.077
09-30	798738.15	876963.34	0.098
12-31	1189461.17	1324763.72	0.114

Étape 2 — l'erreur de période. On compare le cumul à fin septembre 2025 au total **annuel** de 2024, puis à son cumul de **même période**.

```
c25 = cumul_a_date(2025, "09-30")
print("contre l'année 2024 entière :", round(c25 / cumul_a_date(2024, "12-31") - 1, 3))
print("contre fin septembre 2024 :", round(c25 / cumul_a_date(2024, "09-30") - 1, 3))
```

```
contre l'année 2024 entière : -0.263
contre fin septembre 2024 : 0.098
```

Étape 3 — l'équivalent de DIVIDE. On calcule l'évolution mensuelle de chaque produit en 2025 ; quand le mois précédent est nul, la division donne l'infini, que l'on remplace par une valeur manquante.

```
m = v25.assign(mois=v25["date_commande"].dt.month).pivot_table(index="id_produit", columns="mois",
    values="montant", aggfunc="sum", fill_value=0)
evol = (m / m.shift(1, axis=1) - 1).iloc[:, 1:]
print("divisions par zéro :", int(np.isinf(evol.to_numpy()).sum()), "sur", evol.size, "cases")
evol = evol.replace([np.inf, -np.inf], np.nan)
print("évolutions définies :", int(evol.notna().to_numpy().sum()), "| médiane :",
    round(float(np.nanmedian(evol.to_numpy())), 3))
```

```
divisions par zéro : 2 sur 1320 cases
évolutions définies : 1318 | médiane : 0.057
```

Lecture. À période égale, l'évolution du cumul reste comprise entre +7,7 % et +11,4 % selon la date de coupure ; comparée à l'année 2024 entière, elle devient -26,3 %, une absurdité de période. Dans la grille produit × mois, 2 divisions sur 1 320 cases tombent sur un dénominateur nul : sans remplacement, ces infinis fausseraient une moyenne ou un graphique. La médiane des évolutions mensuelles définies est de +5,7 %.

Pour conclure. Ecrivez ce que votre page affichera quand l'évolution n'est pas définie, et pourquoi ce choix est une décision de définition.

Application 2.7 — Sécurité au niveau des lignes (section 2.4.3)

Objectif. Construire des vues par rôle, écrire le contrôle qui prouve qu'elles sont complètes, et le voir détecter une donnée mal classée.

Étape 1 — la table de droits et la vue. Un rôle dynamique : la table dit qui voit quelle région.

```
droits = pd.DataFrame({"utilisateur": ["direction"] * 4 + ["resp_1", "resp_3", "resp_12", "resp_12"],
                      "region": ["Région 1", "Région 2", "Région 3", "Région 4", "Région 1", "Région 3", "Région 1", "Région 2"]})

def lignes_avec_region(dim_client):
    return faits[faits["date"] >= "2025-01-01"].merge(dim_client[["id_client", "region"]],
                                                    on="id_client", how="left")
f25 = lignes_avec_region(star["dim_client"])
vue = lambda u, t=f25: t[t["region"].isin(droits.loc[droits["utilisateur"] == u, "region"])]
print({u: round(vue(u)["montant"].sum()) for u in ["direction", "resp_1", "resp_3", "resp_12", "inconnu"]})
```

```
{'direction': 1324764, 'resp_1': 511532, 'resp_3': 537175, 'resp_12': 641872, 'inconnu': 0}
```

Étape 2 — le contrôle. Deux tests : la vue de la direction égale le total, et aucune ligne n'est « invisible » (aucune ligne sans région).

```
def controle(t, droits):
    vue_direction = t[t["region"].isin(droits.loc[droits["utilisateur"] == "direction", "region"])]
    return {"vue direction = total": bool(np.isclose(vue_direction["montant"].sum(),
                                                    t["montant"].sum())), "lignes sans région": int(t["region"].isna().sum())}
print(controle(f25, droits))
```

```
{'vue direction = total': True, 'lignes sans région': 0}
```

Étape 3 — une donnée mal classée. On efface la région de 60 clients (une ville inconnue), on refait le contrôle, puis on corrige par une région « Non classé » **que seule la direction voit**.

```
clients_abimes = star["dim_client"].copy()
clients_abimes.loc[clients_abimes.sample(60, random_state=3).index, "region"] = np.nan
f_abime = lignes_avec_region(clients_abimes)
print("abîmé :", controle(f_abime, droits), "| CA invisible :",
      round(f_abime.loc[f_abime["region"].isna(), "montant"].sum()), "€")
f_corrige = f_abime.assign(region=f_abime["region"].fillna("Non classé"))
droits_c = pd.concat([droits, pd.DataFrame({"utilisateur": ["direction"], "region": ["Non classé"]})], ignore_index=True)
print("corrigé :", controle(f_corrige, droits_c) | {"CA Non classé":
      round(f_corrige.loc[f_corrige["region"] == "Non classé", "montant"].sum())})
```

```
abîmé : {'vue direction = total': False, 'lignes sans région': 282} | CA invisible : 12368 €
corrigé : {'vue direction = total': True, 'lignes sans région': 0, 'CA Non classé': 12368}
```

Lecture. Avec 60 clients mal classés, 282 lignes de faits (12 368 €) disparaissent des vues de **tous** les responsables, sans aucun message : seul le contrôle « lignes sans région » le révèle. La région « Non classé » rétablit la complétude, mais elle ne doit être visible que de la direction (un responsable verrait sinon des clients qui ne sont pas les siens) et elle crée une **tâche** : classer ces clients.

Pour conclure. Pourquoi la région « Non classé » ne doit-elle être visible que de la direction ? Que se passerait-il sans le test « lignes sans région » ?

Application 2.8 — Choisir un outil (section 2.5)

Objectif. Pondérer des critères, puis tester la **robustesse** du classement quand les poids changent.

Étape 1 — le score. Trois options fictives notées de 1 à 5 sur quatre critères.

```
notes = pd.DataFrame({"coût": [5, 2, 4], "facilité": [4, 3, 2], "gouvernance": [2, 5, 3],
    ↪ "flexibilité": [2, 4, 5]}, index=["A", "B", "C"])
poids = pd.Series({"coût": 3, "facilité": 3, "gouvernance": 2, "flexibilité": 2})
print((notes @ poids).sort_values(ascending=False))
```

```
A    35
C    34
B    33
dtype: int64
```

Étape 2 — des poids au hasard. On tire 2 000 jeux de poids au hasard (somme égale à 1) et l'on compte combien de fois chaque option gagne.

```
rng = np.random.default_rng(7)
tirages = rng.dirichlet(np.ones(4), 2000)
scores = tirages @ notes.to_numpy().T
gagnants =
    ↪ pd.Series(np.array(notes.index)[scores.argmax(axis=1)].value_counts(normalize=True).sort_index())
print((gagnants * 100).round(1).to_string())
```

```
A    32.6
B    40.2
C    27.2
```

Étape 3 — le poids qui fait basculer. On fait varier le poids de la gouvernance de 0 à 6 en gardant les autres, et l'on note le premier poids où B devient meilleure que A.

```
for g in range(0, 7):
    s = notes @ poids.mask(poids.index == "gouvernance", g)
    print(g, s.idxmax(), s.round(0).to_dict())
```

```
0 A {'A': 31, 'B': 23, 'C': 28}
1 A {'A': 33, 'B': 28, 'C': 31}
2 A {'A': 35, 'B': 33, 'C': 34}
3 B {'A': 37, 'B': 38, 'C': 37}
4 B {'A': 39, 'B': 43, 'C': 40}
5 B {'A': 41, 'B': 48, 'C': 43}
6 B {'A': 43, 'B': 53, 'C': 46}
```

Lecture. Avec les poids de départ, A gagne d'un point seulement (35 contre 34 et 33). Sur 2 000 jeux de poids aléatoires, **B gagne 40,2 %** des fois, A 32,6 % et C 27,2 % : aucune option ne domine, et B dépasse A dès que la gouvernance pèse 3. La recommandation honnête est donc « le choix dépend de l'importance donnée à la gouvernance », plutôt que « A est le meilleur ».

Pour conclure. Rédigez une recommandation en trois phrases qui **cite** la robustesse du classement et non seulement le classement.

Exercices

Exercice 2.1 ★ — Fait ou dimension ? (section 2.1.2)

Pour chacune des colonnes suivantes, dites si elle appartient à la **table de faits** ou à une **dimension** (et laquelle) : quantité, catégorie, montant, région, mois, type (physique ou en ligne), cout_achat, remise_pct. Une colonne peut être ambiguë : justifiez en une phrase.

Exercice 2.2 ★★ — Une dimension qui contient des doublons (section 2.1.2)

On ajoute à la dimension des clients dix lignes en double (les dix premiers clients, copiés). **Avant de calculer**, prévoyez le nombre de lignes de faits et l'écart de chiffre d'affaires après la jointure des faits avec cette dimension. Puis vérifiez avec pandas, et écrivez le contrôle (une ligne) qui aurait signalé le problème.

Exercice 2.3 ★ — Mesure ou colonne calculée ? (section 2.1.3)

Classez, en justifiant, chaque besoin en **colonne calculée** ou **mesure** : (a) la marge d'une ligne de commande ; (b) le taux de marge d'une catégorie ; (c) le nombre de clients distincts d'un mois ; (d) l'année d'une date ; (e) le panier moyen du canal sélectionné ; (f) la tranche d'âge d'un client.

Exercice 2.4 ★★ — Parts et rangs par canal (section 2.2.2)

Calculez, pour chaque canal, la **part de chaque catégorie** dans le chiffre d'affaires du canal (une seule instruction avec transform), puis le **rang** de chaque catégorie dans le canal. Une catégorie change-t-elle de rang, ou de part, d'un canal à l'autre ? Que conclure pour un tableau de bord : le canal est-il un bon axe de ventilation ?

Exercice 2.5 ★★ — Un niveau de détail : clients par fréquence (section 2.2.2)

En 2025, calculez le **nombre de commandes par client** (niveau « client »), puis le **chiffre d'affaires moyen par client** selon cette fréquence (1 commande, 2, 3 et plus). La moyenne des lignes aurait-elle répondu à la même question ?

Exercice 2.6 ★ — La décision de la responsable logistique (section 2.3.1)

Une responsable logistique (fictive) veut un tableau de bord. Rédigez, comme au tableau de 2.3.1, **quatre lignes** : décision, question, indicateur (avec définition d'une phrase) et seuil d'action. Dites ce que vous **ne mettez pas** sur la page et pourquoi.

Exercice 2.7 ★★ — Quelle référence pour quel indicateur ? (section 2.3.3)

Pour la semaine du 22 décembre 2025, calculez pour la **conversion du site**, la **rupture** et les **livraisons à l'heure** : la valeur, l'habituel (26 semaines précédentes), les limites à trois écarts-types et l'état (hors limite ou non). Quelle référence (l'habituel ou l'an dernier) reprenez-vous pour chacun, et pourquoi ?

Exercice 2.8 ★★ — Un dictionnaire qui se teste (section 2.3.5)

Écrivez le dictionnaire des indicateurs de la page de la gérante comme un `DataFrame` (nom, définition, source, propriétaire), puis un **test automatique** qui vérifie que chaque colonne de hebdo citée dans la page a une fiche, et qu'aucune fiche n'est vide.

Exercice 2.9 ★★★ — Une recette qui trouve l'erreur (section 2.3.5)

On vous donne un « tableau de bord » dont les chiffres viennent d'une table où l'on a **dupliqué par erreur** 200 lignes de faits. Écrivez une recette qui compare, pour la semaine du 22 décembre 2025, le chiffre d'affaires, les commandes et le panier moyen calculés par **deux chemins** (pandas et SQL sur une table dédoublée par `id_ligne`), puis trouvez les lignes dupliquées.

Exercice 2.10 ★★ — À période égale (section 2.4.1)

Un collègue affirme que « le chiffre d'affaires de 2025 à fin juin est en baisse de 53 % sur 2024 ». Retrouvez son calcul, expliquez l'erreur et donnez le bon chiffre. Faites de même pour un trimestre (le troisième) pris isolément.

Exercice 2.11 ★★★ — Droits : le cas qui échappe (section 2.4.3)

La table des droits contient une faute de frappe : « Region 3 » (sans accent) au lieu de « Région 3 » pour `resp_3`. Montrez ce que voit `resp_3`, écrivez un contrôle qui **détecte** les régions de la table de droits absentes du modèle, et proposez deux protections (une dans les données, une dans le processus).

Exercice 2.12 ★★ — Pondérer vos propres critères (section 2.5.2)

Choisissez **cinq critères** et **trois options** de votre contexte (réel ou imaginaire), notez les options de 1 à 5, calculez le score pondéré, puis testez la robustesse du classement par 1 000 jeux de poids aléatoires. Rédigez une recommandation de cinq lignes.

Corrigés

Corrigé 2.1

Colonne	Où	Pourquoi
quantite	faits	un nombre mesuré par événement
montant	faits	idem
remise_pct	faits	décrit cette ligne, pas le produit
cout_achat	dimension produit	une valeur du produit, répétée sur chaque ligne si on la mettait dans les faits
categorie	dimension produit	attribut du produit
region	dimension client	attribut du client (via sa ville)
mois	dimension date	dérivé de la date, recalculable
type	dimension canal	attribut du canal

Un cas ambigu : `cout_achat` pourrait figurer dans les faits (on y gardant le coût **au moment** de la vente) ; c'est un choix correct si le coût d'achat change dans le temps et que l'on veut conserver l'historique. Ici, il est constant par produit : on le garde dans la dimension, et la marge se calcule avec `RELATED` (2.4.1).

Corrigé 2.2

```
dim_cli = star["dim_client"]
dup = pd.concat([dim_cli, dim_cli.head(10)], ignore_index=True)
j = faits.merge(dup, on="id_client", how="left")
n_dix = int(faits["id_client"].isin(dim_cli.head(10)["id_client"]).sum())
print("lignes de faits des dix clients :", n_dix, "| après jointure :", len(j), "(", len(j) -
    ↳ len(faits), "en plus)")
print("CA :", round(faits["montant"].sum()), "->", round(j["montant"].sum()), "| écart :",
    ↳ round(j["montant"].sum() - faits["montant"].sum(), 2))
print("contrôle :", "clé unique ?", dup["id_client"].is_unique)
```

```
lignes de faits des dix clients : 280 | après jointure : 84185 ( 280 en plus )
CA : 3653157 -> 3664792 | écart : 11634.53
contrôle : clé unique ? False
```

Prédiction : chaque ligne de faits d'un des dix clients est comptée **deux fois**, donc l'écart de lignes est égal au nombre de lignes de ces dix clients, et l'écart de chiffre d'affaires à leur chiffre d'affaires. Les dix clients ont 280 lignes de faits : la jointure en donne 84 185 (+280) et un chiffre d'affaires de 3 664 792 € (+11 634,53 €). Le contrôle en une ligne est le test d'**unicité de la clé** : `dup["id_client"].is_unique`.

Corrigé 2.3

- (a) **colonne calculée** : calculée par ligne, indépendante du filtre. (b) **mesure** : un ratio de sommes, dépendant du contexte. (c) **mesure** : comptage de valeurs distinctes, non additif. (d) **colonne calculée** (ou colonne de la dimension de dates) : dérivée d'une seule ligne. (e) **mesure** : dépend du canal sélectionné. (f) **colonne calculée** : attribut d'un client, stable ; on la met dans la dimension.

Corrigé 2.4

```
c = v25.groupby(["canal", "categorie"], as_index=False)["montant"].sum()
c["part_canal"] = c["montant"] / c.groupby("canal")["montant"].transform("sum") * 100
c["rang"] = c.groupby("canal")["montant"].rank(ascending=False).astype(int)
print(c.pivot(index="categorie", columns="canal", values="rang").to_string())
print(c.pivot(index="categorie", columns="canal", values="part_canal").round(1).to_string())
```

canal	Boutique	Réseaux	Site
categorie			
Bien-être	5	5	5
Cuisine	4	4	4
Décoration	3	3	3
Jardin	1	1	1
Maison	2	2	2
Papeterie	6	6	6
canal	Boutique	Réseaux	Site
categorie			
Bien-être	8.7	9.4	8.9
Cuisine	17.6	17.2	17.7
Décoration	19.9	19.2	19.2
Jardin	26.6	26.5	26.9

Maison	23.0	23.8	22.8
Papeterie	4.2	4.0	4.4

La première table donne le rang de chaque catégorie dans chaque canal ; on compare les colonnes Site et Réseaux pour repérer les catégories qui changent de place. `transform("sum")` est l'équivalent d'un calcul de table « part de la colonne » : il garde toutes les lignes. Résultat : **aucune** catégorie ne change de rang d'un canal à l'autre, et les parts sont presque identiques (Maison : 23,0 % en boutique, 23,8 % sur les Réseaux, 22,8 % sur le Site). Le canal n'est donc pas un bon axe pour ventiler les catégories : on le garde comme **filtre**, pas comme dimension d'un graphique de plus.

Corrigé 2.5

```
cmd = v25.groupby("id_client").agg(n=("id_commande", "nunique"), ca=("montant", "sum"))
cmd["classe"] = pd.cut(cmd["n"], [0, 1, 2, np.inf], labels=["1", "2", "3 et plus"])
print(cmd.groupby("classe", observed=True).agg(clients=("ca", "size"), ca_moyen=("ca",
↳ "mean")).round(1))
print("moyenne des lignes :", round(v25["montant"].mean(), 2), "€")
```

	clients	ca_moyen
classe		
1	1221	99.0
2	806	201.8
3 et plus	1848	563.4
moyenne des lignes :	44.41 €	

Le calcul se fait en **deux temps** : le niveau « client » (nombre de commandes, total), puis la moyenne **par classe de fréquence**. La moyenne des lignes (44,41 €) répond à une autre question (le montant d'une ligne de commande) : elle ne dit rien d'un client. Les 1 848 clients qui ont passé trois commandes ou plus dépensent en moyenne 563,40 €, contre 99,00 € pour les 1 221 clients à une seule commande : la **fréquence** structure la valeur d'un client.

Corrigé 2.6

Exemple de réponse :

Décision	Question	Indicateur (définition)	Seuil d'action
Changer de transporteur pour un trajet	Quelle part des colis arrive en retard ?	Retards (colis livrés dans la semaine avec retard ÷ colis livrés)	au-dessus de la limite haute de l'habituel
Prévoir du personnel à l'expédition	Combien de colis partent cette semaine ?	Colis expédiés par jour (comptage)	au-dessus de la capacité
Relancer un fournisseur	Les réapprovisionnements arrivent-ils à temps ?	Délai moyen de réapprovisionnement (jours, du bon de commande à la réception)	au-dessus de l'habituel + 3 écarts-types
Rouvrir un entrepôt de proximité	Les délais sont-ils trop longs dans une région ?	Délai de livraison médian par région	médiane au-dessus de l'objectif

Ce qu'on ne met pas : le stock de chaque produit (120 lignes : c'est un état de gestion, pas un indicateur de pilotage), le détail de chaque colis (niveau 3) et les chiffres d'affaires (sans décision pour cette personne). Chaque indicateur garde sa **fonction** de propriétaire et sa **définition écrite**.

Corrigé 2.7

```
t0 = pd.Timestamp("2025-12-22")
lignes = {}
for nom in ["conversion", "rupture", "a_l_heure"]:
    m, bas, haut = 0.limite hebdo[nom], t0)
    v = hebdo.loc[t0, nom]
    lignes[nom] = {"valeur": v, "habituel": m, "limite basse": bas, "limite haute": haut, "hors
    ↪ limites": bool(v < bas or v > haut)}
print(pd.DataFrame(lignes).T.round(3).to_string())
```

	valeur	habituel	limite basse	limite haute	hors limites
conversion	0.065031	0.049115	0.022735	0.075495	False
rupture	0.35	0.083516	-0.116381	0.283414	True
a_l_heure	0.445255	0.763073	0.485427	1.040719	True

Pour la **conversion**, la valeur est dans les limites : pas d'alerte, et aucune comparaison à l'an dernier n'est possible (les sessions n'existent que pour 2025). Pour la **rupture** et les **livraisons à l'heure**, la semaine est hors limites. La référence est l'**habituel** (26 semaines précédentes) pour les trois : ce sont des indicateurs de fonctionnement, pas saisonniers, et une comparaison à l'an dernier peut tromper (en décembre dernier, les livraisons étaient déjà en difficulté). Remarquez que la limite haute des livraisons dépasse 100 % et la limite basse de la rupture est négative : **seule la limite pertinente compte** pour chaque indicateur (une alerte à sens unique).

Corrigé 2.8

```
dico = pd.DataFrame([
    ("ca", "somme des montants TTC de la semaine", "fait_ventes", "la gérante"),
    ("commandes", "commandes distinctes de la semaine", "fait_ventes", "la gérante"),
    ("panier", "CA ÷ commandes", "fait_ventes", "la gérante"),
    ("taux_marge", "marge HT ÷ CA HT (TVA 20 %)", "fait_ventes, dim_produit", "le directeur
    ↪ financier"),
    ("a_l_heure", "colis livrés dans la semaine sans retard ÷ colis livrés", "livraisons", "la
    ↪ responsable logistique"),
    ("conversion", "sessions avec commande ÷ sessions", "sessions_web", "la responsable du site"),
    ("rupture", "jours-produits en rupture ÷ jours-produits (20 produits)", "stock_quotidien", "la
    ↪ responsable logistique)],
    columns=["nom", "definition", "source", "proprietaire"]).set_index("nom")
page = ["ca", "commandes", "panier", "taux_marge", "a_l_heure", "conversion", "rupture"]
manquantes = [c for c in page if c not in dico.index or c not in hebdo.columns]
vides = dico.index[(dico == "").any(axis=1)].tolist()
assert not manquantes and not vides, (manquantes, vides)
print("dictionnaire complet :", len(dico), "fiches, aucune manquante ni vide")
```

dictionnaire complet : 7 fiches, aucune manquante ni vide

Le test échoue **bruyamment** (par assert) si l'on ajoute une colonne à la page sans documenter, ou si une fiche reste vide : on l'exécute à chaque modification.

Corrigé 2.9

```
import sqlite3
abime = pd.concat([faits, faits.sample(200, random_state=5)], ignore_index=True)
con = sqlite3.connect(":memory:")
abime.to_sql("t", con, index=False)
q = "SELECT SUM(montant), COUNT(DISTINCT id_commande) FROM (SELECT DISTINCT id_ligne, id_commande,
    ↪ montant, date FROM t) WHERE date >= '2025-12-22' AND date < '2025-12-29'"
ca_sql, n_sql = con.execute(q).fetchone()
s = abime[(abime["date"] >= "2025-12-22") & (abime["date"] < "2025-12-29")]
```

```
print("pandas sur la table abîmée :", round(s["montant"].sum(), 2), s["id_commande"].nunique(),
      ↪ round(s["montant"].sum() / s["id_commande"].nunique(), 2))
print("SQL dédoublé :", round(ca_sql, 2), n_sql, round(ca_sql / n_sql, 2))
print("lignes dupliquées :", int(abime.duplicated("id_ligne").sum()), "| clés de lignes concernées
      ↪ :", abime.loc[abime.duplicated("id_ligne", keep=False), "id_ligne"].nunique())
```

```
pandas sur la table abîmée : 44229.59 441 100.29
SQL dédoublé : 44167.99 441 100.15
lignes dupliquées : 200 | clés de lignes concernées : 200
```

Le chiffre d'affaires et le panier diffèrent entre les deux chemins, **mais le nombre de commandes ne change pas** (un comptage de valeurs distinctes ignore les doublons) : c'est un indice. La signature d'un doublon est une **clé de ligne qui n'est pas unique** : `duplicated("id_ligne")`. Le test d'unicité de la clé du modèle aurait suffi à arrêter la publication.

Corrigé 2.10

```
a = cumul_a_date(2025, "06-30"); b = cumul_a_date(2024, "12-31"); b2 = cumul_a_date(2024, "06-30")
print("contre l'année 2024 entière :", round(a / b - 1, 3), "| à période égale :", round(a / b2 - 1,
      ↪ 3))
t3 = lambda an: faits[(faits["date"] >= f"{an}-07-01") & (faits["date"] <=
      ↪ f"{an}-09-30")]["montant"].sum()
print("troisième trimestre :", round(t3(2025)), "contre", round(t3(2024)), "->", round(t3(2025) /
      ↪ t3(2024) - 1, 3))
```

```
contre l'année 2024 entière : -0.527 | à période égale : 0.077
troisième trimestre : 314572 contre 276375 -> 0.138
```

Le collègue a comparé six mois de 2025 aux **douze** mois de 2024 : l'écart de -52% vient du **dénominateur**, pas des ventes. À période égale (fin juin contre fin juin), l'évolution est de $+7,7\%$: le collègue a trouvé $-52,7\%$ en divisant par le total de 2024. Même principe pour le trimestre isolé : on compare le trimestre au **même trimestre** de l'an dernier. Le troisième trimestre rapporte 314 572 € contre 276 375 € ($+13,8\%$).

Corrigé 2.11

```
droits_f = droits.assign(region=droits["region"].where(droits["utilisateur"] != "resp_3", "Region
      ↪ 3"))
vue_f = lambda u: f25[f25["region"].isin(droits_f.loc[droits_f["utilisateur"] == u, "region"])]
print("resp_3 voit :", len(vue_f("resp_3")), "lignes,", round(vue_f("resp_3")["montant"].sum(), "€")
inconnues = sorted(set(droits_f["region"]) - set(star["dim_client"]["region"].dropna()))
print("régions de la table de droits absentes du modèle :", inconnues)
```

```
resp_3 voit : 0 lignes, 0 €
régions de la table de droits absentes du modèle : ['Region 3']
```

`resp_3` ne voit **rien**, sans aucun message d'erreur : le refus par défaut est sûr mais **silencieux**. Le contrôle compare l'ensemble des régions de la table de droits à celui du modèle. **Deux protections** : dans les **données**, une **liste de valeurs autorisées** pour `region` dans la table de droits (clé étrangère ou menu déroulant) plutôt qu'une saisie libre ; dans le **processus**, tester chaque rôle (« afficher en tant que ») avant publication et avoir un **contrôle automatique** de ce test à chaque actualisation.

Corrigé 2.12

Exemple pour le choix d'un outil de tableaux de bord dans une petite équipe, avec cinq critères (coût, facilité, gouvernance, intégration à la base existante, formation disponible) et trois options fictives.

```
notes = pd.DataFrame({"coût": [5, 2, 4], "facilité": [4, 3, 2], "gouvernance": [2, 5, 3],
↳ "intégration": [3, 4, 5], "formation": [4, 3, 2]}, index=["X", "Y", "Z"])
poids = pd.Series({"coût": 3, "facilité": 3, "gouvernance": 2, "intégration": 3, "formation": 1})
base = (notes @ poids).sort_values(ascending=False)
tir = np.random.default_rng(11).dirichlet(np.ones(5), 1000)
gagne = pd.Series(np.array(notes.index)[(tir @
↳ notes.to_numpy().T).argmax(axis=1)].value_counts(normalize=True).sort_index())
print(base.to_dict()); print((gagne * 100).round(1).to_dict())
```

```
{'X': 44, 'Z': 41, 'Y': 40}
{'X': 58.6, 'Y': 29.6, 'Z': 11.8}
```

La recommandation doit citer **à la fois** le classement avec vos poids et le **pourcentage de jeux de poids** où l'option arrive en tête : « X arrive en tête avec nos priorités, et reste première dans une majorité des 1 000 pondérations aléatoires ; elle n'est donc pas un choix fragile. Un essai de deux jours sur la même page, mesuré par la recette et le test de cinq secondes, confirmera ou infirmera ce classement avant tout engagement. »

Avec les poids donnés, X arrive en tête (44 points, contre 41 et 40) ; sur 1 000 jeux de poids aléatoires, X gagne 58,6 % des fois, Y 29,6 % et Z 11,8 % : le choix est **assez solide, pas certain**.

Chapitre 3 : Visualisation avec Python — exercices et applications

Ce chapitre du cahier prolonge le chapitre 3 du livre. Les **applications** sont de petites études guidées sur les données de la boutique : vous les refaites pas à pas, puis vous répondez aux **exercices**, dont les **corrigés** sont à la fin. Presque tous les exercices demandent de **produire** ou de **critiquer** un graphique, et la plupart des corrigés **montrent** la figure attendue. Les captures d'outils interactifs sont de vraies captures d'outils libres, lancés localement.

Une première cellule charge les bibliothèques et les données ; les cellules suivantes reprennent les noms ainsi définis.

```
import os, sys
sys.path.insert(0, "build")
import numpy as np, pandas as pd
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import seaborn as sns
import style, utils_ch03 as U
style.setup()

x = U.ventes()                # une ligne par ligne de commande : montant, date, canal, client,
    ↪ catégorie
j = U.jours()                 # une ligne par jour d'exploitation
liv = U.lire("livraisons.csv", parse_dates=["date_commande", "date_expedition", "date_livraison"])
print(len(x), "lignes de commande |", len(j), "jours |", len(liv), "livraisons")
```

83905 lignes de commande | 1096 jours | 19420 livraisons

Applications

Application 3.1 — Du graphique par défaut au graphique qui dit quelque chose (sections 3.1.1 et 3.1.2)

Objectif. Partir du graphique que matplotlib produit sans réglage, puis le transformer en graphique qui porte un message, en suivant les cinq retouches du livre.

Étape 1 — Le graphique par défaut. Le chiffre d'affaires 2025 par catégorie de produits, sans aucun réglage.

```
cat = x[x["annee"] == 2025].groupby("categorie")["montant"].sum().sort_values(ascending=False) / 1000
fig, ax = plt.subplots()
ax.bar(cat.sort_index().index, cat.sort_index().values)
```

```
0.sauver(fig, "ch03-cah-categories-default.png")
print(cat.sort_index().round(1).to_dict())
```

```
figure : ch03-cah-categories-default.png
{'Bien-être': 117.5, 'Cuisine': 233.3, 'Décoration': 258.7, 'Jardin': 354.0, 'Maison': 304.6,
↳ 'Papeterie': 56.6}
```

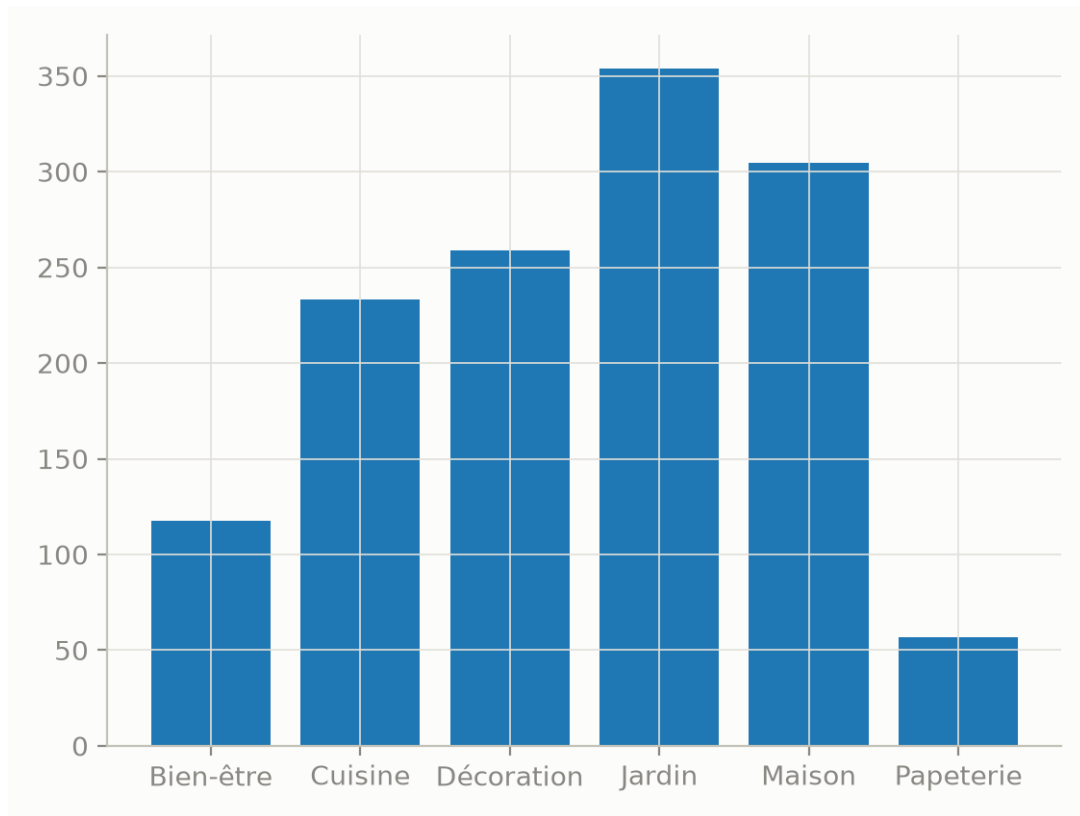


Figure 6: Le graphique par défaut : barres de la même couleur, dans l'ordre alphabétique.

Étape 2 — Les cinq retouches. Ordre décroissant, barres **horizontales** (les noms de catégories se lisent mieux), étiquettes directes, suppression de l'axe et du quadrillage, **une** couleur d'accent sur la catégorie qui compte, titre qui dit le message.

```
tri = cat.sort_values()                                # barres horizontales : la plus grande en
↳ haut
fig, ax = plt.subplots(figsize=(5.8, 3.3))
couleurs = [style.BLEU if c == tri.idxmax() else style.MUET for c in tri.index]
barres = ax.barh(tri.index, tri.values, color=couleurs, height=0.62)
ax.bar_label(barres, labels=[f"{fr(v, 0)} k€" for v in tri.values], padding=3)
ax.set(xlim=(0, tri.max() * 1.15), xticks=[]); ax.grid(False); ax.spines["bottom"].set_visible(False)
part = tri.max() / tri.sum() * 100
ax.set_title(f"{tri.idxmax()} est la première catégorie : {part:.0f} % du chiffre d'affaires 2025",
↳ loc="left", fontweight="bold", fontsize=10)
0.sauver(fig, "ch03-cah-categories-final.png")
print(tri.idxmax(), "|", round(part, 1), "% | rapport entre la première et la dernière :",
↳ round(tri.max() / tri.min(), 2))
```

```
figure : ch03-cah-categories-final.png
Jardin | 26.7 % | rapport entre la première et la dernière : 6.25
```

Étape 3 — Vérifier le titre. Le titre est calculé : il ne doit jamais mentir. Comptez, en une ligne, combien de catégories dépassent un sixième du chiffre d'affaires (la part d'une catégorie

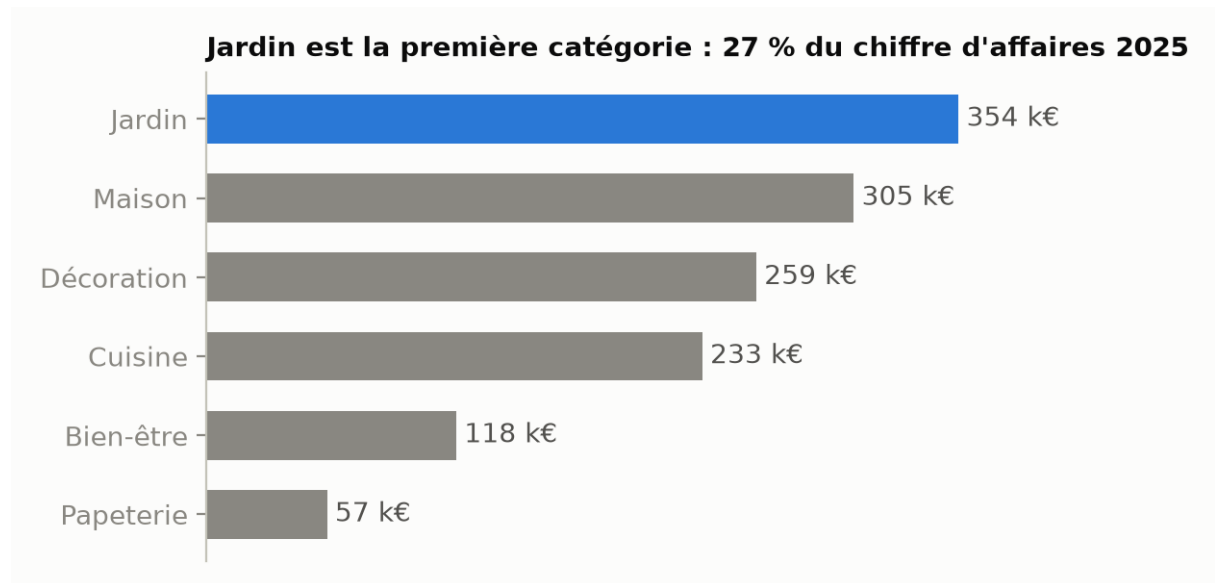


Figure 7: Les mêmes données après les cinq retouches.

« moyenne ») et dites si la catégorie mise en avant est vraiment une exception ou si les six catégories se ressemblent.

```
print((cat / cat.sum()).round(3).to_dict(), "| au-dessus d'un sixième :", int((cat / cat.sum() > 1 / 6).sum()))
```

```
{'Jardin': 0.267, 'Maison': 0.23, 'Décoration': 0.195, 'Cuisine': 0.176, 'Bien-être': 0.089, 'Papeterie': 0.043} | au-dessus d'un sixième : 4
```

Quatre catégories sur six dépassent un sixième du chiffre d'affaires : Jardin (27 %) est **la première**, devant Maison (23 %), mais elle ne « domine » pas. Le titre « première catégorie » est honnête ; « domine » ne le serait pas.

À retenir. Une couleur d'accent n'a de sens que si **un** élément se distingue vraiment ; si les barres sont proches, le message honnête est « elles se ressemblent », pas « une catégorie domine ».

Application 3.2 — Séries temporelles et petits multiples (sections 3.1.3 à 3.1.5)

Objectif. Comparer trois années mois par mois avec des étiquettes directes, puis voir ce que change l'échelle partagée dans des petits multiples.

Étape 1 — Trois années superposées. On met les mois de 1 à 12 en abscisse et une ligne par année ; l'étiquette de l'année est écrite au bout de la ligne.

```
mm = x.groupby(["annee", x["date_commande"].dt.month])[ "montant" ].sum().unstack(0) / 1000 # index :
↳ mois 1 à 12 ; colonnes : années
couleurs = {2023: style.MUET, 2024: style.ENCRE2, 2025: style.BLEU}
fig, ax = plt.subplots(figsize=(6.4, 3.3))
for a in mm.columns:
    ax.plot(mm.index, mm[a], color=couleurs[a], linewidth=2.2 if a == 2025 else 1.4)
    ax.text(12.15, mm[a].iloc[-1] + {2023: -5, 2024: 5, 2025: 0}[a], str(a), color=couleurs[a],
    ↳ va="center", fontweight="bold")
ax.set(xlim=(1, 12.9), xticks=range(1, 13), xlabel="mois", ylabel="k€")
croiss = mm[2025].sum() / mm[2024].sum() * 100 - 100
ax.set_title(f"2025 dépasse 2024 de {croiss:.0f} % et la forme saisonnière ne change pas", loc="left",
↳ fontweight="bold", fontsize=10)
0.sauver(fig, "ch03-cah-annees.png")
print({a: round(mm[a].sum()) for a in mm.columns}, "| meilleur mois 2025 :", mm[2025].idxmax())
```

figure : ch03-cah-annees.png

{2023: 1139, 2024: 1189, 2025: 1325} | meilleur mois 2025 : 12

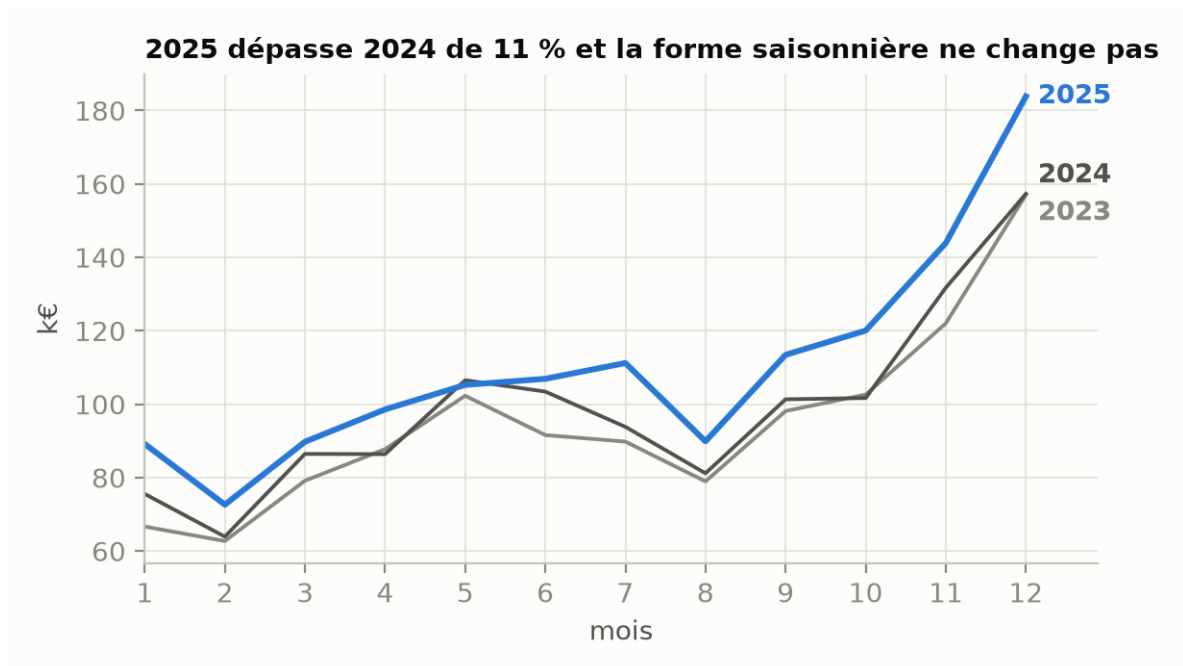


Figure 8: Trois années superposées, mois par mois.

Étape 2 — Des petits multiples, avec et sans échelle commune. En haut, chaque canal a sa échelle ; en bas, tous partagent la même.

```
m = x.groupby(["mois", "canal"])["montant"].sum().unstack()[0.CANAU] / 1000
t = m.index.to_timestamp()
fig, axes = plt.subplots(2, 3, figsize=(8, 4.4), sharex=True)
for k, canal in enumerate(0.CANAU):
    for l_ in (0, 1):
        axes[l_, k].plot(t, m[canal], color=0.COUL[canal])
        axes[0, k].set_title(canal, loc="left", color=0.COUL[canal], fontweight="bold")
        axes[1, k].set_ylim(0, m.values.max() * 1.05)
        axes[1, k].xaxis.set_major_locator(mdates.YearLocator()); axes[1,
        ↪ k].xaxis.set_major_formatter(mdates.DateFormatter("%Y"))
axes[0, 0].set_ylabel("échelle libre (k€)"); axes[1, 0].set_ylabel("échelle commune (k€)")
0.sauver(fig, "ch03-cah-multiples.png")
print("maximum mensuel (k€) :", {c: round(m[c].max()) for c in 0.CANAU})
```

figure : ch03-cah-multiples.png

maximum mensuel (k€) : {'Boutique': 81, 'Site': 90, 'Réseaux': 20}

À retenir. Le haut répond à « quelle **forme** a chaque canal ? » (la saison est visible partout), le bas répond à « **combien** pèse chaque canal ? » (les Réseaux sont petits). On ne choisit pas au hasard : on choisit selon la question.

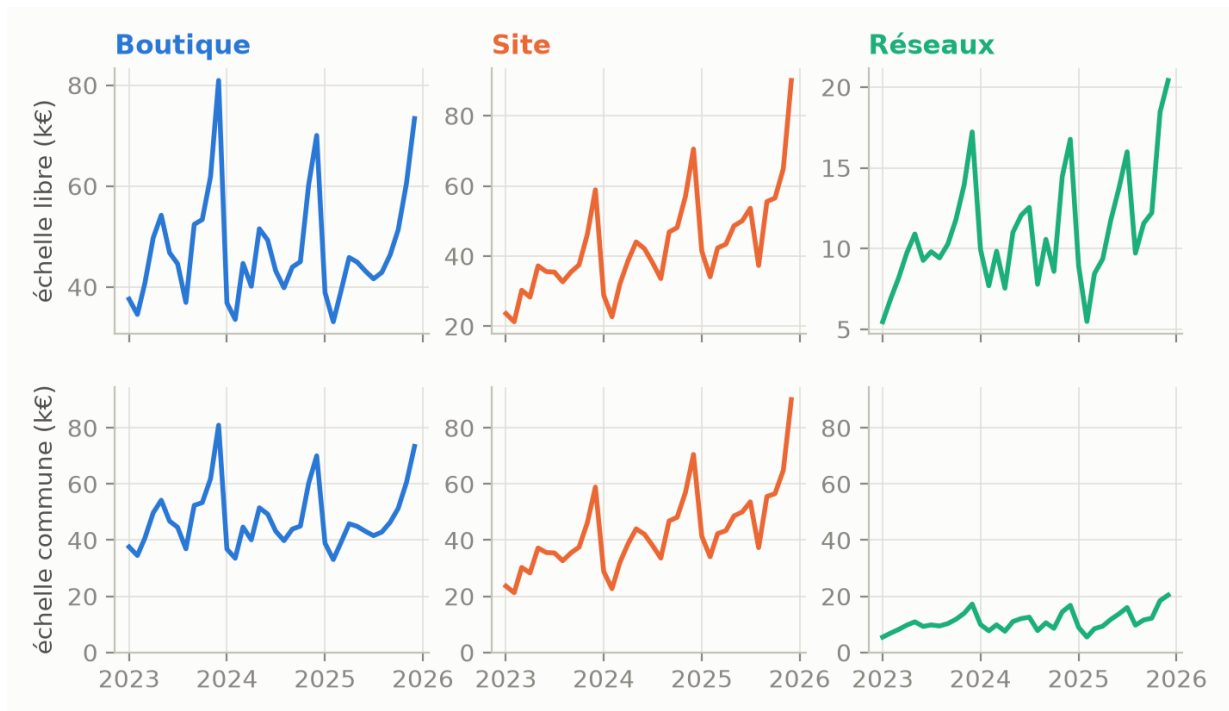


Figure 9: Petits multiples : échelle libre en haut, échelle commune en bas.

Application 3.3 — Un thème, un contraste, un format (section 3.1.6)

Objectif. Ranger les réglages dans un thème appliqué localement, mesurer le contraste de la palette, et choisir un format d'enregistrement.

Étape 1 — Un thème local. `plt.rc_context` applique des réglages **le temps d'un bloc**, sans modifier le reste du script.

```
theme = {"axes.spines.top": False, "axes.spines.right": False, "axes.grid": True, "grid.color":
↳ style.GRILLE,
        "axes.titleweight": "bold", "axes.titlelocation": "left", "legend.frameon": False,
        ↳ "axes.axisbelow": True}
ca = 0.ca_canal(x) / 1000
with plt.rc_context(theme):
    fig, ax = plt.subplots(figsize=(4.6, 2.6))
    ax.bar(ca.index, ca.values, color=[0.COUL[c] for c in ca.index], width=0.6)
    ax.set_title("Chiffre d'affaires 2025 (k€)")
0.sauver(fig, "ch03-cah-theme.png")
print("réglages du thème :", len(theme), "| grille après le bloc :", plt.rcParams["axes.grid"])
```

```
figure : ch03-cah-theme.png
réglages du thème : 8 | grille après le bloc : True
```

Étape 2 — Mesurer le contraste. La fonction du livre donne le rapport de contraste ; on l'applique aux couleurs de la palette **sur le fond blanc** et sur le fond du livre, et l'on cherche, pour l'aqua, une version assez foncée pour du **texte** (rapport d'au moins 4,5).

```
def lum(h):
    c = [int(h[i:i + 2], 16) / 255 for i in (1, 3, 5)]
    c = [v / 12.92 if v <= 0.03928 else ((v + 0.055) / 1.055) ** 2.4 for v in c]
    return 0.2126 * c[0] + 0.7152 * c[1] + 0.0722 * c[2]

def contraste(a, b):
    la, lb = lum(a), lum(b)
    return (max(la, lb) + 0.05) / (min(la, lb) + 0.05)
```

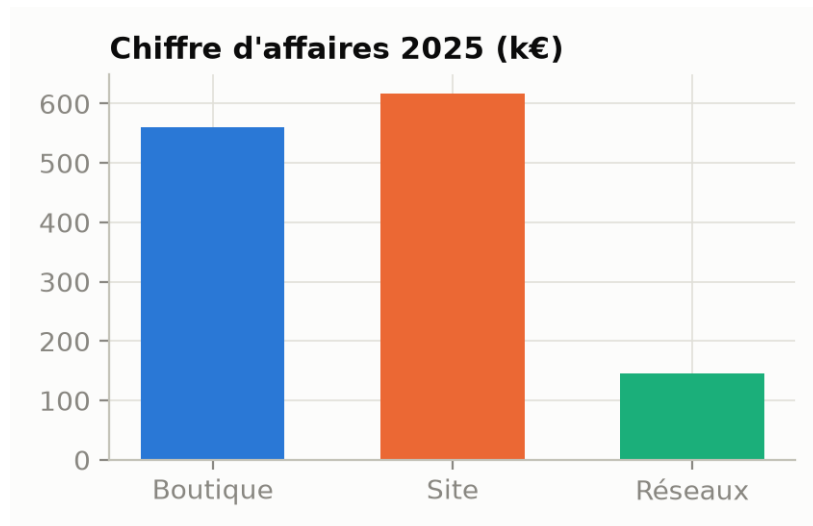


Figure 10: Un graphique tracé avec le thème local.

```
def assombrir(h, f):
    return "#" + "".join(f"{int(int(h[i:i + 2], 16) * f):02x}" for i in (1, 3, 5))

noms = ("BLEU", "ORANGE", "AQUA", "VIOLET", "ROUGE")
print({n: (round(contraste(getattr(style, n), "#ffffff"), 1), round(contraste(getattr(style, n),
    ↳ style.SURFACE), 1)) for n in noms})
f = next(f for f in np.arange(1.0, 0.2, -0.02) if contraste(assombrir(style.AQUA, f), style.SURFACE)
    ↳ >= 4.5)
print("aqua assombri pour du texte :", assombrir(style.AQUA, f), "| facteur", round(f, 2), "|
    ↳ contraste", round(contraste(assombrir(style.AQUA, f), style.SURFACE), 1))

{'BLEU': (4.4, 4.3), 'ORANGE': (3.2, 3.1), 'AQUA': (2.8, 2.7), 'VIOLET': (8.6, 8.3), 'ROUGE': (4.0,
    ↳ 3.9)}
aqua assombri pour du texte : #14845c | facteur 0.76 | contraste 4.6
```

Étape 3 — Choisir un format. On enregistre le même nuage de 20 000 points en PNG, en SVG et en PDF, avec et sans rasterized=True pour le vectoriel.

```
rng = np.random.default_rng(0)
px_, py_ = rng.normal(size=20000), rng.normal(size=20000)
dossier = 0.dossier_temp()
tailles = {}
for nom, ras in (("png", False), ("svg", False), ("svg rasterisé", True), ("pdf", False)):
    fig, ax = plt.subplots(figsize=(4, 3)); ax.scatter(px_, py_, s=3, rasterized=ras)
    chemin = os.path.join(dossier, "nuage." + nom.split()[0]); fig.savefig(chemin, dpi=200);
    ↳ plt.close(fig)
    tailles[nom] = os.path.getsize(chemin) // 1024
0.supprimer_dossier(dossier)
print("taille en Ko :", tailles)
```

```
taille en Ko : {'png': 46, 'svg': 2090, 'svg rasterisé': 46, 'pdf': 300}
```

À retenir. Un vectoriel pèse autant que le nombre d'objets : pour un nuage dense, on **rasterise** les points tout en gardant les axes et les textes vectoriels.

Application 3.4 — seaborn : distribution, carte thermique et intervalle (section 3.1.7)

Objectif. Comparer les délais de livraison des trois transporteurs, repérer les retards par mois, puis distinguer l'incertitude d'une moyenne de la dispersion des observations.

Étape 1 — Les délais par transporteur. Le délai est le nombre de jours entre l'expédition et la livraison.

```
liv["delai"] = (liv["date_livraison"] - liv["date_expedition"]).dt.days
g = sns.catplot(data=liv, x="transporteur", y="delai", kind="box", hue="transporteur",
               ↪ palette=[style.BLEU, style.ORANGE, style.AQUA], height=3.2, aspect=1.5, fliersize=2)
g.set_axis_labels("", "délai (jours)")
O.sauver(g.figure, "ch03-cah-delais.png")
print(liv.groupby("transporteur")["delai"].agg(["median", "mean", "max"]).round(1).to_string())
```

```
figure : ch03-cah-delais.png
              median mean max
transporteur
Transporteur A    3.0  3.6  10
Transporteur B    4.0  4.2  10
Transporteur C    5.0  5.0  10
```

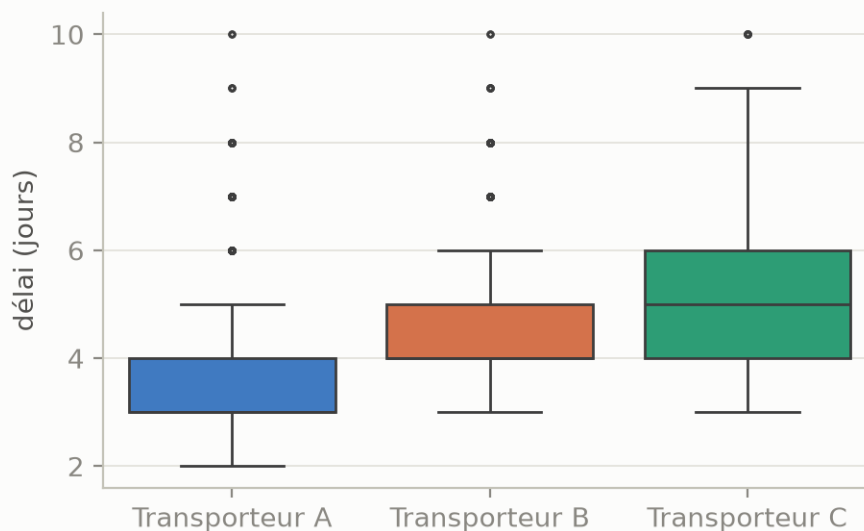


Figure 11: Délais de livraison par transporteur (boîtes à moustaches).

Étape 2 — Le taux de retard par mois et transporteur. Une carte thermique avec les valeurs écrites dans les cases.

```
retard = liv.pivot_table(index="transporteur", columns=liv["date_commande"].dt.month,
                        ↪ values="retard", aggfunc="mean") * 100
fig, ax = plt.subplots(figsize=(7.4, 2.4))
sns.heatmap(retard, annot=True, fmt=".0f", cmap=style.SEQ, cbar=False, linewidths=0.5, ax=ax)
ax.set(xlabel="mois de la commande", ylabel=""); ax.grid(False); ax.set_title("Part des livraisons en
↪ retard (%) : décembre ressort", loc="left", fontweight="bold", fontsize=10)
O.sauver(fig, "ch03-cah-retards.png")
pire, mieux = retard.stack().idxmax(), retard.stack().idxmin()
print(f"pire case : {pire[0]}, mois {int(pire[1])} ({retard.stack().max():.0f} %) | meilleure :
↪ {mieux[0]}, mois {int(mieux[1])} ({retard.stack().min():.0f} %)")
```

```
figure : ch03-cah-retards.png
pire case : Transporteur C, mois 12 (85 %) | meilleure : Transporteur A, mois 4 (10 %)
```

Étape 3 — Un intervalle n'est pas une dispersion. `sns.barplot` trace par défaut un intervalle de

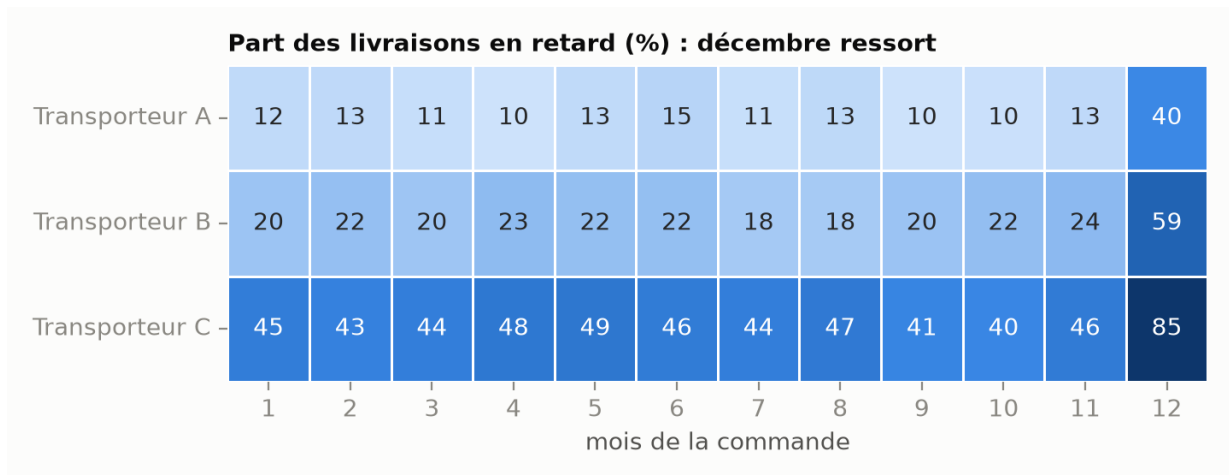


Figure 12: Taux de retard par transporteur et par mois.

confiance de la moyenne ; avec `errorbar="sd"`, il trace l'écart-type. On lit les deux sur un même graphique.

```
fig, (g1, g2) = plt.subplots(1, 2, figsize=(7, 2.9), sharey=True)
sns.barplot(data=liv, x="transporteur", y="delai", color=style.BLEU, ax=g1)
sns.barplot(data=liv, x="transporteur", y="delai", color=style.BLEU, errorbar="sd", ax=g2)
g1.set_title("intervalle de confiance de la moyenne"); g2.set_title("écart-type des livraisons");
↪ g1.set_ylabel="délai (jours)"; g2.set_ylabel=""
for a in (g1, g2):
    a.set_xticks(range(3), ["A", "B", "C"]); a.set_xlabel("transporteur")
0.sauver(fig, "ch03-cah-ic-sd.png")
s = liv.groupby("transporteur")["delai"].agg(["mean", "std", "count"])
print((1.96 * s["std"] / np.sqrt(s["count"])).round(2).to_dict(), "| écarts-types :",
↪ s["std"].round(2).to_dict())
```

figure : ch03-cah-ic-sd.png

```
{'Transporteur A': 0.02, 'Transporteur B': 0.02, 'Transporteur C': 0.03} | écarts-types :
↪ {'Transporteur A': 1.02, 'Transporteur B': 1.0, 'Transporteur C': 1.0}
```

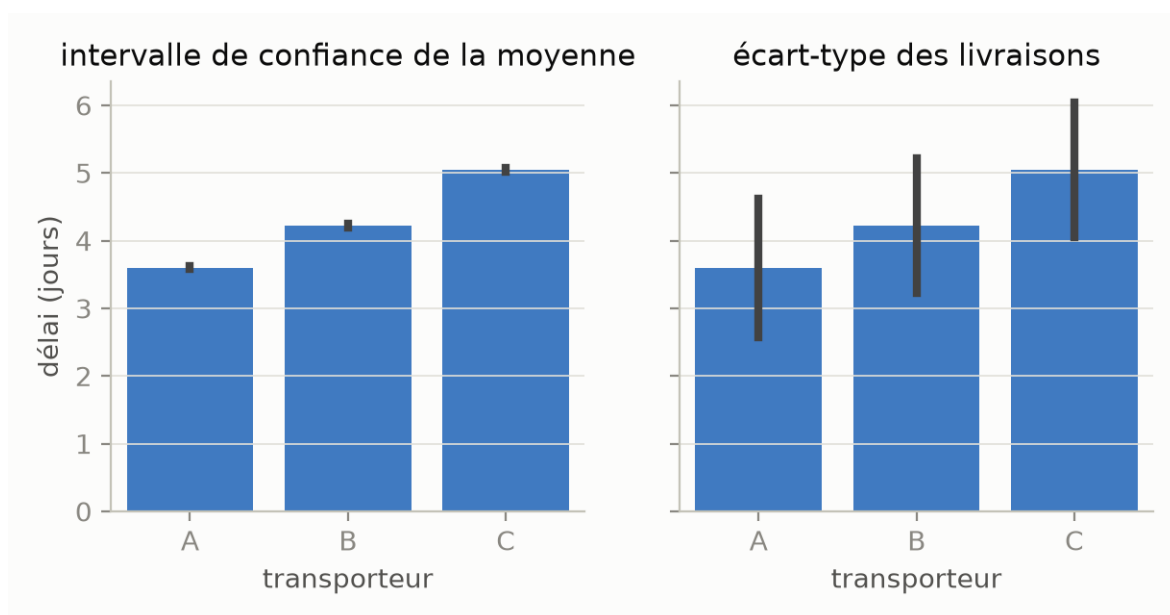


Figure 13: À gauche l'intervalle de la moyenne (minuscule), à droite l'écart-type (large).

À retenir. Avec près de 20 000 livraisons, l'intervalle de la moyenne est minuscule : il dit que la moyenne est précise, pas que tous les colis arrivent au même moment. La légende doit nommer ce que la barre d'erreur représente.

Application 3.5 — Une figure testée, et la même en R (sections 3.1.8 et 3.1.9)

Objectif. Enfermer un graphique dans une fonction, tester les données qu'il dessine, puis l'écrire en ggplot2.

Étape 1 — La fonction et son test. La fonction reçoit le tableau des jours et une année, et retourne la figure du chiffre d'affaires mensuel.

```
def figure_mensuelle(jours, annee):
    d = jours[jours["annee"] == annee].groupby("mois")["chiffre_affaires"].sum() / 1000
    fig, ax = plt.subplots(figsize=(5.6, 3))
    ax.plot(d.index, d.values, color=style.BLEU, marker="o")
    ax.set(xticks=range(1, 13), ylabel="k€"); ax.set_title(f"Chiffre d'affaires {annee} :
    ↳ {fr(d.sum(), 0)} k€", loc="left", fontweight="bold")
    return fig

fig = figure_mensuelle(j, 2025)
tracé = fig.axes[0].lines[0].get_ydata()
attendu = j[j["annee"] == 2025].groupby("mois")["chiffre_affaires"].sum().to_numpy() / 1000
print("points tracés :", len(tracé), "| égaux au tableau :", np.allclose(tracé, attendu), "| titre :",
    ↳ fig.axes[0].get_title(loc="left"))
O.sauver(fig, "ch03-cah-test-mensuel.png")
```

```
points tracés : 12 | égaux au tableau : True | titre : Chiffre d'affaires 2025 : 1 325 k€
figure : ch03-cah-test-mensuel.png
```

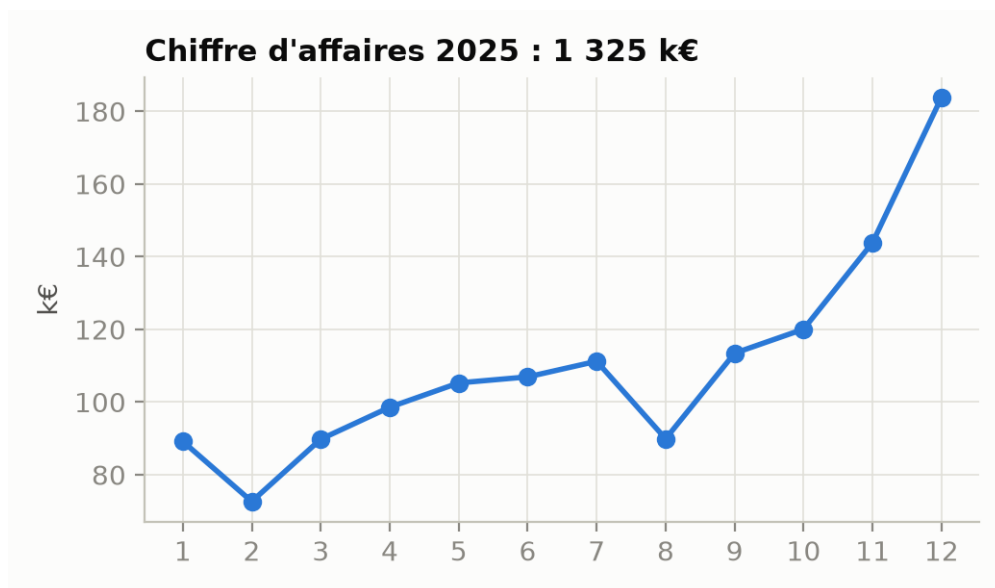


Figure 14: La figure produite par la fonction testée.

Étape 2 — La même chose en R. On refait la série en ggplot2 et l'on compare le total au total Python.

```
library(ggplot2)
jr <- read.csv(file.path(Sys.getenv("DONNEES"), "jours_exploitation.csv"))
jr$annee <- as.integer(substr(jr$date, 1, 4)); jr$mois <- as.integer(substr(jr$date, 6, 7))
d <- aggregate(chiffre_affaires ~ mois, data = jr[jr$annee == 2025, ], FUN = function(v) sum(v) / 1000)
p <- ggplot(d, aes(mois, chiffre_affaires)) + geom_line(color = "#2a78d6") + geom_point(color =
    ↳ "#2a78d6") +
```

```
scale_x_continuous(breaks = 1:12) + labs(x = NULL, y = "k€") + theme_minimal()
ggsave(file.path(dirname(Sys.getenv("DONNEES")), "figures", "ch03-cah-ggplot-mensuel.png"), p, width
  ↪ = 5.6, height = 3, dpi = 200)
cat("total 2025 (k€) :", round(sum(d$chiffre_affaires), 1), "\n")
```

```
total 2025 (k€) : 1324.8
```

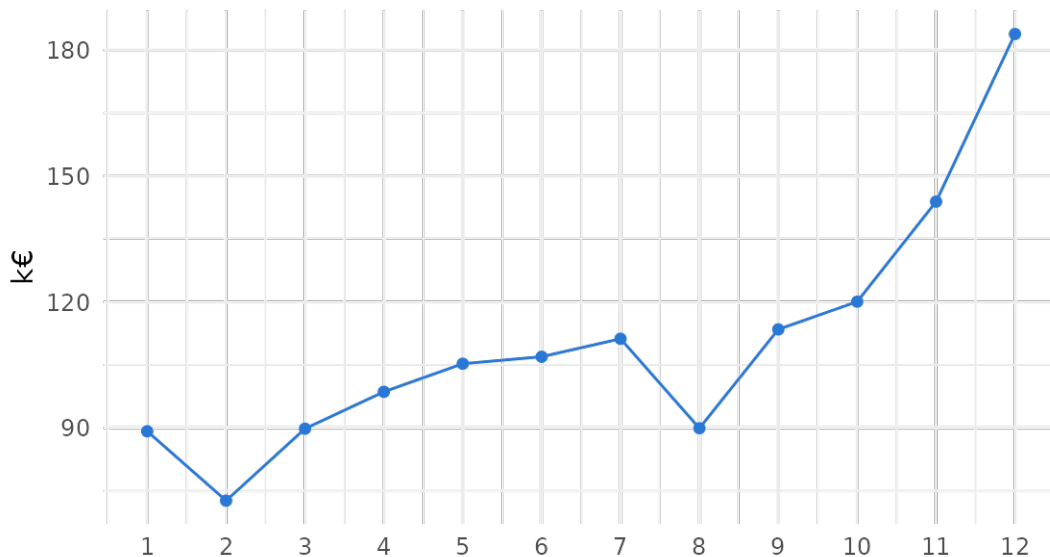


Figure 15: La même série tracée avec ggplot2.

```
print("total 2025 (k€) côté Python :", round(attendu.sum(), 1))
```

```
total 2025 (k€) côté Python : 1324.8
```

À retenir. Les deux langages donnent le **même total** : le test porte sur les nombres, pas sur l'apparence des deux figures.

Application 3.6 — plotly : une page interactive (section 3.2)

Objectif. Construire une figure plotly avec une infobulle utile, vérifier sa structure, mesurer le poids de la page et la photographier.

```
import plotly.express as px
cc = x[x["annee"] == 2025].groupby(["categorie", "canal", as_index=False)["montant"].sum()
fig = px.bar(cc, x="categorie", y="montant", color="canal", color_discrete_map=0.COUL,
  ↪ template="simple_white", category_orders={"canal": 0.CANAU})
fig.update_traces(hovertemplate="<b>{x}</b> - {fullData.name}<br>{y:,.0f} €<extra></extra>")
fig.update_layout(separators=", ", xaxis_title=None, yaxis_title="€, 2025", legend_title=None,
  ↪ title="Chiffre d'affaires 2025 par catégorie et par canal")
dossier = 0.dossier_temp()
for nom, mode in (("autonome", True), ("cdn", "cdn")):
    chemin = os.path.join(dossier, nom + ".html"); fig.write_html(chemin, include_plotlyjs=mode)
    print(nom, ":", round(os.path.getsize(chemin) / 1024), "Ko")
print("séries :", [t.name for t in fig.data], "| une série par canal :", len(fig.data) == 3)
0.capturer_html(os.path.join(dossier, "autonome.html"), os.path.join(0.FIG, "ch03-cah-plotly.png"),
  ↪ 900, 460, survol=".bars .point >> nth=4")
0.supprimer_dossier(dossier)
```

```
autonome : 4713 Ko
```

```
cdn : 11 Ko
```

```
séries : ['Boutique', 'Site', 'Réseaux'] | une série par canal : True
```

capture existante : ch03-cah-plotly.png

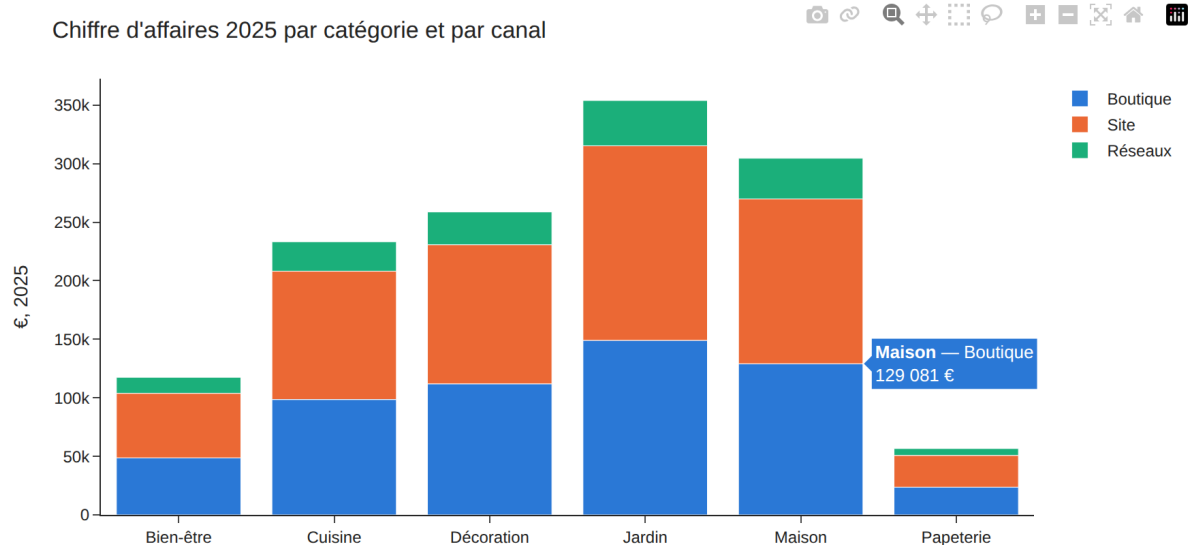


Figure 16: Capture réelle de la page plotly : infobulle d'une barre empilée.

Les barres sont **empilées** par défaut (`barmode="relative"`) : la hauteur totale se lit bien, la comparaison d'un canal d'une catégorie à l'autre beaucoup moins, sauf pour le canal du bas. Essayez `barmode="group"` et dites ce que vous gagnez et ce que vous perdez (exercice 3.11).

Application 3.7 — Streamlit et Dash : tester sans navigateur (section 3.3)

Objectif. Écrire une toute petite application Streamlit, la tester avec AppTest, puis tester de la même façon la fonction de rappel d'une application Dash.

Étape 1 — Streamlit. L'application a un sélecteur de catégorie et deux chiffres.

```
from streamlit.testing.v1 import AppTest
dossier = 0.dossier_temp()
code = '''import os, pandas as pd, streamlit as st
d = os.environ["DONNEES"]
l = pd.read_csv(f"{d}/lignes_commande.csv").merge(pd.read_csv(f"{d}/produits.csv"))[["id_produit",
    "categorie"]], on="id_produit")
cat = st.selectbox("Catégorie", sorted(l["categorie"].unique()))
v = l[l["categorie"] == cat]
st.metric("Chiffre d'affaires (€)", f"{v['montant'].sum():,.0f}".replace(",", " "))
st.metric("Lignes vendues", len(v))
'''
app = os.path.join(dossier, "mini.py"); open(app, "w").write(code)
at = AppTest.from_file(app, default_timeout=120).run()
print("première catégorie :", at.selectbox[0].value, [m.value for m in at.metric])
at.selectbox[0].select("Jardin").run()
lj = x[x["categorie"] == "Jardin"]
print("Jardin :", [m.value for m in at.metric], "| attendu :", fr(lj["montant"].sum(), 0), len(lj))
```

```
première catégorie : Bien-être ['324 037', '10347']
Jardin : ['970 379', '14977'] | attendu : 970 379 14977
```

Étape 2 — Dash. On déclare une application avec une fonction de rappel, puis on appelle la fonction directement.

```

from dash import Dash, dcc, html, Input, Output
ap = Dash(__name__)
ap.layout = html.Div([dcc.Dropdown(["Boutique", "Site", "Réseaux"], "Site", id="canal"),
↳ html.Div(id="sortie")])

@ap.callback(Output("sortie", "children"), Input("canal", "value"))
def maj(canal):
    return f"{canal} : {x[(x['canal'] == canal) & (x['annee'] == 2025)]['montant'].sum():,.0f}
↳ €.replace(","," ")

print(maj("Site"), "|", maj("Réseaux"))
print("attendu :", fr(0.ca_canal(x)["Site"], 0), "|", fr(0.ca_canal(x)["Réseaux"], 0))
0.supprimer_dossier(dossier)

```

```

Site : 617 715 € | Réseaux : 146 074 €
attendu : 617 715 | 146 074

```

À retenir. Dans les deux cas, le test vérifie le **calcul** sans lancer de serveur ni de navigateur ; la page elle-même se regarde à l'œil (exercice 3.14).

Application 3.8 — Shiny : un graphe réactif qui ne recalcule qu'une fois (section 3.3.4)

Objectif. Vérifier avec `testServer` qu'une expression reactive est calculée **une seule fois** même si deux sorties la lisent.

```

library(shiny)
d <- Sys.getenv("DONNEES")
l <- merge(read.csv(file.path(d, "lignes_commande.csv")), read.csv(file.path(d, "produits.csv"))[,
↳ c("id_produit", "categorie")], by = "id_produit")
calculs <- 0
serveur <- function(input, output, session) {
  sel <- reactive({ calculs <- calculs + 1; l[l$categorie == input$categorie, ] })
  output$ca <- renderText(format(round(sum(sel())$montant)), big.mark = " "))
  output$n <- renderText(nrow(sel()))
}
testServer(serveur, {
  session$setInputs(categorie = "Jardin")
  cat("chiffre d'affaires :", output$ca, "| lignes :", output$n, "| calculs du filtre :", calculs,
↳ "\n")
  session$setInputs(categorie = "Cuisine")
  cat("chiffre d'affaires :", output$ca, "| lignes :", output$n, "| calculs du filtre :", calculs,
↳ "\n")
})

```

```

chiffre d'affaires : 970 379 | lignes : 14977 | calculs du filtre : 1
chiffre d'affaires : 658 996 | lignes : 14750 | calculs du filtre : 2

```

Deux sorties lisent `sel()`, mais le filtre n'est calculé **qu'une fois par changement de catégorie** : c'est ce que « graphe réactif » veut dire.

Application 3.9 — Cartes sur un plan fictif (section 3.4)

Objectif. Comparer les villes par nombre de clients et par nombre de clients pour 1 000 habitants, avec des cercles proportionnels, et voir ce que change la normalisation.

```

v = 0.ventes_villes(x)
v["clients"] = 0.lire("clients.csv").groupby("ville").size()
v["clients_1000"] = v["clients"] / v["habitants"] * 1000
fig, axes = plt.subplots(1, 2, figsize=(9, 3.9))
for ax, col, titre in zip(axes, ("clients", "clients_1000"), ("nombre de clients", "clients pour 1 000
↳ habitants")):

```

```

ax.scatter(v["x_km"], v["y_km"], s=v[col] / v[col].max() * 700, color=style.BLEU, alpha=0.55,
↪ edgecolor="white")
for ville, r in v.nlargest(3, col).iterrows():
    ax.annotate(ville, (r["x_km"], r["y_km"]), ha="center", va="center", fontsize=8)
ax.set(xlim=(-5, 125), ylim=(-5, 95), aspect="equal", xticks=[], yticks=[]); ax.grid(False);
↪ ax.set_title(titre, loc="left", fontsize=10)
O.sauver(fig, "ch03-cah-cartes.png")
print("trois premières villes :", list(v.nlargest(3, "clients").index), "| par 1 000 habitants :",
↪ list(v.nlargest(3, "clients_1000").index))

```

figure : ch03-cah-cartes.png

trois premières villes : ['Ville A', 'Ville B', 'Ville C'] | par 1 000 habitants : ['Ville C', 'Ville
↪ D', 'Ville A']

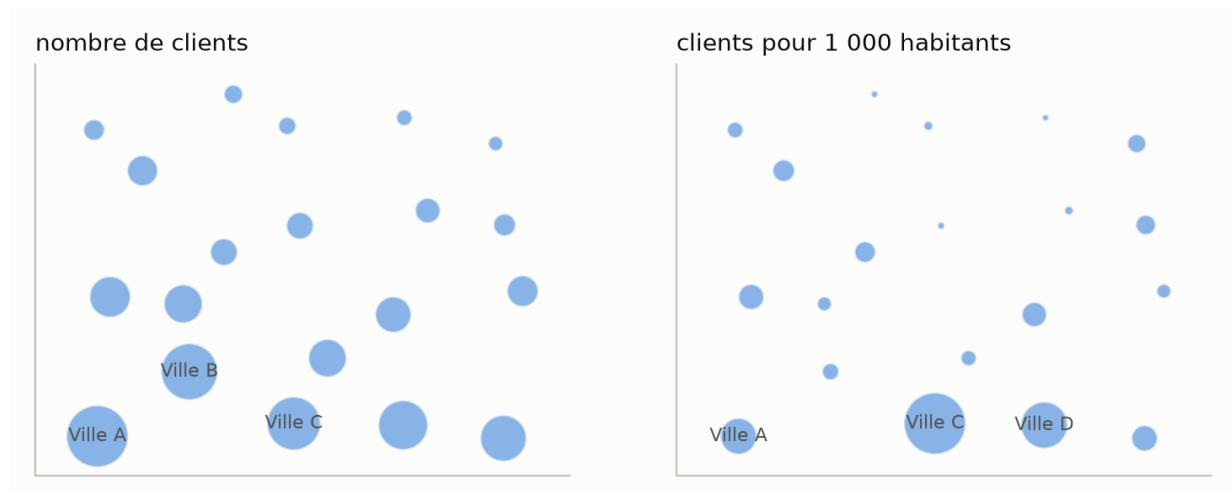


Figure 17: Cercles proportionnels : à gauche le nombre de clients, à droite la densité de clients.

Ville B sort du podium quand on normalise (Ville D y entre) : le nombre de clients suit la population, la densité mesure la **pénétration** de la boutique.

Exercices

Exercice 3.1 ★ — Trois couches (section 3.1.1)

Un graphique montre, pour chacun des 120 produits, le prix de vente (abscisse) et la quantité vendue en 2025 (ordonnée) ; les points sont colorés par catégorie et leur taille est proportionnelle au chiffre d'affaires du produit. Décrivez ce graphique selon les **trois couches** : données, esthétiques, géométrie. Combien d'attributs visuels sont utilisés ?

Exercice 3.2 ★ — Critiquer et refaire (section 3.1.2)

Le code ci-dessous trace le chiffre d'affaires par canal avec cinq défauts, tous traités dans la section 3.1.2. **Listez les cinq défauts**, puis **refaites** le graphique en les corrigeant.

```

ca = x[x["annee"] == 2025].groupby("canal")["montant"].sum()
fig, ax = plt.subplots()
ax.bar(ca.index, ca.values, color=["red", "green", "blue"])
ax.set_ylim(100000, ca.max() * 1.02)
ax.set_title("Graphique 1")

```

Exercice 3.3 ★★ — Un titre qui dit le message, et qui reste vrai (section 3.1.3)

Pour la série mensuelle du chiffre d'affaires de 2025, écrivez en une phrase le message que vous voulez faire passer. Puis écrivez le titre **calculé** (avec f"...") et ajoutez une **assertion** qui échoue si le message cesse d'être vrai (par exemple si le mois de pointe change). Pourquoi est-ce important quand le script sera rejoué sur les données du mois suivant ?

Exercice 3.4 ★★ — Échelle commune ou libre ? (section 3.1.5)

Tracez le chiffre d'affaires 2025 de chacune des six catégories en six petits graphiques (par mois), d'abord avec `sharey=True`, puis avec `sharey=False`. Pour chaque version, écrivez la **question** à laquelle elle répond le mieux.

Exercice 3.5 ★ — Un orange lisible (section 3.1.6)

Le rapport de contraste de l'orange de la palette sur le fond du livre est de 3,1 : suffisant pour une barre, insuffisant pour du texte (qui demande 4,5). Trouvez une version **assombrie** de l'orange dont le contraste dépasse 4,5, et dites pourquoi on n'écrit pas toutes les étiquettes avec la couleur pure de la série.

Exercice 3.6 ★★ — Quel format pour quel usage ? (section 3.1.6)

Pour chacun des quatre usages suivants, choisissez PNG, SVG ou PDF et justifiez en une phrase : (a) une figure à insérer dans un courriel à la direction ; (b) un nuage de 200 000 points dans un rapport imprimé ; (c) une figure simple à agrandir sur une affiche ; (d) une figure pour une page web.

Exercice 3.7 ★★ — Écrire la légende d'un intervalle (section 3.1.7)

Avec `sns.lineplot`, tracez la **moyenne mensuelle du nombre de commandes par jour** en 2025 (une valeur par jour, groupée par mois), avec la bande d'incertitude par défaut. Écrivez la **légende** qui dit ce que représente la bande, puis une seconde version avec `errorbar="sd"` et sa légende. Laquelle répond à « de combien varient les jours d'un même mois ? »

Exercice 3.8 ★★ — Le test qui attrape une erreur invisible (section 3.1.8)

La fonction ci-dessous trie les valeurs mais pas les étiquettes : le graphique est **faux sans que rien ne se voie** à l'exécution. Écrivez le test qui l'attrape, puis corrigez la fonction.

```
def barres_canaux(ca):
    fig, ax = plt.subplots()
    ax.bar(ca.index, ca.sort_values(ascending=False).values)
    return fig
```

Exercice 3.9 ★★ — De matplotlib à ggplot2 (section 3.1.9)

Traduisez en R avec `ggplot2` un histogramme des paniers 2025 (par commande), **un par canal** (trois petits histogrammes sur la même échelle), avec une ligne verticale à la médiane. Quel équivalent de `sharey=True` utilise-t-on ?

Exercice 3.10 ★ — Même figure, deux niveaux (section 3.2.1)

Construisez avec `plotly.graph_objects` la courbe du chiffre d'affaires mensuel du canal Site. Comparez avec la version `plotly.express` : combien de séries dans chacune ? Quelle clé de `fig.to_dict()` contient la mise en page ?

Exercice 3.11 ★★ — Une infobulle utile (section 3.2.2)

Écrivez un `hovertemplate` qui affiche, pour chaque barre du chiffre d'affaires par catégorie, le nom de la catégorie, le chiffre d'affaires en k€ et **sa part dans le total**. Vérifiez que les parts de toutes les barres somment à 100 %. Essayez ensuite `barmode="group"` pour la figure de l'application 3.6 et dites ce que vous gagnez et ce que vous perdez.

Exercice 3.12 ★★ — Interactif ou statique ? (section 3.2.3)

Pour chaque situation, dites si une figure interactive est un bon choix, et pourquoi : (a) un rapport mensuel envoyé en PDF ; (b) un tableau de bord consulté chaque lundi par la gérante ; (c) une diapositive projetée en réunion ; (d) une page d'exploration pour l'analyste qui cherche les jours aberrants.

Exercice 3.13 ★★ — Que rejoue Streamlit ? (section 3.3.2)

Une application Streamlit charge un fichier de 200 Mo (sans cache), affiche un sélecteur, puis un graphique filtré. L'utilisateur change quatre fois le sélecteur. Combien de fois le fichier est-il lu ? Que change `@st.cache_data` ? Qu'est-ce que ce modèle d'exécution a de plus simple et de plus coûteux que celui de Shiny ?

Exercice 3.14 ★★★ — Le cas limite (section 3.3.3)

La fonction de rappel de l'application 3.7 (Dash) est appelée avec une liste de canaux. Écrivez une version qui reçoit une **liste vide** (l'utilisateur a tout décoché) : que se passe-t-il avec une version naïve ? Écrivez le test qui l'attrape, puis la version robuste qui affiche un message plutôt que de planter.

Exercice 3.15 ★★ — Normaliser une carte (section 3.4)

Classez les villes par chiffre d'affaires 2025 puis par chiffre d'affaires **par habitant**. Combien de villes figurent dans les cinq premières des deux classements ? Quelle conclusion un lecteur de chaque classement tirerait-il ?

Exercice 3.16 ★★★ — Le piège de la moyenne des taux (section 3.4)

On veut le chiffre d'affaires par habitant de chacune des quatre régions. Calculez-le de **deux façons** : la moyenne des rapports de chaque ville, et le rapport des totaux (chiffre d'affaires de la région divisé par ses habitants). Les résultats diffèrent-ils ? Laquelle est la bonne, et pourquoi ? Faut-il une carte ou des barres pour présenter le résultat ?

Corrigés

Corrigé 3.1

- **Données** : un tableau de 120 lignes (une par produit) avec le prix, la quantité vendue, la catégorie et le chiffre d'affaires.
- **Esthétiques** : prix → position horizontale ; quantité → position verticale ; catégorie → couleur ; chiffre d'affaires → taille. **Quatre attributs visuels.**
- **Géométrie** : des points (un par produit).

Le graphique est lisible mais chargé : la taille s'estime mal (voir le chapitre 1, section 1.1, sur la comparaison des aires). Si la taille ne sert pas la question posée, mieux vaut la retirer.

Corrigé 3.2

Les cinq défauts : (1) des couleurs **sans sens** (rouge, vert, bleu : le rouge et le vert se confondent pour un daltonien, et aucune ne relie le canal à sa couleur du reste du document) ; (2) un **axe tronqué** qui commence à 100 000 : les barres ne partent plus de zéro et exagèrent les écarts ; (3) des barres **non triées** (ordre alphabétique) ; (4) **aucun chiffre** sur les barres ni unité sur l'axe ; (5) un titre qui **décrit** au lieu de dire le message (« Graphique 1 »). Le cadre et le quadrillage superflus sont un sixième défaut mineur.

```
ca = x[x["annee"] == 2025].groupby("canal")["montant"].sum().sort_values(ascending=False) / 1000
fig, ax = plt.subplots(figsize=(5.6, 3.2))
barres = ax.bar(ca.index, ca.values, color=[0.COUL[c] for c in ca.index], width=0.62)
ax.bar_label(barres, labels=[f"{fr(v, 0)} k€" for v in ca.values], padding=3)
ax.set(ylim=(0, ca.max() * 1.15), yticks=[]); ax.grid(False); ax.spines["left"].set_visible(False)
ax.set_title(f"Le Site et la Boutique font {ca[['Site', 'Boutique']].sum() / ca.sum() * 100:.0f} % du  
↪ chiffre d'affaires 2025", loc="left", fontweight="bold")
O.sauver(fig, "ch03-cah-c32.png")
print("axe tronqué à 100 000 : la barre des Réseaux paraît", round((ca.min() - 100) / (ca.max() - 100)
↪ * 100), "% de la plus grande, alors qu'elle en fait", round(ca.min() / ca.max() * 100), "%")
```

figure : ch03-cah-c32.png

axe tronqué à 100 000 : la barre des Réseaux paraît 9 % de la plus grande, alors qu'elle en fait 24 %

Corrigé 3.3

Message : « décembre est le mois de pointe de 2025 ». Le titre est calculé, et une assertion protège le message.

```
d = j[j["annee"] == 2025].groupby("mois")["chiffre_affaires"].sum() / 1000
pic = d.idxmax()
assert pic == 12, "le message « décembre est le mois de pointe » n'est plus vrai"
titre = f"Le mois {pic} est le mois de pointe de 2025 : {fr(d.max(), 0)} k€, soit {fr(d.max() /
↪ d.drop(pic).mean(), 1)} fois un mois ordinaire"
fig, ax = plt.subplots(figsize=(5.8, 3))
ax.plot(d.index, d.values, color=style.BLEU, marker="o"); ax.set(xticks=range(1, 13), ylabel="k€")
ax.set_title(titre, loc="left", fontweight="bold", fontsize=9)
O.sauver(fig, "ch03-cah-c33.png")
print(titre)
```

figure : ch03-cah-c33.png

Le mois 12 est le mois de pointe de 2025 : 184 k€, soit 1,8 fois un mois ordinaire

Sans l'assertion, un script rejoué sur d'autres données pourrait afficher « Le mois 11 est le mois de pointe » avec un commentaire écrit à la main qui parle encore de décembre : le graphique et le

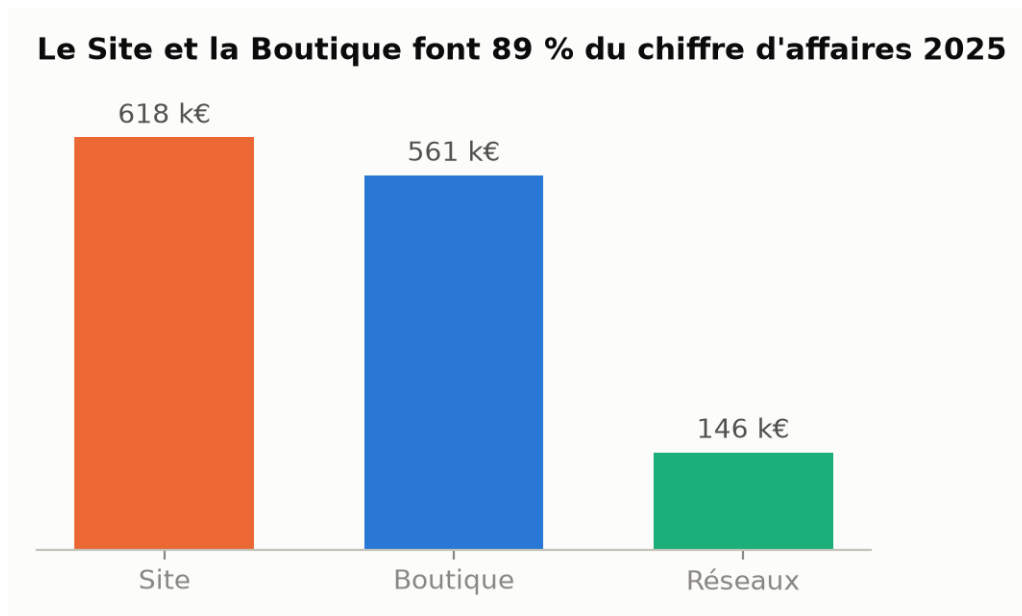


Figure 18: Le graphique corrigé.

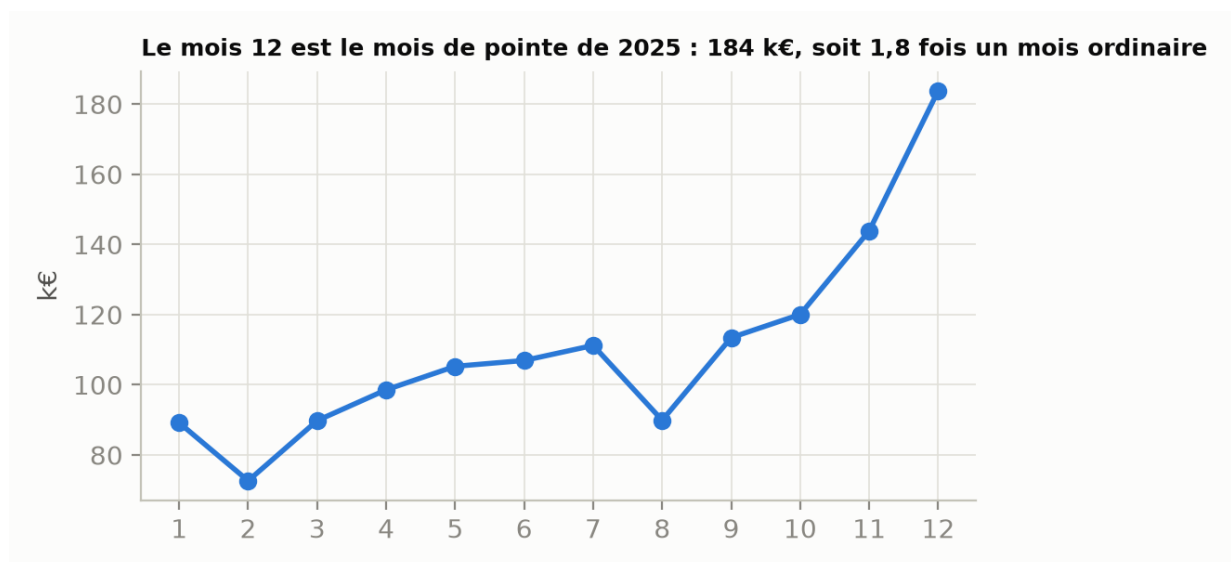


Figure 19: Le titre calculé dit le message.

texte se contrediraient sans que personne le voie. L'assertion **échoue bruyamment** : c'est ce qu'on veut.

Corrigé 3.4

```
cm = x[x["annee"] == 2025].groupby(["categorie",
↳ x["date_commande"].dt.month])["montant"].sum().unstack(0) / 1000
fig, axes = plt.subplots(2, 6, figsize=(13, 4.2), sharex=True)
fig.subplots_adjust(wspace=0.35)
for k, c in enumerate(cm.columns):
    for l_ in (0, 1):
        axes[l_, k].plot(cm.index, cm[c], color=style.BLEU)
        axes[0, k].set_title(c, loc="left", fontsize=9, fontweight="bold")
        axes[1, k].set_ylim(0, cm.values.max() * 1.05); axes[1, k].set_xticks([1, 6, 12])
axes[0, 0].set_ylabel("échelle libre"); axes[1, 0].set_ylabel("échelle commune")
O.sauver(fig, "ch03-cah-c34.png")
print("maximum mensuel (k€) :", cm.max().round(1).to_dict())
```

figure : ch03-cah-c34.png
 maximum mensuel (k€) : {'Bien-être': 18.0, 'Cuisine': 33.7, 'Décoration': 60.9, 'Jardin': 56.7,
 ↳ 'Maison': 44.7, 'Papeterie': 7.8}

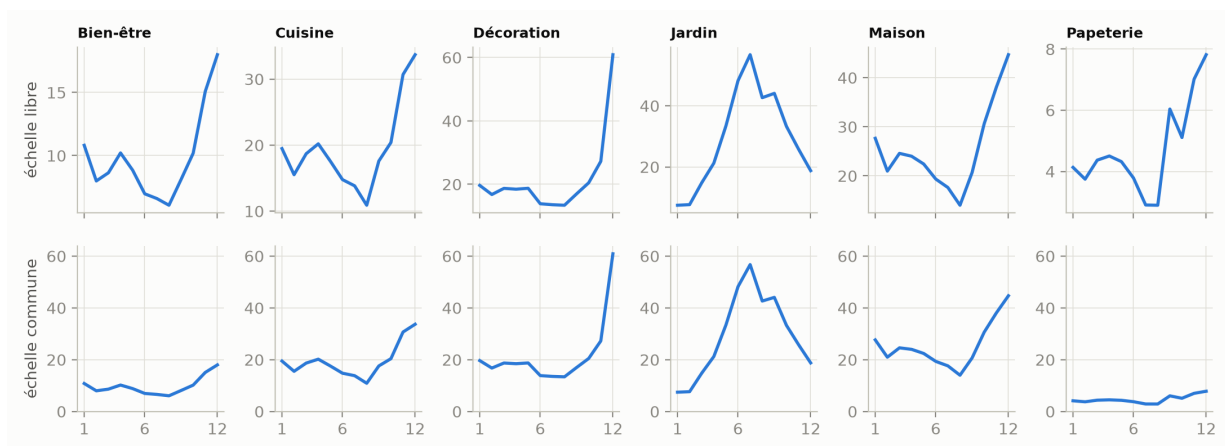


Figure 20: Six catégories : échelle libre en haut, échelle commune en bas.

L'**échelle commune** répond à « quelle catégorie pèse le plus, et de combien ? » ; l'**échelle libre** répond à « la saison est-elle la même dans toutes les catégories ? » (on compare des formes). Ici les maxima mensuels vont de 8 k€ (Papeterie) à 61 k€ (Décoration) : avec l'échelle commune, la Papeterie est écrasée mais on voit qu'elle est petite ; avec l'échelle libre, on lit sa saison mais on croirait qu'elle pèse autant que les autres. Plus les catégories diffèrent en taille, plus le choix compte.

Corrigé 3.5

```
def lum(h):
    c = [int(h[i:i + 2], 16) / 255 for i in (1, 3, 5)]
    c = [v / 12.92 if v <= 0.03928 else ((v + 0.055) / 1.055) ** 2.4 for v in c]
    return 0.2126 * c[0] + 0.7152 * c[1] + 0.0722 * c[2]

def contraste(a, b):
    la, lb = lum(a), lum(b)
    return (max(la, lb) + 0.05) / (min(la, lb) + 0.05)
```

```
for f in (1.0, 0.8, 0.7, 0.6, 0.5):
    h = "#" + "".join(f"{int(int(style.ORANGE[i:i + 2], 16) * f):02x}" for i in (1, 3, 5))
    print(f"facteur {f:.1f} : {h} -> contraste {contraste(h, style.SURFACE):.1f}")
```

```
facteur 1.0 : #eb6834 -> contraste 3.1
facteur 0.8 : #bc5329 -> contraste 4.6
facteur 0.7 : #a44824 -> contraste 5.8
facteur 0.6 : #8d3e1f -> contraste 7.2
facteur 0.5 : #75341a -> contraste 9.0
```

Dès 80 % de la teinte d'origine (#bc5329), le contraste atteint 4,6 : c'est la réponse minimale ; à 70 % (#a44824) il atteint 5,8, avec une marge de sécurité. On n'écrit pas toutes les étiquettes avec la couleur pure de la série parce que **l'orange pur est trop clair pour du texte** : une barre est un grand aplat, qu'un contraste de 3 suffit à distinguer, alors que de petites lettres exigent 4,5. Solution courante : colorer la **barre** avec la couleur pure et l'**étiquette** avec le gris foncé du texte, ou avec la version assombrie.

Corrigé 3.6

- (a) **PNG** : universel, léger pour une figure simple, lisible par tous les logiciels de courrier. (b) **PNG** (ou PDF avec `rasterized=True`) : 200 000 points en vectoriel donneraient un fichier énorme et lent ; l'image fixe garde la taille constante. (c) **SVG ou PDF** : le vectoriel reste net à n'importe quelle taille. (d) **SVG** pour un graphique simple (net sur tous les écrans, léger), **PNG** pour un graphique dense.

Corrigé 3.7

```
d = j[j["annee"] == 2025]
fig, (g1, g2) = plt.subplots(1, 2, figsize=(8, 3), sharey=True)
sns.lineplot(data=d, x="mois", y="nb_commandes", color=style.BLEU, ax=g1)
sns.lineplot(data=d, x="mois", y="nb_commandes", color=style.BLEU, errorbar="sd", ax=g2)
g1.set_title("bande : intervalle de confiance à 95 % de la moyenne", fontsize=9); g2.set_title("bande
↳ : écart-type des jours", fontsize=9)
for a in (g1, g2):
    a.set(xticks=range(1, 13), xlabel="mois", ylabel="commandes par jour")
0.sauver(fig, "ch03-cah-c37.png")
s = d[d["mois"] == 12]["nb_commandes"]
print("décembre : moyenne", round(s.mean(), 1), "| demi-largeur de l'intervalle", round(1.96 *
↳ s.std() / np.sqrt(len(s)), 1), "| écart-type", round(s.std(), 1))
```

```
figure : ch03-cah-c37.png
décembre : moyenne 59.8 | demi-largeur de l'intervalle 5.1 | écart-type 14.5
```

Légende de la version par défaut : « la ligne est le nombre moyen de commandes par jour ; la bande est **l'intervalle de confiance à 95 %** de cette moyenne (elle mesure la précision de la moyenne mensuelle, pas la variation des jours) ». Légende de la seconde : « la bande est **plus ou moins un écart-type** des commandes quotidiennes du mois ». C'est la **seconde** qui répond à « de combien varient les jours d'un même mois ? ».

Corrigé 3.8

Le test compare les hauteurs des barres à **leurs étiquettes** : il attrape l'erreur d'une barre qui porte le nom d'un autre canal.

```
def barres_canaux(ca):
    fig, ax = plt.subplots()
```

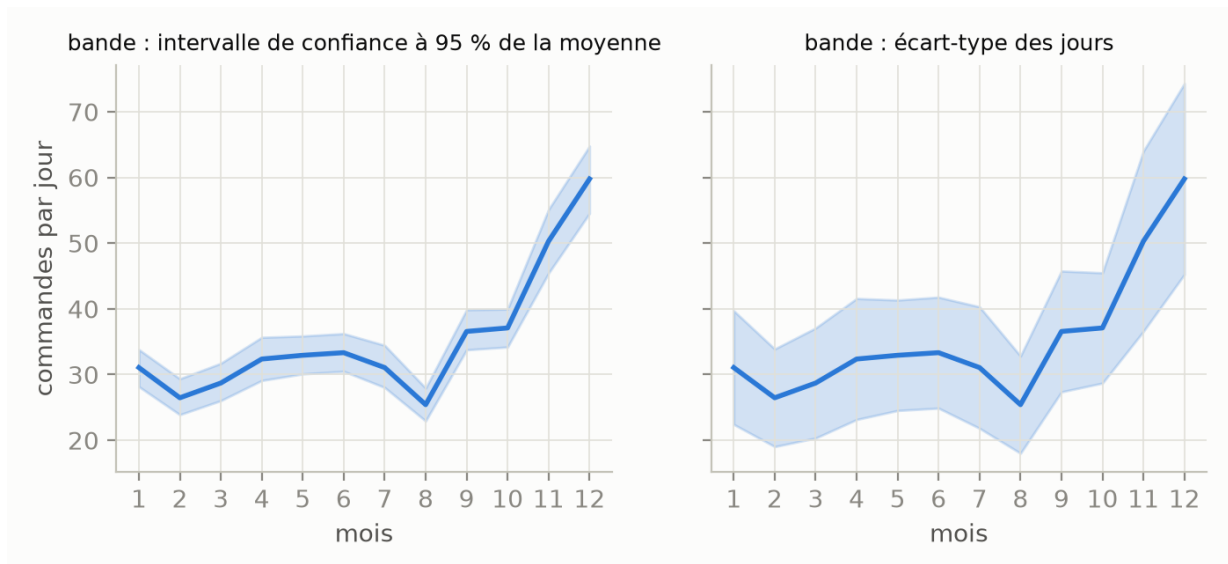


Figure 21: À gauche l'intervalle de la moyenne, à droite l'écart-type.

```

ax.bar(ca.index, ca.sort_values(ascending=False).values)           # version fautive : valeurs
↪ triées, étiquettes non triées
return fig

def barres_canaux_juste(ca):
    ca = ca.sort_values(ascending=False)
    fig, ax = plt.subplots()
    ax.bar(ca.index, ca.values)
    return fig

def lire_barres(fig):
    ax = fig.axes[0]
    return {t.get_text(): b.get_height() for t, b in zip(ax.get_xticklabels(), ax.patches)}

ca = x[x["annee"] == 2025].groupby("canal")["montant"].sum()
for nom, f in (("fautive", barres_canaux), ("corrigée", barres_canaux_juste)):
    fig = f(ca); fig.canvas.draw()
    lu = lire_barres(fig)
    print(nom, ": toutes les barres portent le bon chiffre :", all(np.isclose(lu[c], ca[c]) for c in
    ↪ lu))
    plt.close(fig)

```

fautive : toutes les barres portent le bon chiffre : False
 corrigée : toutes les barres portent le bon chiffre : True

La version fautive passe l'œil (trois barres, des étiquettes) mais échoue au test : une barre porte le chiffre d'un autre canal. C'est l'erreur la plus grave d'un graphique, précisément parce qu'elle est **invisible**.

Corrigé 3.9

```

library(ggplot2); library(dplyr, warn.conflicts = FALSE)
d <- Sys.getenv("DONNEES")
l <- read.csv(file.path(d, "lignes_commande.csv")); cm <- read.csv(file.path(d, "commandes.csv"))
pa <- inner_join(l, cm, by = "id_commande") |> filter(substr(date_commande, 1, 4) == "2025") |>
  group_by(id_commande, canal) |> summarise(panier = sum(montant), .groups = "drop")
med <- pa |> group_by(canal) |> summarise(mediane = median(panier))
p <- ggplot(pa, aes(panier)) + geom_histogram(binwidth = 20, fill = "#2a78d6") +

```

```
geom_vline(data = med, aes(xintercept = mediane), color = "#eb6834") + facet_wrap(~ canal) +
  coord_cartesian(xlim = c(0, 700)) + labs(x = "panier (€)", y = "commandes") + theme_minimal()
ggsave(file.path(dirname(d), "figures", "ch03-cah-c39.png"), p, width = 7, height = 2.6, dpi = 200)
print(as.data.frame(med |> mutate(mediane = round(mediane, 1))))
```

	canal	mediane
1	Boutique	82.0
2	Réseaux	81.2
3	Site	81.2

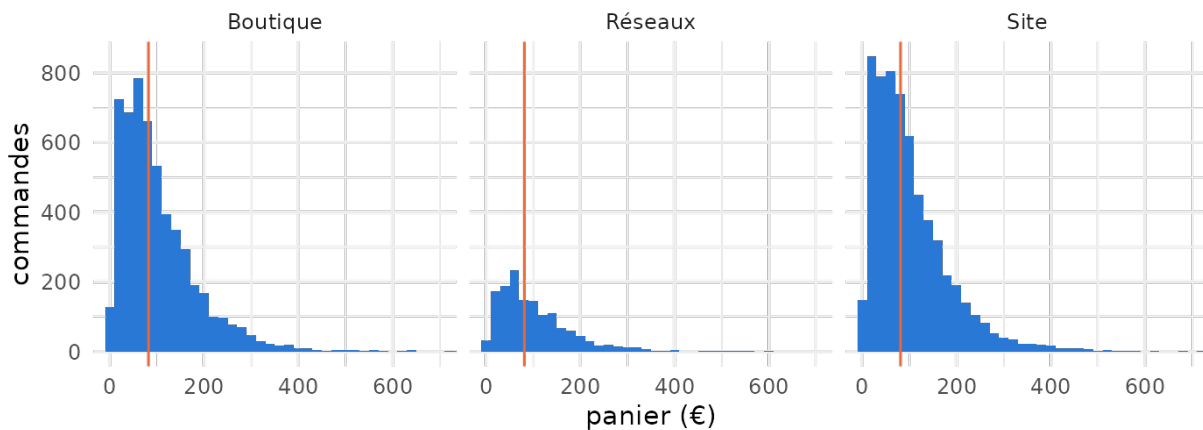


Figure 22: Trois histogrammes de paniers, une ligne à la médiane.

Dans `facet_wrap`, les échelles sont **communes par défaut** (`scales = "fixed"`), l'équivalent de `sharey=True` ; pour les libérer, on écrit `facet_wrap(~ canal, scales = "free_y")`.

Corrigé 3.10

```
import plotly.express as px
import plotly.graph_objects as go
site = x[x["canal"] == "Site"].groupby(x["date_commande"].dt.to_period("M").dt.to_timestamp())[
  "montant"].sum().reset_index()
f_go = go.Figure(go.Scatter(x=site["date_commande"], y=site["montant"], mode="lines",
  line_color=style.ORANGE))
f_px = px.line(site, x="date_commande", y="montant")
print("séries : go", len(f_go.data), "| px", len(f_px.data), "| clés de to_dict :",
  sorted(f_go.to_dict()), "| mise en page : 'layout'")
print("mêmes ordonnées :", list(f_go.data[0].y) == list(f_px.data[0].y))
```

```
séries : go 1 | px 1 | clés de to_dict : ['data', 'layout'] | mise en page : 'layout'
mêmes ordonnées : True
```

Chaque figure a une série ; la mise en page se trouve sous la clé `layout`. Avec `px`, la couleur n'est pas fixée (elle prend la couleur par défaut du gabarit) ; avec `go`, on la règle explicitement (`line_color`).

Corrigé 3.11

```
cat = x[x["annee"] == 2025].groupby("categorie", as_index=False)["montant"].sum()
cat["part"] = cat["montant"] / cat["montant"].sum() * 100
cat["k"] = cat["montant"] / 1000
fig = px.bar(cat.sort_values("k", ascending=False), x="categorie", y="k", custom_data=["part"],
  template="simple_white")
```

```
fig.update_traces(hovertemplate="<b>{x}</b><br>{y:,.0f} k€ — %{customdata[0]:.1f} % du  
↳ total<extra></extra>", marker_color=style.BLEU)  
fig.update_layout(separators=", ", axis_title=None, yaxis_title="k€", title="Chiffre d'affaires 2025  
↳ par catégorie")  
print("somme des parts :", round(cat["part"].sum(), 6), "| infobulle de la première barre :",  
↳ fig.data[0].hovertemplate[:40])  
dossier = 0.dossier_temp(); chemin = os.path.join(dossier, "cat.html"); fig.write_html(chemin,  
↳ include_plotlyjs=True)  
0.capturer_html(chemin, os.path.join(0.FIG, "ch03-cah-c311.png"), 800, 420, survol=".bars .point >>  
↳ nth=0")  
0.supprimer_dossier(dossier)
```

somme des parts : 100.0 | infobulle de la première barre : {x}
{y:,.0f} k€ — %{customda
capture existante : ch03-cah-c311.png

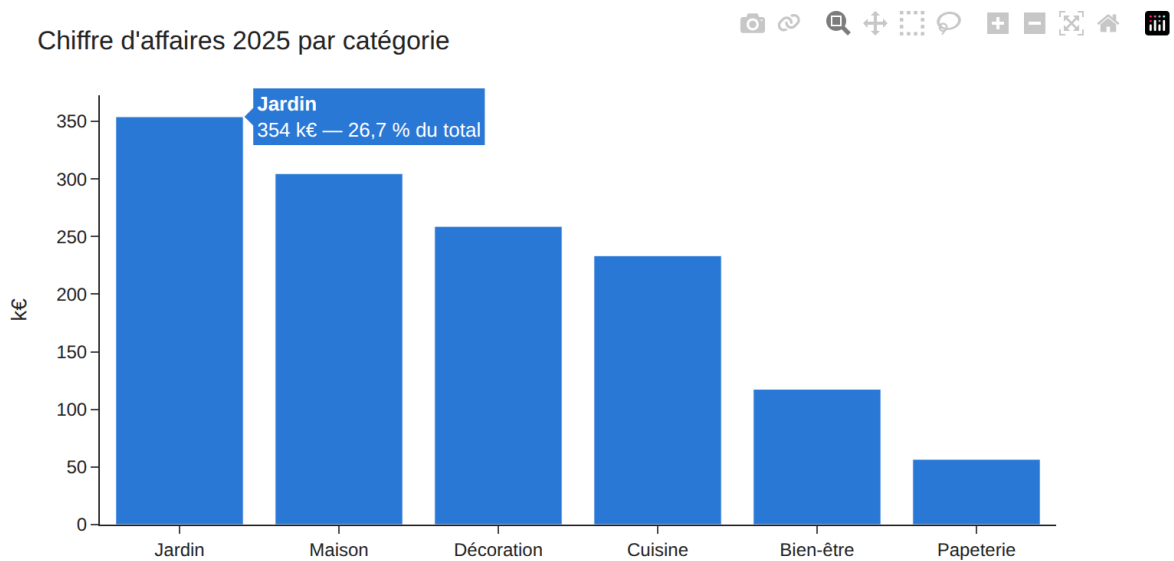


Figure 23: Capture réelle : l'infobulle donne la valeur et la part.

Avec `barmode="group"` (barres côte à côte), on **gagne** la comparaison d'un canal d'une catégorie à l'autre (toutes les barres partent de zéro) ; on **perd** la lecture du total de la catégorie. Les barres **empilées** font l'inverse. Le bon choix dépend de la question : « combien au total ? » ou « quel canal domine ? ».

Corrigé 3.12

- (a) **Non** : un PDF n'a pas de survol ; la figure doit se suffire (étiquettes directes, titre qui dit le message). (b) **Oui** : un lecteur régulier explore par canal et par mois ; mais le message essentiel reste visible sans geste. (c) **Non** : on ne clique pas en réunion ; une image nette, avec le fait marquant annoté. (d) **Oui** : l'analyste cherche, survole et zoome ; c'est l'usage où l'interactivité rapporte le plus. Dans tous les cas où l'on **livre**, on garde une image de ce qui a été montré.

Corrigé 3.13

Le script se **rejoue en entier** à chaque changement : sans cache, le fichier est lu **cinq fois** (une fois au chargement et une fois après chacun des quatre changements). Avec `@st.cache_data`, il n'est lu qu'**une fois** (le résultat est gardé et réutilisé). Le modèle est plus **simple** (un script linéaire, pas de graphe de dépendances à déclarer) mais plus **coûteux** : à chaque clic, tout ce qui n'est pas en cache est recalculé, alors que Shiny ne recalcule que les expressions qui dépendent de l'entrée modifiée.

Corrigé 3.14

Avec une liste vide, la fonction naïve filtre un tableau vide, et `.sum()` d'un tableau vide donne 0 : l'application affiche « 0 € » sans signaler que l'utilisateur n'a rien sélectionné. Selon le graphique, un tableau vide peut aussi lever une erreur. Le test appelle la fonction avec la liste vide.

```
def maj_robuste(canaux):
    if not canaux:
        return "Choisissez au moins un canal."
    s = x[x["canal"].isin(canaux) & (x["annee"] == 2025)]
    return f'{', '.join(canaux)} : {s["montant"].sum():,.0f} €'.replace(",", " ")

def maj_naive(canaux):
    s = x[x["canal"].isin(canaux) & (x["annee"] == 2025)]
    return f"{s['montant'].sum():,.0f} €"

print("naïve, liste vide :", maj_naive([]), "| robuste, liste vide :", maj_robuste([]))
assert maj_robuste([]) != "0 €"
print("robuste, un canal :", maj_robuste(["Site"]))
```

```
naïve, liste vide : 0 € | robuste, liste vide : Choisissez au moins un canal.
robuste, un canal : Site : 617 715 €
```

La version naïve ne plante pas mais **ment par omission** : un « 0 € » affiché en gros laisse croire que les ventes sont nulles. La version robuste dit ce qui se passe. Un tableau de bord doit prévoir l'état « rien de sélectionné » comme les autres.

Corrigé 3.15

```
v = 0.ventes_villes(x)
r1, r2 = v["ca"].rank(ascending=False), v["par_habitant"].rank(ascending=False)
commun = sorted(set(r1[r1 <= 5].index) & set(r2[r2 <= 5].index))
print("cinq premières en chiffre d'affaires :", list(r1.nsmallest(5).index))
print("cinq premières par habitant          :", list(r2.nsmallest(5).index))
print("villes en commun :", len(commun), commun)
```

```
cinq premières en chiffre d'affaires : ['Ville A', 'Ville B', 'Ville C', 'Ville D', 'Ville E']
cinq premières par habitant          : ['Ville C', 'Ville D', 'Ville A', 'Ville E', 'Ville I']
villes en commun : 4 ['Ville A', 'Ville C', 'Ville D', 'Ville E']
```

Quatre villes sur cinq sont communes aux deux classements (A, C, D et E) : la différence tient à **Ville B**, deuxième en chiffre d'affaires mais absente du classement par habitant (170 000 habitants, environ 1 € chacun), remplacée par **Ville I**. Le lecteur du classement en **chiffre d'affaires** voit en Ville B un marché majeur ; celui du classement **par habitant** y voit un marché à peine exploité. Les deux ont raison pour leur question ; l'erreur est de présenter l'un comme s'il répondait à l'autre.

Corrigé 3.16

```

par_ville = v.groupby("region").agg(ca=("ca", "sum"), habitants=("habitants", "sum"),
    ↳ moy_taux=("par_habitant", "mean"))
par_ville["rapport_totaux"] = par_ville["ca"] / par_ville["habitants"]
print(par_ville[["moy_taux", "rapport_totaux"]].round(2).to_string())
fig, ax = plt.subplots(figsize=(5, 2.8))
tri = par_ville["rapport_totaux"].sort_values()
ax.barh(tri.index, tri.values, color=style.BLEU, height=0.55); ax.grid(axis="y", visible=False);
    ↳ ax.set_axisbelow(True); ax.set(xlabel="€ par habitant, 2025")
ax.set_title("Chiffre d'affaires par habitant et par région", loc="left", fontweight="bold",
    ↳ fontsize=10)
O.sauver(fig, "ch03-cah-c316.png")
print("écart maximal entre les deux méthodes :", round((par_ville["moy_taux"] -
    ↳ par_ville["rapport_totaux"]).abs().max(), 2), "€")

```

region	moy_taux	rapport_totaux
Région 1	2.01	1.42
Région 2	1.07	0.69
Région 3	3.92	2.01
Région 4	0.53	0.30

figure : ch03-cah-c316.png
 écart maximal entre les deux méthodes : 1.92 €

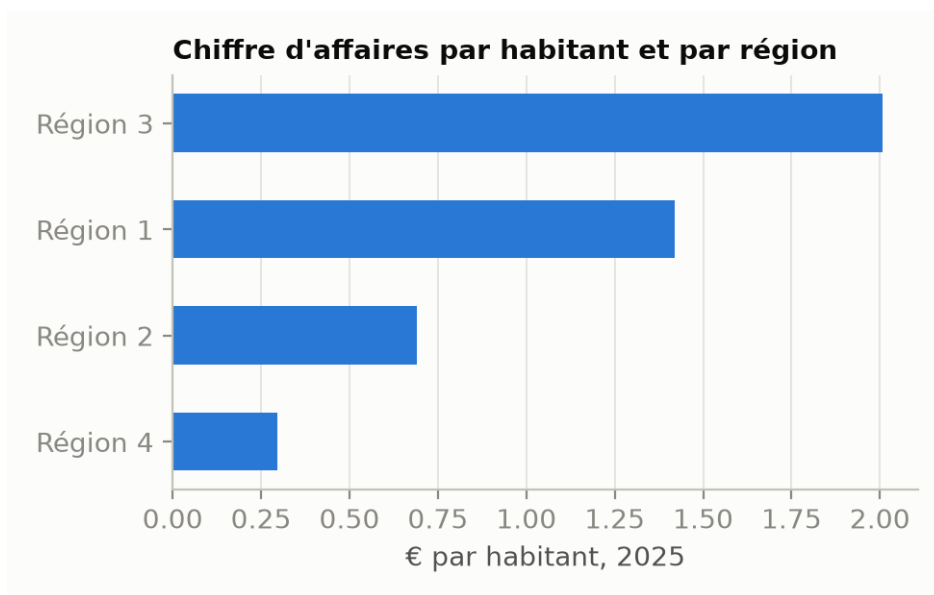


Figure 24: Chiffre d'affaires par habitant et par région, calculé par le rapport des totaux.

La **bonne** méthode est le **rapport des totaux** : le chiffre d'affaires de la région divisé par sa population. La moyenne des rapports donne le même poids à une petite ville et à une grande, alors qu'elles n'ont pas le même nombre d'habitants : c'est une moyenne **non pondérée** de taux. Ici le classement des régions est le même, mais les valeurs sont presque **doublées** (3,92 € contre 2,01 € pour la Région 3) : on aurait annoncé un chiffre faux. Pour quatre régions, des **barres** suffisent et valent mieux : le classement est immédiat et les valeurs sont écrites ; une carte de quatre zones n'apporterait que la position, dont la question ne parle pas.

Chapitre 4 : Storytelling et rédaction de rapports — exercices et applications

Ce cahier prolonge le chapitre 4 : on y **construit** des storyboards et des titres qui concluent, on **relit** des textes avec un outil qui contrôle les chiffres, on **génère** des résumés par le code, on **mesure** la sensibilité d'une recommandation, et l'on **teste** les règles d'un rapport automatique. Le cahier est autonome : il recharge ses données. Les données sont **simulées** ; la plupart des réponses sont des textes, que le code aide à vérifier.

```
import os, sys, warnings
import numpy as np, pandas as pd
warnings.filterwarnings("ignore")
sys.path.insert(0, "build")
import outils_ch04 as O
D = os.environ["DONNEES"]
d = O.charger(D)
a = O.analyse_promo(d)
j, x, liv = d["j"], d["x"], d["liv"]

def sig(v, n=2):
    return round(v, n - 1 - int(np.floor(np.log10(abs(v)))))

print("jours :", len(j), "| lignes de commande :", len(x), "| livraisons :", len(liv))
print("effet des promotions :", 0.fr(a["e"] * 100, 1, True), "% | incrément de marge :",
      ↪ 0.fr(a["inc"], 0), "€")
```

```
jours : 1096 | lignes de commande : 83905 | livraisons : 19420
effet des promotions : +19,2 % | incrément de marge : -17 884 €
```

Applications

Application 4.1 — Le storyboard des livraisons (section 4.1)

Objectif. Passer de chiffres à un storyboard de cinq pages dont chaque titre est une conclusion, sur un sujet autre que les promotions : les **livraisons en retard**.

Étape 1 — les faits. On calcule d'abord ce qui sera dit : le taux de retard de 2025, la saison, les transporteurs.

```
l = liv[liv["date_commande"].dt.year == 2025].assign(dec=lambda t: t["date_commande"].dt.month == 12)
print("retards 2025 :", 0.fr(l["retard"].mean() * 100, 1), "% | hors décembre :",
      ↪ 0.fr(l[~l["dec"]]["retard"].mean() * 100, 1), "% | décembre :", 0.fr(l[l["dec"]]["retard"].mean()
      ↪ * 100, 1), "%")
```

```
t = l.groupby(["transporteur", "dec"])["retard"].mean().unstack().mul(100).round(0)
t.columns = ["hors décembre (%)", "décembre (%)"]
print(t.to_string())
```

```
retards 2025 : 26,5 % | hors décembre : 21,7 % | décembre : 54,0 %
           hors décembre (%)  décembre (%)
transporteur
Transporteur A              11.0         38.0
Transporteur B              21.0         56.0
Transporteur C              46.0         85.0
```

Étape 2 — le storyboard. Un tableau : une ligne par page, un titre qui conclut, la preuve qui l'appuie. Les nombres des titres viennent des calculs ci-dessus.

```
g, h = l[l["dec"]]["retard"].mean() * 100, l[~l["dec"]]["retard"].mean() * 100
tc = l[l["transporteur"] == "Transporteur C"]["retard"].mean() * 100
story = pd.DataFrame({"titre": [f"Plus d'un colis sur quatre arrive en retard
↳ ({0.fr(l['retard'].mean() * 100, 0)} %)", f"En décembre, le retard double ({0.fr(g, 0)} % contre
↳ {0.fr(h, 0)} %)",
                                "Tous les transporteurs reculent en décembre", f"Le transporteur C est
↳ en retard une fois sur deux ({0.fr(tc, 0)} %)", "Prévoir décembre
↳ et réduire la part du transporteur C"],
                    "preuve": ["taux global", "taux par mois", "taux par transporteur et saison",
                               "taux par transporteur", "plan d'action"]}, index=range(1, 6))
print(story.to_string())
```

	titre	preuve
1	Plus d'un colis sur quatre arrive en retard (27 %)	taux global
2	En décembre, le retard double (54 % contre 22 %)	taux par mois
3	Tous les transporteurs reculent en décembre	taux par transporteur et saison
4	Le transporteur C est en retard une fois sur deux (52 %)	taux par transporteur
5	Prévoir décembre et réduire la part du transporteur C	plan d'action

À vous. Quel élément de l'analyse placeriez-vous en annexe ? Quelle objection du lecteur la page 3 prévient-elle ?

Application 4.2 — Des titres qui concluent (section 4.1)

Objectif. Calculer la valeur qu'un titre affirme, puis vérifier qu'elle est sourcée.

Étape 1 — le fait. La part de novembre et décembre dans le chiffre d'affaires de chaque année.

```
m = j.groupby(["annee", "mois"])["chiffre_affaires"].sum().unstack(0)
part = m.loc[[11, 12]].sum() / m.sum() * 100
print(part.round(1).to_string())
```

```
annee
2023    24.5
2024    24.3
2025    24.7
```

Étape 2 — le titre et son contrôle. Le contrôle des nombres signale tout chiffre que le calcul ne justifie pas. L'année, elle aussi, est un nombre : on la déclare.

```
for an in part.index:
    titre = f"Novembre et décembre font {0.fr(part[an], 0)} % du chiffre d'affaires {an}"
    print(titre, "| sans source :", 0.verifier_nombres(titre, [part[an], an]))
titre_faux = "Novembre et décembre font 35 % du chiffre d'affaires 2025"
print(titre_faux, "| sans source :", 0.verifier_nombres(titre_faux, [part[2025], 2025]))
```

```

Novembre et décembre font 25 % du chiffre d'affaires 2023 | sans source : []
Novembre et décembre font 24 % du chiffre d'affaires 2024 | sans source : []
Novembre et décembre font 25 % du chiffre d'affaires 2025 | sans source : []
Novembre et décembre font 35 % du chiffre d'affaires 2025 | sans source : [35.0]

```

À vous. Écrivez un titre qui conclut sur l'écart entre 2023 et 2025, puis contrôlez-le.

Application 4.3 — Relire un brouillon (section 4.2)

Objectif. Utiliser le contrôle des nombres sur un texte qui contient plusieurs erreurs, puis corriger.

Étape 1 — les valeurs calculées. Les trois morceaux de l'incrément de marge des promotions.

```

B = a["n_cmd"] / (1 + a["e"])
sans, gain = B * a["mo_np"], (a["n_cmd"] - B) * a["mo_np"]
remise = a["marge_reelle"] - sans - gain
print("sans promotion :", 0.fr(sans, 0), "€ | commandes en plus :", 0.fr(gain, 0, True), "€ | remises
↳ :", 0.fr(remise, 0, True), "€ | solde :", 0.fr(gain + remise, 0, True), "€")

```

```
sans promotion : 145 861 € | commandes en plus : +27 969 € | remises : -45 854 € | solde : -17 884 €
```

Étape 2 — le brouillon. Il contient deux nombres que le calcul ne justifie pas.

```

brouillon = ("Sur 153 jours de promotion, les commandes en plus rapportent 28 000 € de marge, mais les
↳ remises en retirent 47 000 €, soit un solde de -19 000 €. "
             "Chaque commande rapporte 24 € au lieu de 32 €.")
permis = [a["jours"], gain, remise, gain + remise, a["mo_p"], a["mo_np"]]
print("nombres sans source :", 0.verifier_nombres(brouillon, permis))

```

```
nombres sans source : [47000.0, 19000.0]
```

À vous. Corrigez les deux nombres, puis ajoutez une phrase qui cite l'intervalle de l'effet (14 à 25 %) et contrôlez-la.

Application 4.4 — Un résumé par édition (section 4.2)

Objectif. Produire par le code une phrase par édition de la promotion, sans recopier un seul nombre.

Étape 1 — la marge par commande de chaque édition.

```

pr = j[j["promo_active"] == 1].assign(edition=lambda t: t["mois"].map({1: "d'hiver", 6: "d'été", 7:
↳ "d'été", 11: "de fin novembre"}))
g = pr.groupby("edition").agg(jours=("date", "count"), marge=("marge", "sum"), cmd=("nb_commandes",
↳ "sum"))
g["marge_par_cmd"] = g["marge"] / g["cmd"]
print(g.round(1).to_string())

```

	jours	marge	cmd	marge_par_cmd
edition				
d'hiver	63	41116.8	1901	21.6
d'été	63	53219.0	2033	26.2
de fin novembre	27	33641.0	1484	22.7

Étape 2 — les phrases, puis le contrôle.

```

phrases = [f"L'édition {ed} compte {int(r['jours'])} jours et rapporte {0.fr(r['marge_par_cmd'], 0)} €
↳ de marge par commande, contre {0.fr(a['mo_np'], 0)} € hors promotion." for ed, r in g.iterrows()]
print("\n".join(phrases))

```

```
print("nombres sans source :", 0.verifier_nombres(" ".join(phrases), list(g["jours"]) +
↳ list(g["marge_par_cmd"]) + [a["mo_np"]]))
```

L'édition d'hiver compte 63 jours et rapporte 22 € de marge par commande, contre 32 € hors promotion.
 L'édition d'été compte 63 jours et rapporte 26 € de marge par commande, contre 32 € hors promotion.
 L'édition de fin novembre compte 27 jours et rapporte 23 € de marge par commande, contre 32 € hors
 ↳ promotion.
 nombres sans source : []

À vous. Quelle édition est la moins mauvaise ? Quelle précaution prendre avant d'en conclure quelque chose ?

Application 4.5 — Sensibilité d'une recommandation (section 4.3)

Objectif. Répondre à « et si... ? » avec un tableau : l'incrément de marge selon l'effet sur les commandes et la part de la remise actuelle que l'on conserve. C'est un **scénario**, pas une prévision : l'effet est supposé indépendant de la remise.

```
N, ecart = a["n_cmd"], a["mo_np"] - a["mo_p"]          # commandes observées, écart de marge par
↳ commande
def increment(effet, part):
    return N * (a["mo_np"] - ecart * part) - N / (1 + effet) * a["mo_np"]    # marge obtenue moins
↳ marge sans promotion
effets = {"borne basse": a["e_bas"], "estimé": a["e"], "borne haute": a["e_haut"]}
t = pd.DataFrame({nom: [increment(e, p) / 1000 for p in (1, 0.75, 0.5, 0.25)] for nom, e in
↳ effets.items()}, index=["100 %", "75 %", "50 %", "25 %"])
t.index.name = "remise conservée"
print(t.round(0).to_string())
```

	borne basse	estimé	borne haute
remise conservée			
100 %	-25.0	-18.0	-11.0
75 %	-14.0	-6.0	0.0
50 %	-2.0	5.0	12.0
25 %	9.0	17.0	23.0

À vous. Quelle remise garder pour que l'incrément soit positif quel que soit l'effet de l'intervalle ? Quelle hypothèse cachée rend ce résultat fragile ?

Application 4.6 — Un rapport qui se tait, pour le panier moyen (section 4.4)

Objectif. Appliquer la règle du chapitre (ne commenter que ce qui dépasse une fois et demie la variation ordinaire) à un autre indicateur : le **panier moyen hebdomadaire**.

```
js = j.set_index("date")
panier = (js["chiffre_affaires"].resample("W-SUN").sum() /
↳ js["nb_commandes"].resample("W-SUN").sum()).iloc[1:-1]
v = (panier / panier.shift(52) - 1).dropna()          # variation sur un an
bruit = v.shift(1).rolling(26).std()                 # variation ordinaire des 26
↳ semaines précédentes
signal = (v.abs() >= 1.5 * bruit)["2025-01-12:"]
print("semaines 2025 :", len(signal), "| commentées par le générateur prudent :", int(signal.sum()))
print("écart-type de la variation annuelle du panier :", 0.fr(v.std() * 100, 1), "% | médiane :",
↳ 0.fr(v.median() * 100, 1, True), "%")
```

```
semaines 2025 : 51 | commentées par le générateur prudent : 10
écart-type de la variation annuelle du panier : 7,5 % | médiane : +1,3 %
```

À vous. Les semaines signalées sont-elles de hausse, de baisse, ou des deux ? Que devrait ajouter un humain au rapport ?

Exercices

Exercice 4.1 ★ — Journal ou récit ? (section 4.1.1)

Voici quatre débuts de rapport sur les livraisons. Classez-les en « journal de bord » ou « récit », et réécrivez le plus mauvais en une phrase qui donne la réponse. (a) « Ce rapport présente les résultats de l'analyse des livraisons de 2023 à 2025. » (b) « Les retards de livraison doublent en décembre : plus d'un colis sur deux arrive après la date promise. » (c) « Nous avons commencé par nettoyer le fichier des livraisons, puis nous avons calculé les délais. » (d) « Faut-il changer de transporteur ? Pas pour décembre, qui touche tous les transporteurs ; mais le transporteur C est en retard toute l'année. »

Exercice 4.2 ★ — Écrire un titre qui conclut (section 4.1.4)

Pour chaque titre descriptif, calculez le fait puis écrivez un titre-conclusion : (a) « Marge brute par catégorie en 2025 » ; (b) « Taux de retard par mois en 2025 » ; (c) « Taux de retard par transporteur en 2025 ». Contrôlez vos nombres avec `0.verifier_nombres`.

Exercice 4.3 ★★ — Le test du verbe (section 4.1.7)

Pour chaque phrase, dites ce que l'on a **établi** et corrigez le verbe : (a) « La nouvelle page d'accueil a augmenté la conversion de 12 %, comparée au mois précédent. » (b) « Un test aléatoire de deux semaines montre que le message de livraison offerte augmente les commandes de 4 % (intervalle de 1 à 7 %). » (c) « Les clients qui reçoivent la lettre d'information achètent deux fois plus que les autres. » (d) « À saison égale, les jours de promotion comptent 19 % de commandes de plus. »

Exercice 4.4 ★★ — La baisse de janvier (section 4.1.7)

« Les commandes s'effondrent en janvier ! » Calculez, pour chacun des hivers 2023-2024 et 2024-2025, la variation entre les quatre semaines du 25 novembre au 22 décembre et les quatre semaines du 8 janvier au 2 février de l'année suivante. Que répondez-vous à l'auteur de la phrase ?

Exercice 4.5 ★ — Écrire les chiffres (section 4.2.3)

Écrivez pour la gérante, avec l'arrondi et l'unité qui conviennent, les valeurs suivantes : effet = 0,19175, borne basse = 0,13539, borne haute = 0,25091, marge perdue = -17 884,35 €, seuil = 0,35830, livraisons à l'heure = 77,7 % (semaine) et 80,9 % (même semaine de l'an dernier). Pour les livraisons, donnez la baisse en **points** et en **valeur relative**.

Exercice 4.6 ★★ — Réécrire un paragraphe (section 4.2.2)

Réécrivez ce paragraphe en trois phrases au plus, avec la réponse en premier et sans jargon, puis comparez les mesures de lisibilité (`0.lisibilite`) : « Dans le cadre de l'analyse des livraisons de l'année 2025, il a été procédé au calcul du taux de retard, défini comme la proportion de livraisons dont la date effective excède la date promise, lequel s'établit à 26,5 %, avec une disparité importante selon la période de l'année considérée, le taux atteignant 54,0 % en décembre contre 21,7 % le reste de l'année. »

Exercice 4.7 ★★ — Savoir, ne pas savoir, trancher (section 4.2.5)

Pour la conclusion « le transporteur C est en retard toute l'année », remplissez le tableau en trois colonnes de la section 4.2.5 : ce que nous savons, ce que nous ne savons pas, ce qu'il faudrait pour trancher. Citez au moins une limite qui **pourrait changer** la conclusion.

Exercice 4.8 ★★★ — Un résumé généré pour les livraisons (section 4.2.7)

Écrivez une fonction `resume_livraisons(liv, annee)` qui renvoie un résumé de deux phrases (taux de retard de l'année, taux de décembre contre le reste) **entièrement produit par le code**, puis vérifiez avec `0.verifier_nombres` qu'aucun nombre n'est orphelin. Appelez-la pour 2023, 2024 et 2025.

Exercice 4.9 ★★ — Le résumé en cinq lignes du transporteur C (section 4.3.1)

Écrivez, à partir des chiffres calculés, le résumé en cinq lignes (contexte, constat, pourquoi, recommandation, décision demandée) qui propose de réduire la part du transporteur C. Donnez un objet de courriel. Mesurez la lisibilité du résumé.

Exercice 4.10 ★★ — Le point d'équilibre de la remise (section 4.3.4)

Si l'effet sur les commandes était indépendant du niveau de la remise, quelle part de la remise actuelle faudrait-il conserver au maximum pour que l'incrément de marge reste nul ? Donnez la formule, puis calculez-la pour l'effet estimé et pour les deux bornes de l'intervalle. Que penser de la robustesse de l'hypothèse ?

Exercice 4.11 ★★ — Une référence qui tient compte de la croissance (section 4.4.3)

Dans le rapport hebdomadaire, la comparaison à « la même semaine de l'an dernier » signale presque uniquement des hausses, parce que la boutique croît. Corrigez ce biais : retirez de chaque variation annuelle la **médiane** des variations annuelles des 26 semaines précédentes, puis comptez les semaines de 2025 signalées. Les semaines signalées sont-elles les mêmes qu'avant ?

Exercice 4.12 ★★★ — Un contrôle d'ordre de grandeur (section 4.4.4)

Ajoutez un contrôle avant envoi qui détecte une erreur d'**unité** : le chiffre d'affaires de la semaine doit rester entre 0,3 et 3 fois la médiane des 26 semaines précédentes. Écrivez la fonction `controle_grandeur(d, lundi)`, testez-la sur les données intactes puis sur des données où le chiffre d'affaires d'un jour a été multiplié par 100 (une erreur de saisie).

Corrigés

Corrigé 4.1

- (a) **Journal de bord** (annonce du contenu sans rien dire) ; (b) **récit** : la réponse est dans la phrase ; (c) **journal de bord** (l'ordre du travail) ; (d) **récit sous forme de pyramide** : question, réponse, nuance. Le plus mauvais est (a) : elle occupe la première ligne sans rien apprendre. Réécriture : « **Plus d'un colis sur quatre arrive en retard, et en décembre c'est un sur deux ; le transporteur C est le plus en cause.** »

Corrigé 4.2

```
cat = x[x["date_commande"].dt.year ==
↳ 2025].groupby("categorie")["marge"].sum().sort_values(ascending=False)
part_cat = cat / cat.sum() * 100
mois = liv[liv["date_commande"].dt.year ==
↳ 2025].groupby(liv["date_commande"].dt.month)["retard"].mean() * 100
trans = liv[liv["date_commande"].dt.year == 2025].groupby("transporteur")["retard"].mean() * 100
t_a = f"Deux catégories, Jardin et Maison, font {0.fr(part_cat.iloc[:2].sum(), 0)} % de la marge de
↳ 2025"
t_b = f"Le retard passe de {0.fr(mois.drop(12).mean(), 0)} % en moyenne à {0.fr(mois[12], 0)} % en
↳ décembre"
t_c = f"Le transporteur C est en retard {0.fr(trans['Transporteur C'], 0)} % du temps, contre
↳ {0.fr(trans['Transporteur A'], 0)} % pour le A"
for t_ in (t_a, t_b, t_c):
    print(t_)
print("nombres sans source :", 0.verifier_nombres(" ".join([t_a, t_b, t_c]), [part_cat.iloc[:2].sum(),
↳ mois.drop(12).mean(), mois[12], trans["Transporteur C"], trans["Transporteur A"], 2025]))
```

Deux catégories, Jardin et Maison, font 50 % de la marge de 2025
 Le retard passe de 22 % en moyenne à 54 % en décembre
 Le transporteur C est en retard 52 % du temps, contre 15 % pour le A
 nombres sans source : []

Chaque titre énonce une conclusion qu'on peut vérifier sur la figure correspondante. Un piège : la moyenne des onze mois est une **moyenne de moyennes mensuelles**, légèrement différente du taux « hors décembre » de l'application 4.1 (pondéré par le nombre de livraisons). Pour un titre à l'unité près, les deux se valent ; pour un tableau d'annexe, on donnerait la définition.

Corrigé 4.3

- (a) On a établi une **différence avant/après** (aucun contrôle de la saison, ni des autres changements) : « la conversion a augmenté de 12 % depuis le lancement de la nouvelle page d'accueil » ; ne pas écrire « grâce à » ; (b) Une **expérience aléatoire** : on peut écrire « a augmenté » avec l'intervalle, en précisant la durée ; (c) Une **association** (les clients qui s'inscrivent sont déjà plus actifs) : « les clients inscrits achètent deux fois plus, mais ils étaient peut-être déjà de meilleurs clients » ; (d) Une **comparaison à situation égale** (régression avec contrôles), qui autorise « on observe, à saison égale, ... » et n'autorise pas « la promotion cause » sans réserve.

Corrigé 4.4

```
s = j.set_index("date")["nb_commandes"].resample("W-SUN").sum().iloc[1:-1]
for an in (2023, 2024):
    dec, jan = s[f"{an}-11-25":f"{an}-12-22"].mean(), s[f"{an + 1}-01-08":f"{an + 1}-02-02"].mean()
    print(f"hiver {an}-{an + 1} : fin novembre-décembre {dec:.0f} → janvier {jan:.0f} commandes par
↳ semaine ({0.fr((jan / dec - 1) * 100, 0, True)} %)")
```

hiver 2023-2024 : fin novembre-décembre 351 → janvier 217 commandes par semaine (-38 %)
 hiver 2024-2025 : fin novembre-décembre 392 → janvier 215 commandes par semaine (-45 %)

La « chute » de janvier a lieu **chaque année**, c'est la fin de la saison des fêtes. La phrase est exacte mais elle ne dit rien d'inhabituel ; la bonne comparaison est **janvier contre janvier** de l'année précédente (le cas des cerises, section 4.1.7).

Corrigé 4.5

```
print("effet :", 0.fr(sig(0.19175 * 100), 0), "% (entre", 0.fr(sig(0.13539 * 100), 0), "et",
      ↪ 0.fr(sig(0.25091 * 100), 0), "%)")
print("marge perdue :", 0.fr(sig(-17884.35), 0), "€ | seuil :", 0.fr(sig(0.35830 * 100), 0), "% de
      ↪ commandes en plus")
pts, rel = 77.7 - 80.9, (77.7 / 80.9 - 1) * 100
print("livraisons à l'heure :", 0.fr(pts, 1, True), "points, soit", 0.fr(rel, 0, True), "% en valeur
      ↪ relative")
```

```
effet : 19 % (entre 14 et 25 %)
marge perdue : -18 000 € | seuil : 36 % de commandes en plus
livraisons à l'heure : -3,2 points, soit -4 % en valeur relative
```

On écrit : « environ 19 % de commandes en plus (entre 14 et 25 %), une perte de marge d'environ 18 000 €, un seuil de 36 % ; les livraisons à l'heure baissent de 3,2 points (soit 4 % en valeur relative) ». Les décimales supplémentaires ne seraient que du faux savoir.

Corrigé 4.6

Une réécriture possible : « **Un colis sur quatre arrive en retard en 2025 (26,5 %), et un sur deux en décembre (54 %), contre un sur cinq le reste de l'année (22 %).** » Mesurons les deux versions.

```
avant = ("Dans le cadre de l'analyse des livraisons de l'année 2025, il a été procédé au calcul du
      ↪ taux de retard, défini comme la proportion de livraisons dont la date effective excède la date
      ↪ promise, lequel s'établit à 26,5 %, avec une disparité importante selon la période de l'année
      ↪ considérée, le taux atteignant 54,0 % en décembre contre 21,7 % le reste de l'année.")
apres = "Un colis sur quatre arrive en retard en 2025 (26,5 %), et un sur deux en décembre (54 %),
      ↪ contre un sur cinq le reste de l'année (22 %)."
```

```
print(pd.DataFrame({"avant": 0.lisibilite(avant), "après": 0.lisibilite(apres)}).to_string())
```

	avant	après
phrases	1.0	1.0
mots	63.0	27.0
mots_par_phrase	63.0	27.0
termes_techniques	0.0	0.0
chiffres	4.0	4.0

La phrase d'origine compte 63 mots et une seule idée noyée ; la réécriture en compte 27, avec la réponse au début et les trois chiffres comparés entre eux. Les détails de définition vont dans la section « Données » du rapport.

Corrigé 4.7

Ce que nous savons	Ce que nous ne savons pas	Ce qu'il faudrait pour trancher
Le transporteur C est en retard 52 % du temps en 2025, contre 15 % pour le A ; hors décembre, 46 % contre 11 %.	Si les retards viennent du transporteur ou des destinations qu'on lui confie (zones plus lointaines, colis plus lourds).	Comparer à destination et poids égaux, ou envoyer des colis comparables aux deux transporteurs.
Les retards de décembre touchent les trois transporteurs, et la part de chacun est la même qu'en dehors de décembre.	Si le transporteur C est moins cher, et si le coût d'un retard (réclamations, remboursements) dépasse l'économie.	Chiffrer le coût par colis et le coût d'un retard.
Le mix des transporteurs n'explique pas la hausse de décembre.	Si les clients qui reçoivent un colis en retard rachètent moins.	Suivre le réachat selon le retard.

La première limite **pourrait changer la conclusion** : si le transporteur C livre les zones les plus difficiles, le remplacer ne résoudrait rien. C'est pourquoi la recommandation est un test et non un changement immédiat.

Corrigé 4.8

```
def resume_livraisons(liv, annee):
    l = liv[liv["date_commande"].dt.year == annee]
    dec = l[l["date_commande"].dt.month == 12]
    t, td, th = l["retard"].mean() * 100, l[dec]["retard"].mean() * 100, l[~dec]["retard"].mean() * 100
    texte = (f"En {annee}, {0.fr(t, 0)} % des livraisons arrivent en retard. "
            f"En décembre, c'est {0.fr(td, 0)} % des livraisons, contre {0.fr(th, 0)} % le reste de "
            f"↪ l'année.")
    return texte, 0.verifier_nombres(texte, [annee, t, td, th])
for an in (2023, 2024, 2025):
    texte, orphelins = resume_livraisons(liv, an)
    print(texte, "| sans source :", orphelins)
```

```
En 2023, 27 % des livraisons arrivent en retard. En décembre, c'est 55 % des livraisons, contre 22 %
↪ le reste de l'année. | sans source : []
En 2024, 26 % des livraisons arrivent en retard. En décembre, c'est 58 % des livraisons, contre 21 %
↪ le reste de l'année. | sans source : []
En 2025, 27 % des livraisons arrivent en retard. En décembre, c'est 54 % des livraisons, contre 22 %
↪ le reste de l'année. | sans source : []
```

Les nombres sont orphelins s'ils ne correspondent à aucune des valeurs de permis. Ici, tous sont produits par le calcul : la liste est vide, par construction.

Corrigé 4.9

```
lt = liv[liv["date_commande"].dt.year == 2025]
part_c = (lt["transporteur"] == "Transporteur C").mean() * 100
retard_c, retard_ab = lt[lt["transporteur"] == "Transporteur C"]["retard"].mean() * 100,
↪ lt[lt["transporteur"] != "Transporteur C"]["retard"].mean() * 100
cinq = [f"Contexte : le transporteur C livre {0.fr(part_c, 0)} % des colis de 2025.",
        f"Constat : il est en retard {0.fr(retard_c, 0)} % du temps, contre {0.fr(retard_ab, 0)} % "
        f"↪ pour les deux autres.",
        "Pourquoi : l'écart existe aussi hors décembre ; il ne vient pas de la saison.",
        "Recommandation : confier moins de colis au transporteur C pendant trois mois et comparer.",
        "Décision demandée : accord pour ce test avant le 15 novembre."]
print("Objet : Livraisons : le transporteur C en retard une fois sur deux, test proposé")
print("\n".join(cinq)); print(0.lisibilite(" ".join(cinq)))
```

```
Objet : Livraisons : le transporteur C en retard une fois sur deux, test proposé
Contexte : le transporteur C livre 20 % des colis de 2025.
Constat : il est en retard 52 % du temps, contre 20 % pour les deux autres.
Pourquoi : l'écart existe aussi hors décembre ; il ne vient pas de la saison.
Recommandation : confier moins de colis au transporteur C pendant trois mois et comparer.
Décision demandée : accord pour ce test avant le 15 novembre.
{'phrases': 5, 'mots': 60, 'mots_par_phrase': 12.0, 'termes_techniques': 0, 'chiffres': 5}
```

Corrigé 4.10

Avec N commandes observées pendant la promotion, l'effet x (donc $N / (1 + x)$ commandes sans promotion) et la marge par commande m_0 hors promotion, une promotion dont la remise est réduite à la part p de l'actuelle rapporte $N (m_0 - p \cdot \Delta) - N m_0 / (1 + x)$, où $\Delta = m_0 - m_1$ est l'écart de marge par commande (hors promotion moins en promotion), à effet supposé inchangé. Elle est nulle pour

$$p^* = m_0 \cdot x / ((1 + x) \cdot \Delta).$$

```
N, delta = a["n_cmd"], a["mo_np"] - a["mo_p"]
for nom, xx in {"borne basse": a["e_bas"], "estimé": a["e"], "borne haute": a["e_haut"]}.items():
    pstar = a["mo_np"] * xx / ((1 + xx) * delta)
    inc100 = N * a["mo_p"] - N / (1 + xx) * a["mo_np"]
    print(f"{nom:12s}: effet {0.fr(xx * 100, 1, True)} % → conserver au plus {0.fr(pstar * 100, 0)} %
    ↪ de la remise ; incrément à 100 % : {0.fr(inc100 / 1000, 0, True)} k€")
```

```
borne basse : effet +13,5 % → conserver au plus 45 % de la remise ; incrément à 100 % : -25 k€
estimé      : effet +19,2 % → conserver au plus 61 % de la remise ; incrément à 100 % : -18 k€
borne haute : effet +25,1 % → conserver au plus 76 % de la remise ; incrément à 100 % : -11 k€
```

Même à l'estimation basse, il suffit de conserver moins de la moitié de la remise pour revenir à l'équilibre, à effet constant. Mais l'hypothèse est **fragile** : plus la remise baisse, moins elle attire de commandes, et l'effet n'est pas le même à 100 % et à 45 % de la remise. D'où le test aléatoire recommandé, qui mesure l'effet à une remise donnée.

Corrigé 4.11

```
js = j.set_index("date")
ca = js["chiffre_affaires"].resample("W-SUN").sum().iloc[1:-1]
v = (ca / ca.shift(52) - 1).dropna()
croissance = v.shift(1).rolling(26).median() # croissance d'ensemble des 26
    ↪ semaines précédentes
bruit = (v - croissance).shift(1).rolling(26).std()
avant, apres = (v.abs() >= 1.5 * v.shift(1).rolling(26).std())["2025-01-12:], ((v -
    ↪ croissance).abs() >= 1.5 * bruit)["2025-01-12:]
print("semaines signalées avant correction :", int(avant.sum()), "| après :", int(apres.sum()), "|
    ↪ signalées dans les deux cas :", int((avant & apres).sum()))
print("écart de 2025 corrigé de la croissance :", 0.fr((v - croissance)["2025-01-12:].min() * 100,
    ↪ 0, True), "à", 0.fr((v - croissance)["2025-01-12:].max() * 100, 0, True), "%")
```

```
semaines signalées avant correction : 10 | après : 5 | signalées dans les deux cas : 5
écart de 2025 corrigé de la croissance : -22 à +35 %
```

La correction retire la croissance d'ensemble et laisse les écarts à la tendance. Elle réduit le nombre de semaines signalées, et celles qui restent sont plus intéressantes. Elle ajoute une complexité (un second paramètre, la fenêtre de 26 semaines) qu'il faut documenter dans la fiche du rapport.

Corrigé 4.12

```
def controle_grandeur(d, lundi):
    cw = d["j"].set_index("date")["chiffre_affaires"].resample("W-MON", label="left",
    ↪ closed="left").sum()
    lundi = pd.Timestamp(lundi)
    ref = cw[cw.index < lundi].tail(26).median()
    return 0.3 <= cw.loc[lundi] / ref <= 3

abime = d["j"].copy(); abime.loc[abime["date"] == "2025-11-12", "chiffre_affaires"] *= 100
for nom, jj in {"données intactes": d["j"], "un jour × 100": abime}.items():
    print(nom, "→ contrôle d'ordre de grandeur :", "OK" if controle_grandeur({"j": jj}, "2025-11-10")
    ↪ else "ÉCHEC")
```

```
données intactes → contrôle d'ordre de grandeur : OK
un jour × 100 → contrôle d'ordre de grandeur : ÉCHEC
```

Le contrôle attrape l'erreur d'unité que les contrôles de complétude (jours présents, valeurs manquantes) ne voient pas : tous les jours sont là et rien ne manque, mais la semaine vaut quarante

fois la normale. Il faut le régler avec prudence (une semaine de fêtes peut doubler le chiffre) ; la fourchette de 0,3 à 3 est large à dessein.

Pistes des applications

Quelques pistes pour les « À vous » des applications.

Application 4.1 (livraisons). On place en annexe la définition du retard (date effective supérieure à la date promise) et le détail par mode de livraison. La page 3 (« tous les transporteurs reculent en décembre ») prévient l'objection « c'est peut-être juste un transporteur qui flanche en décembre » : elle montre que le recul est général, donc que le mix de transporteurs n'explique pas la hausse de décembre.

Application 4.2 (titres). Pour l'écart entre 2023 et 2025, on calcule la variation du chiffre d'affaires annuel puis on l'écrit avec sa comparaison.

```
ca_an = j.groupby("annee")["chiffre_affaires"].sum()
titre = f"Le chiffre d'affaires a progressé de {0.fr((ca_an[2025] / ca_an[2023] - 1) * 100, 0)} % en
↳ deux ans, de {0.fr(ca_an[2023] / 1000, 0)} à {0.fr(ca_an[2025] / 1000, 0)} k€"
print(titre, "| sans source :", 0.verifier_nombres(titre, [ca_an[2025] / ca_an[2023] * 100 - 100,
↳ ca_an[2023] / 1000, ca_an[2025] / 1000]))
```

Le chiffre d'affaires a progressé de 16 % en deux ans, de 1 139 à 1 325 k€ | sans source : []

Application 4.3 (brouillon). Les deux orphelins sont « 47 000 » (les remises valent 46 k€) et « -19 000 » (le solde est de -18 k€). Pour citer l'intervalle : « l'effet est de 19 %, entre 14 et 25 % » ; ces trois nombres passent le contrôle s'ils figurent dans la liste (`a["e"] * 100`, `a["e_bas"] * 100`, `a["e_haut"] * 100`).

Application 4.4 (éditions). L'édition d'été est la moins mauvaise en marge par commande, mais chaque édition n'a qu'un petit nombre de jours (63, 63 et 27) : la différence peut venir de la saison autant que de la remise. On ne conclut qu'après avoir comparé à la marge par commande **hors promotion de la même saison**.

Application 4.5 (sensibilité). L'incrément est positif sur toute la plage de l'intervalle dès que l'on ne conserve que 45 % de la remise ou moins (la grille le montre à 25 %). L'hypothèse cachée est l'indépendance de l'effet et de la remise.

Application 4.6 (panier moyen). Sur 10 semaines signalées, 8 sont des hausses (de 11 à 15 %) et 2 des baisses (de 11 à 13 %) : les deux sens existent, car le panier ne suit pas la croissance du chiffre d'affaires. Un humain ajouterait la raison connue (la promotion de juin ou de fin novembre, un nouvel assortiment) ou « à investiguer ».

Chapitre 5 : Présenter à des interlocuteurs non techniques — exercices et applications

Ce cahier prolonge le chapitre 5 : on y **prépare** une réunion (qui décide, que dire à qui), on **traduit** des chiffres et du jargon, on **vérifie** la lisibilité d'un support, on **écrit** une recommandation exécutable, un plan de dix minutes, une fiche de cadrage et un compte rendu. Il y a peu de calcul, beaucoup de **phrases** : écrivez-les vraiment, à voix haute si possible. Le cahier est autonome : il recharge ses données.

Applications

Application 5.1 — Un résultat, trois publics (section 5.1.3)

Objectif. Partir des chiffres de l'analyse des soldes et écrire la même conclusion pour trois personnes : la gérante (l'argent), le responsable logistique (la charge de travail), le financeur (le risque).

Étape 1 — les faits. On rassemble les chiffres, **arrondis comme on les dirait**.

```
faits = {"commandes en plus (%)": F["e"] * 100, "intervalle bas (%)": F["e_bas"] * 100, "intervalle  
↪ haut (%)": F["e_haut"] * 100,  
        "marge par commande hors soldes (€)": F["mo_np"], "marge par commande en soldes (€)":  
        ↪ F["mo_p"],  
        "marge perdue (€)": -F["incr"], "seuil de bascule (%)": F["seuil"] * 100, "simulations  
        ↪ négatives (%)": (SIM < 0).mean() * 100}  
print(pd.Series(faits).round(1).to_string())
```

commandes en plus (%)	19.2
intervalle bas (%)	13.5
intervalle haut (%)	25.1
marge par commande hors soldes (€)	32.1
marge par commande en soldes (€)	23.6
marge perdue (€)	17884.4
seuil de bascule (%)	35.8
simulations négatives (%)	99.0

Étape 2 — trois phrases. Complétez chacune avec **un seul chiffre principal**.

Public	Sa question	Votre phrase (à écrire)
Gérante	Combien cela me coûte-t-il ?	« Reconduire les soldes tels quels ferait perdre environ ... € de marge. »
Responsable logistique	Combien de colis en plus ?	« Les soldes ajoutent environ ... % de commandes à préparer. »
Financeur	Quel est le risque ?	« Dans ... % des simulations, ... ; il faudrait ... % de commandes en plus. »

Étape 3 — contrôle. Chaque phrase doit tenir en moins de vingt secondes à voix haute et ne contenir **aucun mot de jargon** (p-valeur, intervalle de confiance, log, résidu).

À vous. Écrivez une quatrième phrase pour un nouveau salarié du service des ventes, qui n'a jamais entendu parler de l'analyse.

Application 5.2 — Points, pour cent et « sur 100 » (sections 5.1.4 et 5.1.5)

Objectif. Dire les mêmes écarts de trois façons, puis choisir la plus honnête.

Étape 1 — le tableau des taux. Conversion par source de trafic.

```
c = pd.Series(F["conv"]).mul(100).round(2).sort_values(ascending=False)
t = pd.DataFrame({"conversion (%)": c, "pour 1 000 visites": (c * 10).round(0), "une visite sur": (100 / c).round(0)})
print(t.to_string())
```

	conversion (%)	pour 1 000 visites	une visite sur
email	8.68	87.0	12.0
direct	6.94	69.0	14.0
organique	4.02	40.0	25.0
referent	3.76	38.0	27.0
payant	3.01	30.0	33.0
reseaux	2.16	22.0	46.0

Étape 2 — points ou pour cent ? Entre le meilleur canal (l'e-mail) et le plus faible (les réseaux sociaux), calculez l'écart **en points** et le **rapport**.

```
haut, bas = c.iloc[0], c.iloc[-1]
print("écart :", round(haut - bas, 1), "points | rapport :", round(haut / bas, 1), "fois")
```

écart : 6.5 points | rapport : 4.0 fois

Étape 3 — la bonne phrase. Pour la gérante : « L'e-mail convertit ... fois mieux que les réseaux ; sur 1 000 visites, cela fait ... commandes de plus. » Pour un audit technique : « L'écart est de ... points. »

À vous. Même travail pour le taux de retard des trois transporteurs (F["retard_transp"]) : écrivez une phrase qui donne **la base de comparaison**.

Application 5.3 — Contraste et daltonisme (section 5.1.6)

Objectif. Vérifier qu'un support est lisible avant de le projeter.

Étape 1 — le contraste de la palette. Rapport de contraste de chaque couleur sur fond blanc, avec le verdict pour le texte courant (seuil de 4,5) et pour les grands éléments (seuil de 3).

```
t = 0.tableau_contrastes()
t["texte courant (≥ 4,5)"] = np.where(t["sur blanc"] >= 4.5, "oui", "non")
t["grands éléments (≥ 3)"] = np.where(t["sur blanc"] >= 3, "oui", "non")
print(t.to_string(index=False))
```

couleur	sur blanc	texte courant (≥ 4,5)	grands éléments (≥ 3)
bleu	4.4	non	oui
orange	3.2	non	oui
aqua	2.8	non	non
violet	8.6	oui	oui
rouge	4.0	non	oui
gris du texte	7.9	oui	oui
gris muet	3.6	non	oui

Étape 2 — un fond sombre. Les mêmes couleurs sur le fond bleu nuit #14213d sont-elles plus lisibles ?

```
for nom, c in {"orange": 0.S.ORANGE, "aqua": 0.S.AQUA, "blanc": "#ffffff", "gris muet":
    0.S.MUET}.items():
    print(f"{nom:10s} sur #14213d : {0.contraste(c, '#14213d'):.1f}")
```

```
orange    sur #14213d : 5.0
aqua      sur #14213d : 5.7
blanc     sur #14213d : 16.0
gris muet sur #14213d : 4.4
```

Étape 3 — décision. Pour chaque couleur, notez où l'utiliser : texte courant, titre, trait, fond.

À vous. Choisissez deux couleurs pour distinguer « Boutique » et « Site » dans un graphique projeté, de façon qu'on les distingue aussi avec la protanopie (figure 5.1.6 du livre) : lesquelles, et quel autre signe ajoutez-vous (étiquette directe, trait pointillé) ?

Application 5.4 — Dire l'incertitude, formuler la recommandation (sections 5.2.3 et 5.2.4)

Objectif. Passer de la simulation à une phrase honnête, puis à une recommandation qu'on peut exécuter.

Étape 1 — ce que dit la simulation. La direction (le signe) et l'ampleur (la fourchette).

```
bas, med, haut = np.percentile(SIM, [5, 50, 95])
print("part de simulations négatives :", round((SIM < 0).mean() * 100), "%")
print("fourchette à 90 % :", round(bas, -2), "à", round(haut, -2), "| médiane :", round(med, -2))
```

```
part de simulations négatives : 99 %
fourchette à 90 % : -32500.0 à -3400.0 | médiane : -17900.0
```

Étape 2 — la phrase. Complétez : « Nous sommes sûrs de la ... : Nous le sommes moins de l'... : de ... à ... €. »

Étape 3 — la recommandation en cinq champs. Remplissez la fiche pour la décision « tester une remise à 10 % ».

Champ	Contenu
Qui	...
Quoi	...
Quand	...
Combien (coût, risque maximal)	...
Comment mesurer (indicateur, seuil, date du bilan)	...

À vous. Imaginez que la gérante réponde : « Je ne peux pas attendre le bilan. » Quelle **version courte** de la recommandation proposez-vous ?

Application 5.5 — Un plan de dix minutes, chronométré (sections 5.2.2 et 5.2.8)

Objectif. Construire le minutage, puis vérifier qu'il tient.

Étape 1 — le plan. On le range dans un tableau avec la durée de chaque bloc.

```
plan = pd.DataFrame({"bloc": ["Réponse", "Preuve 1", "Preuve 2", "Preuve 3", "Recommandation",
↪ "Décision demandée", "Questions"],
                    "minutes": [1, 1.5, 1.5, 1.5, 1.5, 0.5, 2.5]})
print(plan.to_string(index=False), "\ntotal :", plan["minutes"].sum(), "minutes")
```

```
      bloc  minutes
Réponse      1.0
Preuve 1      1.5
Preuve 2      1.5
Preuve 3      1.5
Recommandation 1.5
Décision demandée 0.5
Questions      2.5
total : 10.0 minutes
```

Étape 2 — le budget de mots. À l'oral on dit environ 120 à 140 mots par minute. Calculez le nombre de mots **maximum** pour la partie parlée (hors questions).

```
parole = plan.loc[plan["bloc"] != "Questions", "minutes"].sum()
print("mots maximum :", int(parle * 120), "à", int(parle * 140))
```

```
mots maximum : 900 à 1050
```

Étape 3 — votre brouillon. Écrivez le texte de la « Réponse » (une minute) puis comptez ses mots avec `len(texte.split())`.

À vous. Ajoutez une marge de sécurité de deux minutes : quel bloc supprimez-vous en premier si la réunion est raccourcie, et pourquoi ?

Application 5.6 — Une fiche de cadrage (section 5.3.5)

Objectif. Transformer une demande floue (« les livraisons posent problème en décembre ») en une question testable.

Étape 1 — ce que disent déjà les données. Avant l'entretien, on regarde.

```
t = pd.DataFrame({"retard (%)": {"décembre": F["retard_dec"] * 100, "autres mois":
↪ F["retard_hors_dec"] * 100, "toute l'année": F["retard"] * 100}}).round(1)
print(t.to_string())
```

	retard (%)
décembre	55.5
autres mois	21.6
toute l'année	26.6

Étape 2 — les transporteurs. Le retard par transporteur, et la part de colis abîmés.

```
tr = pd.DataFrame({"retard (%)": F["retard_transp"], "colis abîmés (%)":
↳ F["abime_transp"]}).mul(100).round(1)
print(tr.to_string())
```

	retard (%)	colis abîmés (%)
Transporteur A	16.0	0.9
Transporteur B	26.6	1.4
Transporteur C	51.0	4.1

Étape 3 — la fiche. Remplissez-la, en une phrase par champ.

```
fiche = {"demande initiale": "...", "question testable": "...", "décision que la réponse éclaire": "...",
↳ "indicateur et définition": "...",
    "périmètre et période": "...", "données et limites": "...", "critère de succès": "...", "délai":
↳ "...", "validé par": "..."}
print(len(fiche), "champs à remplir")
```

9 champs à remplir

À vous. Rédigez les **six questions** de l'entretien avec le responsable logistique (une question ouverte, pas de question qui suggère la réponse).

Application 5.7 — Compte rendu et désaccord de chiffres (sections 5.2.8 et 5.4.4)

Objectif. Réconcilier deux chiffres d'affaires, puis rédiger un compte rendu en cinq lignes.

Étape 1 — deux chiffres. La gérante parle du chiffre d'affaires **brut**, la comptable du chiffre d'affaires **net de retours**.

```
cmd = pd.read_csv(os.path.join(D, "commandes.csv")); lig = pd.read_csv(os.path.join(D,
↳ "lignes_commande.csv")); ret = pd.read_csv(os.path.join(D, "retours.csv"))
x = lig.merge(cmd[["id_commande", "date_commande", "canal"]], on="id_commande"); x =
↳ x[x["date_commande"] >= "2025-01-01"]
x = x.merge(ret[["id_ligne", "montant_rembourse"]], on="id_ligne",
↳ how="left").fillna({"montant_rembourse": 0})
t = x.groupby("canal")[["montant", "montant_rembourse"]].sum()
t.loc["Total"] = t.sum(); t["net"] = t["montant"] - t["montant_rembourse"]
print(t.round(0).rename(columns={"montant": "brut", "montant_rembourse": "remboursé"}).to_string())
```

	brut	remboursé	net
canal			
Boutique	560974.0	18456.0	542518.0
Réseaux	146074.0	9429.0	136645.0
Site	617715.0	56583.0	561132.0
Total	1324764.0	84469.0	1240295.0

Étape 2 — l'écart expliqué. Vérifiez que « brut – remboursé = net » est vrai pour chaque ligne du tableau, puis écrivez une phrase qui nomme les deux définitions.

Étape 3 — le compte rendu. Cinq lignes : **décision prise, responsable, échéance, indicateur de suivi, point resté ouvert**. Rédigez-le pour la réunion de jeudi.

À vous. Rédigez le message de **correction** que vous enverriez si, le lendemain, vous découvriez que le brut avait été présenté comme s'il était net.

Exercices

Exercice 5.1 ★ — Qui décide ? (section 5.1.1)

Une réunion porte sur la décision d'**ouvrir un deuxième point de retrait** dans une autre ville. Sont présents : la gérante, la comptable, le responsable logistique, une personne du marketing et un conseiller extérieur qui n'aime pas les chiffres. Classez chacun dans l'un des quatre rôles (décideur, expert, utilisateur, sceptique) et dites, pour chacun, **ce qu'il voudra savoir en premier**.

Exercice 5.2 ★ — Points ou pour cent ? (section 5.1.5)

Dans l'année, le taux de conversion passe de 4,8 % à 5,3 %. Écrivez la variation **en points, en pour cent et en « sur 1 000 »**, puis dites laquelle vous retenez pour une présentation à la gérante, et pourquoi.

Exercice 5.3 ★★ — Traduire le jargon (section 5.1.4)

Traduisez en une phrase pour la gérante, sans aucun terme technique :

1. « Le coefficient de la variable promotion est de 0,175, avec un intervalle de confiance à 95 % de [0,127 ; 0,224]. »
2. « La régression est ajustée sur la saison, le jour de la semaine et la tendance. »
3. « Le test n'a pas rejeté l'hypothèse nulle, la p-valeur est de 0,31. »

Exercice 5.4 ★★ — Le bon texte sur le bon fond (section 5.1.6)

Une diapositive a un fond blanc et trois textes : un titre en orange, une légende en gris muet, un chiffre en gris foncé. Calculez le contraste de chacun avec 0. contraste et corrigez ce qui ne passe pas. Même question sur un fond gris très clair #f2f2f2.

Exercice 5.5 ★ — La réponse d'abord (section 5.2.1)

Voici l'ouverture d'une présentation : « Je vais vous présenter la méthode, puis les données, puis les résultats. » Réécrivez-la en **une phrase** qui donne la réponse, et ajoutez la phrase qui annonce ce que vous demandez à la salle.

Exercice 5.6 ★★ — Des titres qui disent quelque chose (section 5.2.2)

Pour chacune des trois figures, remplacez le titre-étiquette par un titre-phrase **vrai** (vérifiez le chiffre avec les données) :

1. « Retard par transporteur » (barres des trois transporteurs).
2. « Conversion par source » (barres des six sources).
3. « Marge par commande, soldes ou non » (deux barres).

Exercice 5.7 ★★ — Une phrase qui dit l'incertitude (section 5.2.3)

L'effet des soldes sur les commandes est estimé à +19 %, avec un intervalle de 14 à 25 %. Écrivez trois phrases : une pour la gérante, une pour un auditeur critique, une **fausse** (qui promet une précision que l'on n'a pas). Calculez avec F les trois chiffres dont vous avez besoin.

Exercice 5.8 ★★ — Rendre une recommandation exécutable (section 5.2.4)

On vous propose : « Il faudrait améliorer la livraison en décembre. » Réécrivez-la en cinq champs (qui, quoi, quand, combien, comment mesurer), en vous appuyant sur les transporteurs et le mois de décembre.

Exercice 5.9 ★★★ — Une erreur découverte en séance (sections 5.2.6 et 5.2.7)

Pendant la réunion, la comptable demande : « Ce +19 %, c'est 19 points de conversion en plus ? » Vous voyez que la phrase de votre diapositive peut être mal lue. 1) Calculez ce que donnerait réellement +19 % sur une conversion de 4,78 %. 2) Écrivez la **réponse orale** (trois phrases) et la **correction de la diapositive**.

Exercice 5.10 ★ — Une question qui se teste (section 5.3.1)

Transformez chacune de ces demandes en question testable, avec l'indicateur :

1. « Je veux un tableau de bord. »
2. « Il faut comprendre pourquoi on perd des clients. »
3. « Peux-tu regarder si la publicité marche ? »
4. « Il y a trop de retours. »

Exercice 5.11 ★★ — Entretien et carte des parties prenantes (sections 5.3.2 et 5.3.3)

Vous devez cadrer une analyse des retards de livraison. 1) Placez sur une grille pouvoir/intérêt : la gérante, le responsable logistique, le transporteur C, le service client, la comptable. 2) Écrivez cinq questions pour la gérante, dont une qui commence par « Si vous aviez la réponse demain, qu'est-ce que vous feriez de différent ? ».

Exercice 5.12 ★★ — Négocier un délai (section 5.4.2)

La gérante demande, un lundi : « Peux-tu me faire une analyse complète des retards, avec les causes, pour mercredi ? » Il vous faut une semaine pour l'analyse complète. Écrivez votre réponse selon la formule : **reconnaître, énoncer le coût, proposer, demander un choix**, en proposant deux options.

Exercice 5.13 ★★★ — Deux chiffres de conversion (section 5.4.4)

Une collègue annonce une conversion de 4,8 %, un autre de 3,4 %. Tous deux ont utilisé sessions_web.csv et aucun n'a fait d'erreur de calcul. Calculez la conversion **par source**, retrouvez le chiffre global comme **moyenne pondérée**, cherchez quel sous-ensemble de sources donne 3,4 %, et écrivez la **phrase de réconciliation** à envoyer aux deux.

Corrigé 5.1

Personne	Rôle	Ce qu'elle voudra savoir en premier
Gérante	Décideur	Combien cela coûte-t-il, et quand est-ce rentable ?
Comptable	Expert (chiffres)	D'où viennent les chiffres, avec quelles hypothèses ?
Responsable logistique	Utilisateur	Comment le point de retrait sera-t-il approvisionné, par qui ?
Marketing	Utilisateur	Quelle clientèle, quelle communication ?
Conseiller extérieur	Sceptique	Pourquoi faire confiance à ces chiffres ? Que se passe-t-il si c'est faux ?

Le sceptique ne se convainc pas avec un tableau : il faut **un exemple concret** et la mention de ce que l'on ne sait pas.

Corrigé 5.2

```
a, b = 4.8, 5.3
print("points :", round(b - a, 1), "| relatif :", round((b / a - 1) * 100, 1), "% | sur 1 000 :",
      ↪ round(a * 10), "→", round(b * 10))
```

```
points : 0.5 | relatif : 10.4 % | sur 1 000 : 48 → 53
```

La conversion monte de **0,5 point**, soit **+10 %** en relatif, soit **5 commandes de plus pour 1 000 visites** (de 48 à 53). Pour la gérante, on retient « **cinq commandes de plus pour 1 000 visites** » : concret, sans ambiguïté sur la base. Les « +10 % » seuls sont à éviter (on croit à un bond de dix points), les « 0,5 point » seuls paraissent minuscules.

Corrigé 5.3

- « Les jours de soldes, la boutique reçoit environ **19 % de commandes de plus** ; avec un peu de marge d'erreur, entre 14 et 25 %. »
- « Nous avons comparé les jours de soldes à des jours **comparables** (même saison, même jour de la semaine, même tendance), pas à l'ensemble de l'année. »
- « Avec ces données, **nous ne pouvons pas affirmer** qu'il y a une différence ; cela ne veut pas dire qu'il n'y en a pas, plutôt que le test n'est pas assez précis pour le dire. »

La phrase 3 est la plus piègeuse : « il n'y a pas de différence » est **fausse**.

Corrigé 5.4

```
for nom, c in {"orange": 0.S.ORANGE, "gris muet": 0.S.MUET, "gris foncé": 0.S.ENCRE2}.items():
    print(f"{nom:10s} sur blanc : {0.contraste(c):.1f} | sur #f2f2f2 : {0.contraste(c,
      ↪ '#f2f2f2'):.1f}")
```

```
orange      sur blanc : 3.2 | sur #f2f2f2 : 2.9
gris muet   sur blanc : 3.6 | sur #f2f2f2 : 3.2
gris foncé  sur blanc : 7.9 | sur #f2f2f2 : 7.1
```

L'orange (3,2) ne passe pas pour du texte, même gros titre à la limite (3) ; il tombe sous 3 sur gris clair. Le gris muet est tout juste suffisant pour une légende de grande taille. Le **gris foncé** passe partout. Correction : titre en gris foncé ou bleu foncé, l'orange réservé à un **trait** ou à un **gros chiffre mis en valeur**, légende en gris foncé. Sur un gris clair, tout baisse : on évite d'ajouter du gris sur du gris.

Corrigé 5.5

« **Reconduire les soldes tels quels ferait perdre environ 18 000 € de marge ; je vous propose de tester une remise plus faible avant la prochaine édition.** » Puis : « **Je vous demande aujourd'hui d'approuver ce test sur la moitié des produits.** » La première phrase donne la réponse et la décision attendue ; la méthode viendra, si on la demande, **après**.

Corrigé 5.6

```
print({k: round(v * 100) for k, v in F["retard_transp"].items()})
print({k: round(v * 100, 1) for k, v in F["conv"].items()})
print(round(F["mo_np"], 1), round(F["mo_p"], 1), round((F["mo_p"] / F["mo_np"] - 1) * 100))
```

```
{'Transporteur A': 16, 'Transporteur B': 27, 'Transporteur C': 51}
{'direct': 6.9, 'email': 8.7, 'organique': 4.0, 'payant': 3.0, 'referent': 3.8, 'reseaux': 2.2}
32.1 23.6 -26
```

1. « **Un colis sur deux livré par le transporteur C arrive en retard**, contre un sur six chez le transporteur A. » (51 % contre 16 %.)
2. « **L'e-mail convertit quatre fois mieux que les réseaux sociaux** : 8,7 % des visites contre 2,2 %.
3. « **En soldes, chaque commande rapporte 24 € de marge au lieu de 32 €** (un quart de moins). »

Corrigé 5.7

```
print("direction :", round((SIM < 0).mean() * 100), "% de pertes | ampleur :",
      ↵ round(np.percentile(SIM, 5), -2), "à", round(np.percentile(SIM, 95), -2), "€")
```

```
direction : 99 % de pertes | ampleur : -32500.0 à -3400.0 €
```

Pour la gérante : « **Nous sommes sûrs de la direction** : les soldes font perdre de la marge, dans 99 simulations sur 100. **Nous le sommes moins de l'ampleur** : la perte est comprise entre 3 400 et 32 500 €, la meilleure estimation étant d'environ 18 000 €. » Pour l'auditeur : « L'effet estimé sur les commandes est de +19 % (intervalle à 95 % : de 14 à 25 %) ; la marge perdue est de 18 000 € (de 3 400 à 32 500 € à 90 %), et la probabilité d'une marge positive est de moins de 1 %. » La phrase fausse : « Les soldes font gagner exactement **19,2 %** de commandes. » (fausse précision : le chiffre est une estimation, jamais exact.) Les trois chiffres : 19 %, de 14 à 25 %, et le pourcentage de simulations négatives.

Corrigé 5.8

Champ	Contenu
Qui Champ Quoi	Le responsable logistique, avec le transporteur C. Confier à un autre transporteur les colis de novembre à décembre qui sont aujourd'hui pour le transporteur C, sur un échantillon de la moitié des colis.
Quand	Décision avant le 15 octobre ; essai du 15 novembre au 31 décembre.
Combien	Coût supplémentaire à chiffrer avant la décision ; risque maximal : un mois de surcoût sur la moitié des colis.
Comment mesurer	Taux de retard (livraison après la date promise), comparé au transporteur C sur la même période ; bilan le 10 janvier.

Le retard en décembre (55 %) est trois fois plus élevé que dans le reste de l'année (21,6 %) et le transporteur C est le plus en retard (51 %) : le test **vérifie** que le transporteur est bien en cause, plutôt que le mois.

Corrigé 5.9

```
c = F["conv_globale"] * 100
print("conversion :", round(c, 2), "% → avec +19 % :", round(c * (1 + F["e"]), 2), "% (+", round(c *
↪ F["e"], 2), "point)")
```

```
conversion : 4.78 % → avec +19 % : 5.7 % (+ 0.92 point)
```

« Non, c'est **19 % de commandes de plus**, pas 19 points de conversion : sur une conversion d'environ 4,8 %, cela fait **un peu moins d'un point** de plus (de 4,8 à 5,7 %). Votre question est la bonne : ma diapositive prête à confusion, je la corrige tout de suite. » Correction de la diapositive : « **+19 % de commandes (environ 9 commandes de plus pour 1 000 visites)** ». On remercie, on corrige **devant tout le monde**, on envoie la version corrigée avec le compte rendu.

Corrigé 5.10

1. « Quelles trois décisions la gérante prendra-t-elle chaque lundi, et de quels chiffres a-t-elle besoin pour chacune ? » → on part des décisions, pas du tableau.
2. « Parmi les clients de 2024, quelle part n'a pas recommandé en 2025, et ce taux varie-t-il selon le canal d'acquisition ? » (indicateur : taux de réachat à douze mois.)
3. « À budget publicitaire égal, les semaines de dépense supérieure ont-elles plus de commandes, une fois la saison prise en compte ? » (indicateur : commandes quotidiennes ajustées de la saison.)
4. « Le taux de retour dépasse-t-il le niveau habituel de la catégorie, et dans quelle catégorie ? » (indicateur : retours / lignes vendues, par catégorie.)

Corrigé 5.11

	Pouvoir élevé	Pouvoir faible
Intérêt élevé	Gérante, responsable logistique (gérer de près)	Service client (informer)
Intérêt faible	Comptable (tenir informée)	Transporteur C (surveiller, sans le mettre dans la salle)

Questions pour la gérante : (1) « Qu'est-ce qui vous a fait poser la question ? » (2) « Quelle décision dépend de la réponse ? » (3) « Qu'appellez-vous un retard ? » (4) « Qu'est-ce qui vous ferait changer de transporteur ? » (5) « **Si vous aviez la réponse demain, qu'est-ce que vous feriez de différent ?** » La dernière est la plus utile : elle sépare une curiosité d'une décision.

Corrigé 5.12

« Je comprends que vous en ayez besoin vite. Une analyse complète des causes demande **une semaine**, parce qu'il faut croiser les livraisons, les transporteurs et les mois. Je peux vous proposer deux choses : (a) pour mercredi, une **première lecture** de ce qu'on voit déjà (le retard en décembre, par transporteur), sans les causes ; (b) l'analyse complète pour lundi prochain. Laquelle préférez-vous, ou voulez-vous les deux ? » Elle reconnaît, chiffre le coût, **propose**, demande un **choix**. Elle ne dit pas non, et ne promet pas ce qu'on ne pourra pas tenir.

Corrigé 5.13

```
sess = pd.read_csv(os.path.join(D, "sessions_web.csv"))
g = sess.groupby("source")["commande"].agg(["mean", "size"])
print("global :", round((g["mean"] * g["size"]).sum() / g["size"].sum() * 100, 2), "%")
acquis = sess[~sess["source"].isin(["direct", "email"])]
print("sans direct ni e-mail :", round(acquis["commande"].mean() * 100, 2), "% sur", len(acquis),
      ↪ "sessions")
```

```
global : 4.78 %
sans direct ni e-mail : 3.44 % sur 82564 sessions
```

Les deux chiffres sont **justes** : le premier est la conversion de **toutes** les sessions (4,8 %), le second celle des seules sessions qui ne viennent ni de l'accès direct ni de l'e-mail (3,4 %), c'est-à-dire du trafic « à conquérir », moins enclin à acheter que les clients déjà acquis (l'e-mail convertit à 8,7 %, l'accès direct à 6,9 %). Phrase de réconciliation : « **Nos deux chiffres sont justes** : 4,8 % est la conversion de toutes les visites ; 3,4 % est celle des visites hors accès direct et e-mail. Je propose d'écrire les deux définitions dans le dictionnaire (« conversion globale » et « conversion du trafic acquis ») et de dire toujours laquelle on cite. » On ne cherche pas qui a tort : on cherche **la définition**.

Pistes des applications

Quelques pistes pour les « À vous » des applications.

Application 5.1 (nouveau salarié). Une phrase de moins de vingt mots, avec une image : « Quand on fait des soldes, on vend environ un cinquième de commandes en plus, mais chaque vente rapporte moins : au total, on gagne moins d'argent. »

Application 5.2 (retard des transporteurs). « Un colis sur six est en retard chez le transporteur A, un sur quatre chez le B, **un sur deux chez le C.** » La base : le même ensemble de colis livrés dans l'année.

Application 5.3 (palette). Le bleu et le violet distinguent bien les deux canaux et restent distincts en protanopie (le violet est beaucoup plus sombre) ; on **ajoute l'étiquette directe** au bout de chaque courbe plutôt que la légende, et un trait pointillé pour l'un des deux.

Application 5.4 (version courte). « Si on ne peut pas attendre le bilan : **limiter la remise à 10 % sur la moitié des produits pendant la moitié de la période**, ce qui plafonne la perte, et décider du reste après lecture des premiers jours. »

Application 5.5 (marge de sécurité). On coupe d'abord la **preuve 3** (la plus technique : le seuil et la simulation), qui reste en annexe, car les deux premières preuves portent déjà la décision.

Application 5.6 (entretien logistique). (1) Quand avez-vous remarqué le problème ? (2) Quelles commandes sont concernées ? (3) Qu'appellez-vous un retard ? (4) Qu'avez-vous déjà essayé ? (5) Qu'est-ce qui changerait pour vous si le retard baissait de moitié ? (6) Qui d'autre faut-il interroger ?

Application 5.7 (correction). « **Correction** : le chiffre de 1 324 764 € présenté hier était un chiffre d'affaires **brut**. Net de retours, il est de 1 240 295 €. L'écart (84 469 €) correspond aux remboursements. Cela ne change pas la recommandation. Je mets à jour la diapositive 4 et le dictionnaire. »

Projet du volume et auto-évaluation

— exercices et applications

Ce chapitre du cahier clôt le volume IV. Il contient le **projet du volume** : à partir d'une demande floue, concevoir un **tableau de bord d'une page** et une **courte présentation** pour une décision précise (que faire des transporteurs avant décembre ?), puis l'**auto-évaluation** (quarante questions). Il ne demande aucune notion nouvelle : chaque étape s'appuie sur un chapitre du livre. Tous les graphiques sont produits par le code ; aucun logiciel de tableau de bord ou de présentation n'est exécuté (les maquettes sont dessinées avec matplotlib).

Projet du volume

P.1 La demande, et sa traduction en question

La responsable de la logistique vous écrit : « *Les clients râlent à cause des retards de livraison, surtout en fin d'année. Tu peux me faire un tableau de bord ? Et la gérante voudrait qu'on en parle lundi.* » Voilà une demande **floue** : « tableau de bord » est une solution, pas un besoin. Vous menez un court entretien (livre, 5.3) et vous en tirez une **fiche de cadrage** :

Élément	Réponse
Qui décide, et de quoi ?	La gérante, avec la responsable logistique : faut-il réduire la part du transporteur le plus en retard avant décembre ?
Quelle question testable ?	Si l'on transfère les colis du transporteur C vers le transporteur A, de combien baisse la part de livraisons en retard, et à quel prix ?
Qui lira le tableau de bord, et quand ?	La responsable logistique, chaque lundi , en moins de cinq secondes pour savoir si la situation est normale.
Indicateur principal	Part des livraisons à l'heure (délai total inférieur ou égal au délai promis de 6 jours).
Ce que l'on ne sait pas	Si le transporteur A absorbera 20 % de colis en plus sans se dégrader ; le coût réel par colis.

À retenir. Un livrable se commande par une **décision**, pas par un outil. Écrire la question et la fiche de cadrage **avant** de dessiner est ce qui évite de produire un beau tableau de bord que personne n'utilise.

La suite suit neuf étapes, chacune appuyée sur un chapitre du livre :

Étape	Question	Chapitre du livre
P.2 Les chiffres	Quels indicateurs, calculés une seule fois ?	volume III, 6
P.3 Choisir les graphiques	Quel graphique pour quelle question ?	1.1
P.4 Le tableau de bord	Comment l'organiser sur une page ?	1.2, 2.3
P.5 La vérification	Est-il lisible, accessible, honnête ?	1.3, 1.4
P.6 Le scénario	Que se passe-t-il si l'on transfère les colis ?	volume III, 13
P.7 Le récit	Quel enchaînement en cinq diapositives ?	4.1, 4.3
P.8 Le texte	Le message de cinq lignes et les questions attendues	4.2, 5.2
P.9 La critique	Qu'est-ce que cela ne dit pas ?	5.2

Les données. `donnees/livraisons.csv` (19 420 commandes livrées par les canaux Site et Réseaux, 2023–2025 : transporteur, dates, retard, colis abîmés). Elles sont **simulées** ; les coûts de la suite sont des **hypothèses**, signalées comme telles.

P.2 Étape 1 : les chiffres, calculés une seule fois

On calcule tous les indicateurs **une fois**, dans une fonction, pour que le tableau de bord, le texte et la présentation donnent les mêmes nombres (volume III, 6.1).

```
import sys
sys.path.insert(0, "build")
import numpy as np, pandas as pd
import matplotlib.pyplot as plt
from style import setup, BLEU, ORANGE, AQUA, VIOLET, ROUGE, MUET, ENCRE, ENCRE2
setup()

liv = pd.read_csv("donnees/livraisons.csv", parse_dates=["date_commande", "date_livraison"])
liv["annee"], liv["mois"] = liv["date_commande"].dt.year, liv["date_commande"].dt.month
liv["semaine"] = liv["date_commande"].dt.to_period("W-SUN").dt.start_time
liv["delai"] = (liv["date_livraison"] - liv["date_commande"]).dt.days
a25 = liv[liv["annee"] == 2025]

def wilson(k, n, z=1.96):
    p = k / n
    c = (p + z ** 2 / (2 * n)) / (1 + z ** 2 / n)
    h = z * np.sqrt(p * (1 - p) / n + z ** 2 / (4 * n ** 2)) / (1 + z ** 2 / n)
    return c - h, c + h

par_transp = a25.groupby("transporteur").agg(colis=("retard", "size"), retards=("retard", "sum"),
    ↪ abimes=("colis_abime", "sum"))
par_transp["taux_retard"] = par_transp["retards"] / par_transp["colis"]
par_transp[["bas", "haut"]] = [wilson(k, n) for k, n in zip(par_transp["retards"],
    ↪ par_transp["colis"])]
print(par_transp.round(3))
print("à l'heure en 2025 :", round((1 - a25["retard"].mean()) * 100, 1), "% | en décembre :", round((1
    ↪ - a25[a25["mois"] == 12]["retard"].mean()) * 100, 1), "%")
```

	colis	retards	abimes	taux_retard	bas	haut
transporteur						
Transporteur A	3363	516	32	0.153	0.142	0.166
Transporteur B	2663	704	41	0.264	0.248	0.281
Transporteur C	1478	771	62	0.522	0.496	0.547

à l'heure en 2025 : 73.5 % | en décembre : 46.0 %

Lecture. En 2025, 73,5 % des livraisons arrivent à l'heure, mais seulement 46,0 % en décembre. Le transporteur C est en retard dans 52 % des cas (intervalle de 49,6 à 54,7 %), le B dans 26 % et le A dans 15 % : les intervalles ne se recouvrent pas, la hiérarchie est nette.

P.3 Étape 2 : choisir les graphiques

On part de **la question**, pas du graphique (livre, 1.1) : chaque question appelle une forme.

Question de la responsable	Forme retenue	Pourquoi
Sommes-nous dans la normale cette semaine ?	Chiffre clé avec écart à la référence	Lecture en une seconde
La situation se dégrade-t-elle ?	Courbe hebdomadaire avec limites de contrôle	Évolution dans le temps ; le bruit est montré
Quel transporteur pose problème ?	Barres ordonnées avec intervalles de confiance	Comparaison de catégories ; l'incertitude est visible
Est-ce pire en décembre, pour tous ?	Petits multiples (transporteurs × mois)	Compare à échelle commune

Un camembert (parts de colis par transporteur) ou une carte thermique 3D n'auraient répondu à **aucune** de ces questions.

P.4 Étape 3 : le tableau de bord d'une page

On assemble les visuels sur **une page**, avec le plus important en haut à gauche, un titre qui dit la conclusion, des étiquettes directes et la palette du livre (livre, 2.3).

La fonction qui dessine la page (une trentaine de lignes de matplotlib : quatre chiffres clés, une courbe, des barres avec intervalles, trois petits multiples) est dans le code du cahier ; l'appel produit la figure.

P.5 Étape 4 : le vérifier

Avant de l'envoyer, on le passe au crible (livre, 1.3 et 1.4 ; test des cinq secondes) : le message se lit-il en cinq secondes ? Les couleurs se distinguent-elles pour un lecteur daltonien ? Le texte est-il assez contrasté ? Un **calcul** vaut mieux qu'une impression.

```
def luminance(hexa):
    r, g, b = [int(hexa[i:i + 2], 16) / 255 for i in (1, 3, 5)]
    f = lambda c: c / 12.92 if c <= 0.03928 else ((c + 0.055) / 1.055) ** 2.4
    return 0.2126 * f(r) + 0.7152 * f(g) + 0.0722 * f(b)

def contraste(a, b):
    la, lb = sorted([luminance(a), luminance(b)], reverse=True)
    return (la + 0.05) / (lb + 0.05)

for nom, c in [("rouge (décembre)", ROUGE), ("gris (autres transporteurs)", MUET), ("bleu (courbe)",
↵ BLEU), ("encre (texte)", ENCRE2)]:
    print(f"{nom:28s} contraste avec le fond blanc : {contraste(c, '#ffffff'):.1f}")
```

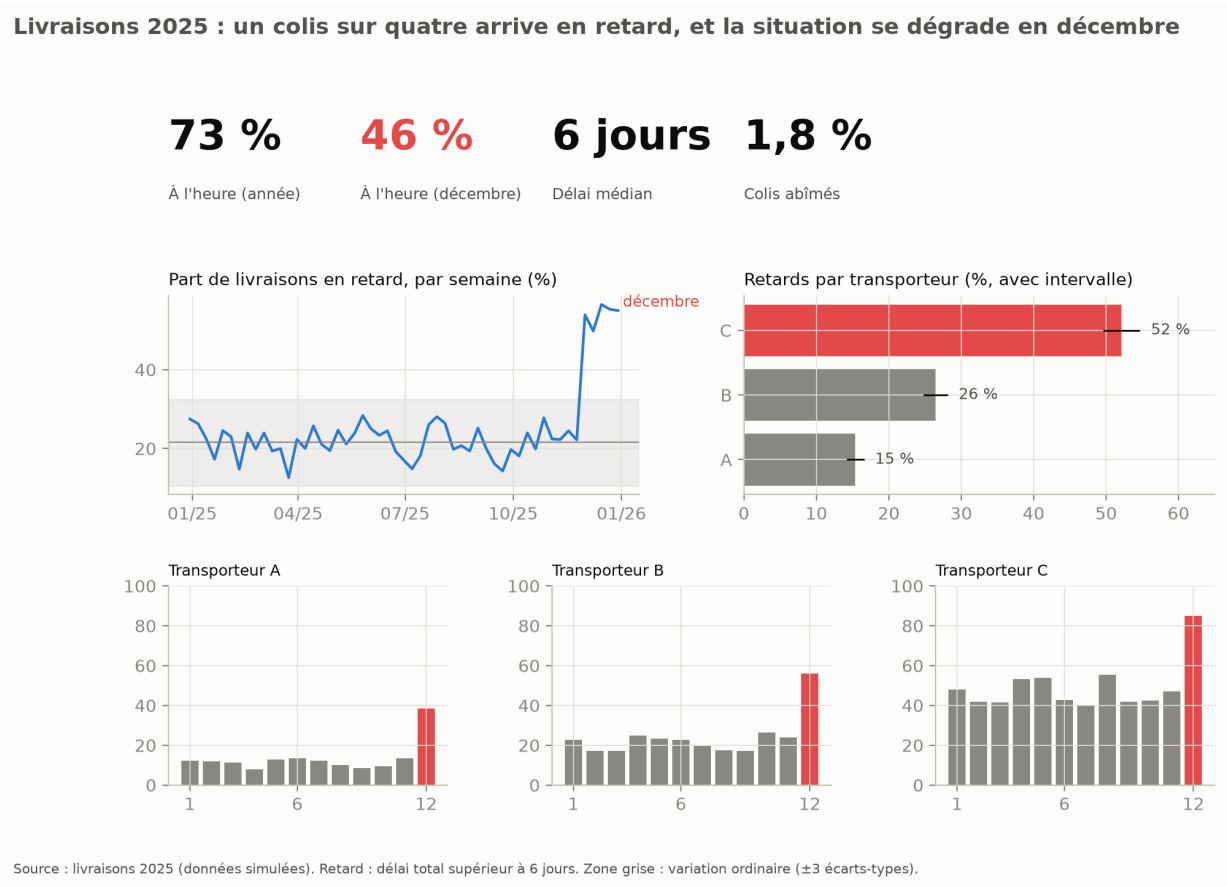


Figure 25: Tableau de bord d'une page : chiffres clés, courbe hebdomadaire des retards, retards par transporteur avec intervalles, et petits multiples par mois.

rouge (décembre)	contraste avec le fond blanc : 4.0
gris (autres transporteurs)	contraste avec le fond blanc : 3.6
bleu (courbe)	contraste avec le fond blanc : 4.4
encre (texte)	contraste avec le fond blanc : 7.9

Les gris et les rouges se **distinguent** surtout par leur **luminosité** (ce que les daltoniens perçoivent), et non par leur teinte : c'est ce qui rend le rouge de décembre lisible pour tous. Le seuil usuel pour du texte est de 4,5 ; les éléments graphiques demandent 3. On retient aussi qu'**aucune information ne repose sur la couleur seule** : décembre est aussi **étiqueté** sur la courbe.

P.6 Étape 5 : le scénario

La décision demande un **chiffre** : de combien baisserait la part de retards si l'on transférait les colis du transporteur C vers le A ? On le calcule à partir des taux observés en 2025 et de la part de chaque transporteur, avec l'**hypothèse** que A absorbe le surplus sans se dégrader (volume III, 13.3). L'incertitude vient des taux estimés : on la propage par simulation.

```
parts = par_transp["colis"] / par_transp["colis"].sum()
dec = a25[a25["mois"] == 12].groupby("transporteur")["retard"].agg(["sum", "size"])
rng = np.random.default_rng(7)
def taux_global(taux, p): return float((taux * p).sum())
actuel_dec = taux_global(dec["sum"] / dec["size"], dec["size"] / dec["size"].sum())
nouv = dec["size"].copy(); nouv["Transporteur A"] += nouv["Transporteur C"]; nouv["Transporteur C"] =
↳ 0
tirages = []
for _ in range(4000):
    t = pd.Series({k: rng.beta(1 + dec.loc[k, "sum"], 1 + dec.loc[k, "size"] - dec.loc[k, "sum"]) for
↳ k in dec.index})
    tirages.append(taux_global(t, nouv / nouv.sum()))
print("part de retards en décembre : actuelle", round(actuel_dec * 100, 1), "% | après transfert
↳ (médiane)", round(np.median(tirages) * 100, 1), "% | intervalle à 90 % :",
↳ round(np.percentile(tirages, 5) * 100, 1), "à", round(np.percentile(tirages, 95) * 100, 1), "%")
```

```
part de retards en décembre : actuelle 54.0 % | après transfert (médiane) 44.3 % | intervalle à 90 % :
↳ 41.6 à 47.1 %
```

Lecture. En décembre, 54,0 % des livraisons sont en retard. Si les 233 colis du transporteur C passaient par A (avec son taux de décembre), la part tomberait à environ 44 % (intervalle à 90 % : 42 à 47 %) : un gain de l'ordre de dix points, pas une disparition du problème, car A est lui-même en retard 38 % du temps en décembre.

Les coûts sont des **hypothèses** à valider avec la logistique : un colis par le transporteur A coûte 0,60 € de plus que par C, et un retard coûte 4 € (assistance, remboursements partiels). Le transfert est-il rentable ?

```
n_dec = len(a25[a25["mois"] == 12]); n_c = int(dec.loc["Transporteur C", "size"])
surcout = n_c * 0.60
retards_evites = (actuel_dec - np.median(tirages)) * n_dec
print("colis transférés en décembre :", n_c, "| surcoût :", round(surcout), "€ | retards évités :",
↳ round(retards_evites), "| gain à 4 € le retard :", round(retards_evites * 4), "€")
print("seuil : le transfert est rentable si un retard coûte plus de", round(surcout / retards_evites,
↳ 2), "€")
```

```
colis transférés en décembre : 233 | surcoût : 140 € | retards évités : 109 | gain à 4 € le retard :
↳ 435 €
seuil : le transfert est rentable si un retard coûte plus de 1.28 €
```

Lecture. Sous ces hypothèses, le transfert évite environ 109 retards pour 140 € de surcoût : il est rentable dès qu'un retard coûte plus de 1,28 €, ce qui est probable (un seul remboursement partiel

l'atteint). Le chiffre à discuter est donc **le coût réel d'un retard**.

P.7 Étape 6 : le récit en cinq diapositives

Le récit suit la structure du livre (4.1 et 4.3) : **réponse d'abord**, trois preuves, une recommandation, la décision demandée. On en fait un **storyboard** : les cinq titres-conclusions et une figure par diapositive, dessiné avec matplotlib (aucun logiciel de présentation n'est exécuté).

```
diapos = [("1. Notre demande", "Décider avant décembre : réduire la part du transporteur C"),
          ("2. Le constat", "Un colis sur quatre arrive en retard en 2025, plus de la moitié en
          ↳ décembre"),
          ("3. La cause", "Le transporteur C est en retard une fois sur deux, A une fois sur sept"),
          ("4. Le scénario", "Transférer C vers A ramènerait les retards de décembre de 54 % à environ
          ↳ 44 %"),
          ("5. La décision", "Valider un essai de transfert en novembre, mesuré chaque semaine")]
for titre, phrase in diapos:
    print(f"{titre:18s} {phrase}")
```

1. Notre demande	Décider avant décembre : réduire la part du transporteur C
2. Le constat	Un colis sur quatre arrive en retard en 2025, plus de la moitié en décembre
3. La cause	Le transporteur C est en retard une fois sur deux, A une fois sur sept
4. Le scénario	Transférer C vers A ramènerait les retards de décembre de 54 % à environ 44 %
5. La décision	Valider un essai de transfert en novembre, mesuré chaque semaine

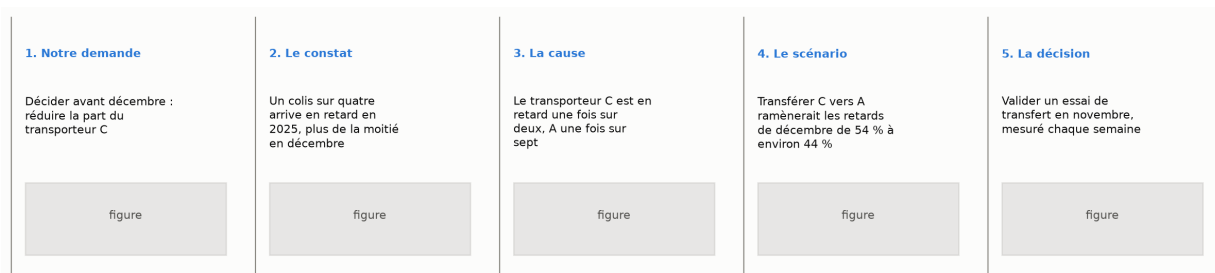


Figure 26: Storyboard de la présentation en cinq diapositives : un titre-conclusion et une figure par diapositive. Maquette dessinée avec matplotlib, pas une capture d'un logiciel de présentation.

P.8 Étape 7 : le message et les questions attendues

On écrit le message de cinq lignes à **partir des calculs**, pour que le texte et la figure disent la même chose.

```
fr = lambda v, nd=0: f"{v:,.{nd}f}".replace(","," ").replace(".",",")
print(f"1) En 2025, {fr((a25['retard'].mean() * 100)} % des livraisons sont arrivées en retard, et
↳ {fr(a25[a25['mois'] == 12]['retard'].mean() * 100)} % en décembre.")
print(f"2) Le transporteur C est en retard {fr(par_transp.loc['Transporteur C', 'taux_retard'] *
↳ 100)} % du temps, contre {fr(par_transp.loc['Transporteur A', 'taux_retard'] * 100)} % pour A.")
print(f"3) Transférer ses colis vers A ramènerait les retards de décembre de {fr(actuel_dec * 100)} %
↳ à environ {fr(np.median(tirages) * 100)} % (intervalle à 90 % : {fr(np.percentile(tirages, 5) *
↳ 100)} à {fr(np.percentile(tirages, 95) * 100)} %).")
print(f"4) Le surcoût estimé est de {fr(surcout)} € pour décembre ; il est compensé si un retard coûte
↳ plus de {fr(surcout / retards_evites, 2)} €.")
print("5) Décision demandée : autoriser un essai en novembre, avec un suivi hebdomadaire de la part de
↳ livraisons à l'heure.")
```

1) En 2025, 27 % des livraisons sont arrivées en retard, et 54 % en décembre.
2) Le transporteur C est en retard 52 % du temps, contre 15 % pour A.
3) Transférer ses colis vers A ramènerait les retards de décembre de 54 % à environ 44 % (intervalle à ↳ 90 % : 42 à 47 %).

- 4) Le surcoût estimé est de 140 € pour décembre ; il est compensé si un retard coûte plus de 1,28 €.
 5) Décision demandée : autoriser un essai en novembre, avec un suivi hebdomadaire de la part de
 ↪ livraisons à l'heure.

Les questions que l'on attendra (livre, 5.2) et les réponses que l'on prépare :

Question probable	Réponse préparée
« Et si le transporteur A ne suit pas ? »	C'est l'hypothèse faible du scénario : on la teste par un essai progressif (20 %, puis 50 % du volume de C), avec la courbe hebdomadaire comme alerte.
« Pourquoi ne pas simplement en changer ? »	Le transporteur B est lui aussi en retard une fois sur quatre ; changer de transporteur ne dit pas lequel est meilleur : les données, si.
« Combien ça coûte ? »	Un surcoût estimé de 140 € pour décembre (233 colis × 0,60 €), sous hypothèses à confirmer avec la logistique ; le transfert se paie dès qu'un retard coûte plus de 1,28 €.
« Êtes-vous sûr ? »	De la direction , oui (écart très net, intervalles qui ne se recouvrent pas) ; de l' ampleur , moins : d'où l'intervalle et l'essai.

P.9 Étape 8 : la critique, et les limites

On relit en se mettant à la place de la gérante : que ne dit pas ce travail ? (livre, 5.2).

- **Hypothèse de capacité.** Le scénario suppose que A traite 20 % de colis en plus avec la même qualité ; si A est déjà saturé en décembre, le gain disparaît.
- **Coûts supposés.** Les 0,60 € et les 4 € sont des ordres de grandeur ; le seuil de rentabilité (le coût d'un retard à partir duquel le transfert paie) est le chiffre à discuter.
- **Causalité.** Les transporteurs n'ont pas reçu les mêmes colis (destinations, tailles) : une partie de l'écart peut venir du mélange de colis, pas du transporteur (volume III, 11.1).
- **Satisfaction et retours non mesurés.** On suppose que moins de retards est bon pour la clientèle ; le lien avec les retours ou la fidélité n'est pas démontré ici.
- **Données simulées.** La vérité (un transporteur plus lent, un effet de décembre) est programmée ; la réalité est plus bruyante.

À retenir. Un livrable de visualisation et de communication se juge à ce qu'il **permet de décider** : le bon graphique, le bon texte et le bon niveau de confiance, **ensemble**.

P.10 Variante : la même chose pour les promotions

Le même chemin (question, indicateur, graphique, scénario, récit) s'applique à la décision sur les promotions du volume III. Voici le **graphique de décision** : l'effet sur les commandes est positif, l'effet sur la marge est négatif, et l'on montre les **deux** avec leur incertitude.

```
jours = pd.read_csv("donnees/jours_exploitation.csv", parse_dates=["date"])
cmd = pd.read_csv("donnees/commandes.csv"); lig = pd.read_csv("donnees/lignes_commande.csv"); prod =
↪ pd.read_csv("donnees/produits.csv")
x = lig.merge(cmd[["id_commande", "date_commande"]], on="id_commande").merge(prod[["id_produit",
↪ "cout_achat"]], on="id_produit")
x["marge"] = x["montant"] / 1.2 - x["quantite"] * x["cout_achat"]
m = x.groupby("date_commande")["marge"].sum().reset_index().rename(columns={"date_commande":
↪ "date"}); m["date"] = pd.to_datetime(m["date"])
```

```
jours = jours.merge(m, on="date")
pr, npr = jours[jours["promo_active"] == 1], jours[jours["promo_active"] == 0]
print("commandes par jour : promotion", round(pr["nb_commandes"].mean(), 1), "| hors promotion",
      ↪ round(npr["nb_commandes"].mean(), 1))
print("marge par jour : promotion", round(pr["marge"].mean()), "€ | hors promotion",
      ↪ round(npr["marge"].mean()), "€")
```

```
commandes par jour : promotion 35.4 | hors promotion 32.8
marge par jour : promotion 836 € | hors promotion 1054 €
```

Auto-évaluation

Mode d'emploi. Répondez à voix haute ou par écrit **avant** de lire le corrigé, en une ou deux phrases. Les sections à relire sont indiquées dans le corrigé. Trente-cinq bonnes réponses sur quarante signalent un volume bien assimilé ; les questions des sections facultatives (⊕ 1.3, 1.4, 2.4, 2.5, 3.3, 3.4, 4.3, 4.4, 5.3, 5.4) comptent si vous les avez lues.

Principes de visualisation (chapitre 1)

1. Quel type de graphique pour chacune de ces questions : « combien par canal ? », « comment évolue le chiffre d'affaires ? », « les gros paniers sont-ils rares ? », « le prix influence-t-il les retours ? », « où perd-on des visiteurs ? »
2. Classez par précision de lecture décroissante : l'aire, la position sur un axe commun, la couleur, la longueur d'une barre, l'angle.
3. Un graphique en barres d'axe tronqué démarre à 90 pour comparer 100 et 108. De combien la barre de 108 paraît-elle plus grande ? Quel est l'écart réel ?
4. Réécrivez le titre « Évolution du chiffre d'affaires par mois » en un titre qui énonce la conclusion (en supposant que le chiffre d'affaires de décembre est 1,6 fois celui de janvier).
5. Un texte gris (#767676) sur fond blanc : quel est le rapport de contraste, et atteint-il le seuil usuel pour du texte ?
6. Pourquoi ne faut-il jamais coder une information par la couleur seule ? Que fait-on à la place ?
7. Quels sont les deux défauts principaux d'un graphique à deux axes verticaux ?
8. Un camembert compte neuf parts. Quelle forme de graphique le remplace avantageusement, et pourquoi ?
9. Comment montrer en une figure le paradoxe de Simpson (un groupe gagne dans chaque sous-groupe mais perd au total) ?
10. À quoi sert une liste de contrôle avant de publier un graphique ? Citez quatre points.

Tableaux de bord (chapitre 2)

11. Dans un modèle en étoile pour la boutique, quelle table est la table de faits, et quelles tables sont des dimensions ?
12. Quelle différence entre une colonne calculée et une mesure ?
13. Que fait une mesure « chiffre d'affaires de la même période l'an dernier » ? Comment la reproduit-on en pandas ?
14. En quoi consiste le test des cinq secondes, et que doit-il montrer ?
15. Quels sont les trois niveaux d'un tableau de bord, et que contient chacun ?
16. Qu'est-ce que la sécurité au niveau des lignes, et comment la reproduit-on en pandas ?

Visualisation avec Python (chapitre 3)

17. Dans matplotlib, quelle est la différence entre Figure et Axes ?
18. Pourquoi écrit-on une figure comme une fonction qui retourne la figure ?
19. Que représente la bande autour d'une courbe dans un graphique seaborn par défaut ?
20. Pour un rapport imprimé, une figure plotly interactive est-elle un bon choix ? Pourquoi ?
21. Résumez en une phrase le modèle d'exécution de Streamlit, de Dash et de Shiny.
22. Dans une carte à cercles proportionnels, une ville réalise quatre fois plus de ventes qu'une autre. Dans quel rapport faut-il choisir les rayons ?
23. Pourquoi une carte colorée par ville ou par région trompe-t-elle sur les grandes zones, et que fait-on ?
24. Pourquoi toute carte plane déforme-t-elle quelque chose ?

Storytelling et rapports (chapitre 4)

25. Qu'est-ce que la pyramide de Minto, et en quoi diffère-t-elle d'un récit chronologique ?
26. Réécrivez ce titre : « Résultats de l'analyse des promotions » (le résultat : plus de commandes, moins de marge).
27. Le chiffre d'affaires progresse de 71 % entre octobre et décembre 2024, mais de 12 % de décembre 2023 à décembre 2024. Quel est le danger de présenter le premier chiffre ?
28. Citez les sections d'un rapport d'analyse, dans l'ordre.
29. La marge par commande passe de 32,08 € à 23,62 €. Quelle variation relative donne-t-on, et comment formule-t-on 17 884 € de marge perdue dans un résumé ?
30. Qu'est-ce qu'un résumé de direction en cinq lignes ?
31. Qu'est-ce qu'un « texte qui sait se taire » dans un rapport automatique, et quels contrôles faire avant l'envoi ?
32. Pourquoi un rapport hebdomadaire automatique qui compare à la semaine précédente risque-t-il de mal conclure ? Quelle comparaison préfère-t-on ?

Présenter à des non-techniciens (chapitre 5)

33. Quelles trois questions se pose-t-on avant de préparer une présentation ?
34. Traduisez « l'effet est significatif au seuil de 5 % » en une phrase pour la gérante.
35. Un taux passe de 6 % à 8 %. Quelle est la variation en points et en pour cent ? Que dit-on à un public non technique ?
36. Donnez la structure d'une présentation de dix minutes.
37. Comment dire une incertitude sans perdre la salle ? Donnez un exemple pour un effet estimé à +19 % (intervalle de 13 à 25 %).
38. Vous découvrez en séance une erreur dans un de vos chiffres. Que faites-vous ?
39. Que contient une fiche de cadrage ?
40. Deux collègues citent deux chiffres d'affaires différents. Quelle démarche suivez-vous ?

Corrigés des questions

Pour les questions numériques, **calculons** plutôt que de nous fier à la mémoire.

```
import numpy as np

print("Q3 hauteur apparente :", round((108 - 90) / (100 - 90), 2), "fois | écart réel :", round(108 /
↳ 100, 2), "fois")
def luminance(h):
    c = [int(h[i:i + 2], 16) / 255 for i in (1, 3, 5)]
```

```

c = [x / 12.92 if x <= 0.03928 else ((x + 0.055) / 1.055) ** 2.4 for x in c]
return 0.2126 * c[0] + 0.7152 * c[1] + 0.0722 * c[2]
la, lb = luminance("#ffffff"), luminance("#767676")
print("Q5 contraste de #767676 sur blanc :", round((la + 0.05) / (lb + 0.05), 2))
print("Q22 rapport des rayons pour des surfaces dans un rapport de 4 :", round(np.sqrt(4), 1))
print("Q29 variation relative de la marge par commande :", round((23.62 / 32.08 - 1) * 100, 1), "%")
print("Q35 variation :", 8 - 6, "points,", round((8 / 6 - 1) * 100, 1), "%")

```

```

Q3 hauteur apparente : 1.8 fois | écart réel : 1.08 fois
Q5 contraste de #767676 sur blanc : 4.54
Q22 rapport des rayons pour des surfaces dans un rapport de 4 : 2.0
Q29 variation relative de la marge par commande : -26.4 %
Q35 variation : 2 points, 33.3 %

```

1. Comparer des catégories : **barres** ; évolution : **courbe** ; distribution (paniers) : **histogramme** ou boîte ; relation entre deux variables numériques : **nuage de points** (avec une droite ou une courbe lissée) ; perte de visiteurs étape par étape : **entonnoir** (barres horizontales décroissantes) (1.1.1, 1.1.4).

2. Du plus précis au moins précis : **position sur un axe commun, longueur, angle, aire, couleur** (teinte ou intensité). C'est pourquoi les barres et les points l'emportent sur les camemberts et les bulles (1.1.2).

3. La barre de 108 paraît **1,8 fois** plus haute que celle de 100, alors que l'écart réel est de **8 %** (code ci-dessus) : un axe qui ne part pas de zéro exagère les écarts (1.4.1).

4. Par exemple : « Le chiffre d'affaires de décembre est 1,6 fois celui de janvier : la saison explique l'essentiel de la hausse ». Un bon titre énonce la **conclusion** que le lecteur doit retenir (1.2.3).

5. Le rapport de contraste est de **4,54** (code ci-dessus) : il atteint de justesse le seuil usuel de 4,5 pour du texte courant. Le seuil pour des éléments graphiques est de 3 (1.3.4).

6. Parce que certaines personnes distinguent mal les couleurs (environ un homme sur douze), et parce qu'un document peut être imprimé en noir et blanc. On double la couleur par une **autre marque** : étiquette, forme, motif, position, ou luminosité différente (1.3.3, 1.3.2).

7. (1) Les deux échelles sont **arbitraires** : en les étirant, on fait croiser ou coller n'importe quelles courbes ; (2) le lecteur ne sait pas à quel axe appartient quelle courbe, et suggère une **corrélation** là où il n'y en a pas forcément (1.4.4).

8. Un **diagramme en barres ordonnées**. Neuf parts voisines se comparent mal par l'angle et l'aire, alors que la longueur sur un axe commun se lit précisément et se trie (1.4.8, 1.1.5).

9. Par un **graphique par sous-groupes** (une couleur par sous-groupe, ou des petits multiples) à côté de la moyenne globale : on voit que, dans chaque groupe, l'effet va dans un sens, et que le mélange des effectifs inverse la tendance globale (1.4.9).

10. Elle évite les oublis : axe qui part de zéro (ou signalé), unités, source, période, titre qui conclut, couleur non porteuse à elle seule, effectifs affichés, échelle log signalée (1.4.11).

11. La table de **faits** est celle des **lignes de commande** (une ligne = un événement mesuré : quantité, montant). Les **dimensions** sont les produits, les clients, les dates, les canaux (2.1.2).

12. Une **colonne calculée** est évaluée **ligne par ligne** à l'actualisation des données et stockée ; une **mesure** est évaluée **à la demande**, dans le contexte du visuel (filtres, regroupements). Le chiffre d'affaires total est une mesure ; le montant hors taxe d'une ligne, une colonne calculée (2.1.3).

13. Elle calcule le chiffre d'affaires de la même période décalée d'un an, dans le contexte de filtres courant. En pandas : on agrège par mois, on décale d'un an (`shift(12)`) sur des mois complets ou une jointure sur le mois de l'an précédent), puis on compare (2.4.1).

14. On montre le tableau de bord pendant cinq secondes à une personne qui ne l'a jamais vu, puis on le cache et on lui demande ce qu'elle en retient : le **message principal** doit s'en dégager sans explication (2.3.4).
15. **Vue d'ensemble** (les chiffres clés et leur tendance, lisibles en cinq secondes), **analyse** (comparer, ventiler, repérer d'où vient l'écart), **détail** (les lignes pour vérifier ou agir) (2.3.2).
16. Chaque utilisateur ne voit que **les lignes auxquelles il a droit** (par exemple sa région). On le reproduit en filtrant la table avec une règle qui dépend de l'utilisateur, avant tout calcul (2.4.3).
17. La Figure est la **page entière** (taille, résolution, titre global) ; un Axes est un **graphique** dans cette page (axes, courbes, étiquettes). Une figure peut contenir plusieurs Axes (3.1.1).
18. Pour la **reproductibilité** : la fonction prend les données en entrée et retourne la figure, on peut la rappeler avec d'autres données, la tester et éviter l'état caché d'un script (3.1.8).
19. Un **intervalle de confiance à 95 %** de la moyenne, estimé par rééchantillonnage ; il dépend du nombre d'observations et suppose qu'elles sont indépendantes, ce qui n'est pas toujours vrai pour des séries (3.1.7).
20. Pas à lui seul : le papier ne reproduit ni le survol ni le zoom ; la figure doit **se suffire** sous forme d'image fixe (titre, étiquettes, annotations). L'interactivité sert à explorer à l'écran (3.2.3).
21. **Streamlit** rejoue tout le script à chaque interaction ; **Dash** appelle des fonctions de rappel qui relient des entrées à des sorties ; **Shiny** construit un graphe réactif où seules les parties concernées se recalculent (3.3.2 à 3.3.4).
22. La **surface** doit être proportionnelle à la valeur, donc le rayon est proportionnel à la **racine carrée** : pour quatre fois plus de ventes, un rayon **deux fois** plus grand (code ci-dessus). Un rayon proportionnel exagérerait l'écart (3.4.2).
23. Les grandes zones occupent beaucoup de surface et attirent le regard quelle que soit leur population. On normalise (**par habitant**, par client), on ajoute des étiquettes, ou l'on remplace la carte par un **tableau** ou des barres si la géographie n'apporte rien (3.4.4, 3.4.5).
24. Parce qu'on ne peut pas aplatir une surface courbe sans déformer **les distances, les surfaces, les angles ou les formes** : on choisit la projection qui préserve ce qui compte pour la question (3.4.6).
25. On commence par la **réponse** (la conclusion et la recommandation), puis on donne les arguments qui la soutiennent, du plus important au moins important ; un récit chronologique raconte ce qu'on a fait, pas ce que le lecteur doit en retenir (4.1.3).
26. Par exemple : « Les promotions font vendre 19 % de commandes en plus, mais font perdre de la marge ». Le titre est la **conclusion**, pas le sujet (4.1.4).
27. C'est un choix de **période** qui exagère : octobre-décembre est la saison haute. Le chiffre de **décembre à décembre** (+12 %) neutralise la saison : présenter le premier serait une **cerise cueillie** qui trompe le lecteur (4.1.7).
28. Résumé (conclusion et recommandation), question, données, méthode, résultats, limites, recommandations, annexes (4.2.1).
29. Variation relative : $23,62/32,08 - 1 \approx -26,4 \%$ (code ci-dessus). Dans un résumé, on arrondit : « environ 18 000 € de marge perdue », avec un ordre de grandeur et une base de comparaison (« sur les 153 jours de promotion ») (4.2.3).
30. Cinq lignes qui donnent : la **décision** attendue, la **réponse**, les **deux ou trois preuves** les plus fortes, le **risque ou la limite**, et la **prochaine étape** (4.3.1).

31. C'est un texte généré qui **n'écrit rien** quand les données ne soutiennent pas de conclusion (variation dans le bruit, données périmées). Contrôles avant envoi : **fraîcheur** des données, **totaux** de contrôle, effectifs, valeurs absentes, cohérence avec la période précédente (4.4.3, 4.4.4).

32. Parce qu'une comparaison à la semaine précédente mélange **saison** et **tendance de fond** : une boutique en croissance annuelle voit surtout des hausses. On compare à la **même semaine de l'année précédente** (ou à une référence saisonnière) et l'on signale ce qui sort du bruit (4.4.7).

33. **Qui** est dans la salle, **ce qu'ils savent et attendent** (et leur temps), et **quelle décision** on veut leur voir prendre (5.1.2).

34. Par exemple : « Si les promotions ne changeaient rien, on n'observerait un écart aussi grand que dans moins d'un cas sur vingt : nous sommes donc assez sûrs qu'il y a un effet ». On évite de dire « il y a 95 % de chances que l'effet existe » (5.1.4).

35. 2 **points** de plus ; en valeur relative $8/6 - 1 \approx 33\%$ de plus (code ci-dessus). On dit « 6 % à 8 %, soit 2 points de plus » et l'on évite « +33 % » sans préciser la base, qui prête à confusion (5.1.5).

36. La réponse d'abord, **trois preuves**, la recommandation, la **décision demandée**, les prochaines étapes ; environ une minute par idée, du temps pour les questions (5.2.1, 5.2.2).

37. En donnant la **direction** avec certitude et l'**ampleur** avec une fourchette : « Les promotions ajoutent des commandes, de l'ordre de 19 %, entre 13 et 25 % ; nous sommes sûrs de la direction, moins de l'ampleur exacte » (5.2.3).

38. On la **dit tout de suite**, on mesure son effet sur la conclusion, on corrige ou on propose de revenir avec le chiffre vérifié, sans improviser ; cacher une erreur coûte bien plus cher que la reconnaître (5.2.7).

39. La **question** à laquelle on répond, la **décision** et son responsable, le **périmètre**, les **critères de succès**, les **délais**, les **données** disponibles et les limites, et la validation écrite des parties prenantes (5.3.5).

40. On ne tranche pas par autorité : on compare les **définitions** (périmètre, période, TVA incluse ou non, annulations), on retrouve l'**écart par étapes** (méthode de réconciliation du volume II), puis on fait **valider** une définition unique par les deux personnes (5.4.4).

Votre grille d'auto-évaluation

Pour chaque ligne : **je sais l'expliquer** / **je sais le faire** / **à revoir**.

Compétence	Où la retravailler
Choisir un graphique selon la question	1.1
Simplifier, annoter, titrer une figure	1.2
Choisir des couleurs accessibles	1.3
Repérer et éviter les graphiques trompeurs	1.4
Construire un modèle en étoile et des mesures	2.1, 2.2
Concevoir un tableau de bord pour un décideur	2.3
Connaître DAX, sécurité par lignes, autres outils	2.4, 2.5
Produire des figures matplotlib et seaborn de qualité	3.1
Utiliser plotly et savoir quand l'interactif aide	3.2
Écrire une application de tableau de bord	3.3

Compétence	Où la retravailler
Représenter des données sur une carte sans tromper	3.4
Structurer un récit de données	4.1
Rédiger un rapport clair	4.2
Écrire une synthèse, une page, une présentation	4.3
Automatiser un rapport récurrent	4.4
Adapter son message à son public	5.1
Présenter résultats et recommandations	5.2
Cadrer une demande et interroger les parties prenantes	5.3
Négocier, collaborer, rester intègre	5.4
Livrer un tableau de bord et une présentation pour une décision	Projet du volume